SKALA

AI Campus 생성형AI

6. RAG Pipeline 설계 및 구축

2026.09

배기주 / SK AX

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX



1. RAG 개요
2. RAG A to Z : 방법론에서 평가지표까지
3. RAG 확장 전략

[부록] RAG 평가

AI Campus 생성형AI

1. RAG 개요

- RAG 필요성 (Biz & Tech)
- RAG 적용 평가

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

1. RAG

용어의 이해

[영어사전]

- Retrieval : (정보의) 검색, 특정한 질문에 관련된 데이터를 찾는 것
- Augmented : 기존의 데이터나 정보를 보완하거나 확장하여 더 나은 결과를 도출하는 과정
- Generation : 주어진 데이터를 기반으로 새로운 내용을 만들어내는 과정

[IT용어사전]

- **사용자의 질문에 대해 외부 데이터베이스나 문서를 검색한 후, 이를 바탕으로 답변을 생성**
- Retrieval : 사용자의 질문을 기반으로 관련 문서를 데이터베이스에서 검색
- Augmented : 검색한 문서를 모델 입력(프롬프트)에 추가하여, 검색된 지식을 활용할 수 있도록 보완
- Generation : 사용자 질의 + 검색 문서로 자연어 생성 모델이 답변 생성

2. RAG 필요성

Key Point

비즈니스의 '정확한 답변' 요구를 충족시키기 위한 기술적 진화의 결과물

RAG 필요성

A 비즈니스 관점

"정확한 답변"을 위한 내부 지식 활용의 중요성

- 기업들은 방대한 내부 문서를 보유하고 있지만, LLM은 해당 데이터 기반으로 사전 학습 되지 않음
- 비즈니스 적용 위해 **기업 내부의 지식에 기반**한 시스템이 필요 ➔ RAG 핵심 가치

RAG는 사내 문서를 연결하여 목적에 맞는 정확한 답변을 생성하는 현실적인 방법

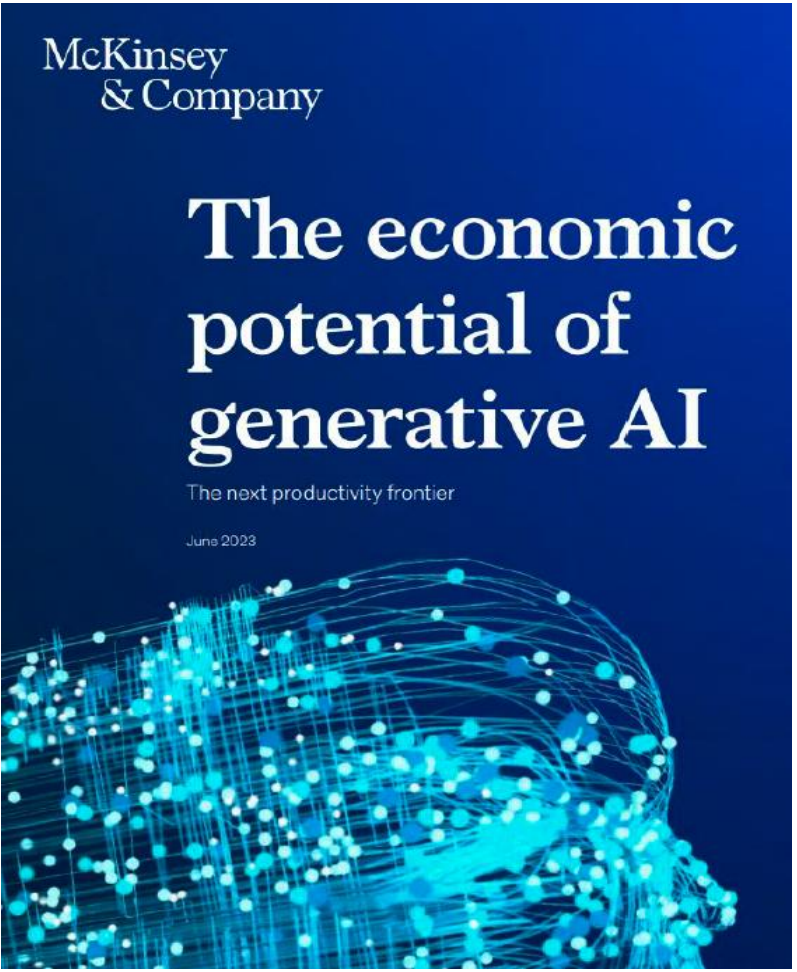
B 기술 관점

"정확도와 효율성"을 모두 잡기 위한 구조적 진화

- 기존 LLM은 모든 지식을 사전에 학습해야 하며, 새로운 정보가 생기면 모델 재학습이 필요한 구조
- RAG는 검색 기반 지식 보강 방식 통해 **검색 및 정답의 정확성 확보**
  - 검색 기반 증강 : 사전에 인코딩된 문서에서 관련 내용을 찾아와 응답 생성에 반영
  - 모듈화된 구조 : 검색기(Retriever)와 생성기(Generator) 라는 독립적인 구조
  - 효율적인 확장성 : 대규모 지식 업데이트가 아닌 **필요한 문서, 또는 검색 기술 개선 통해 대응** 가능

1. 비즈니스 관점

2023-06

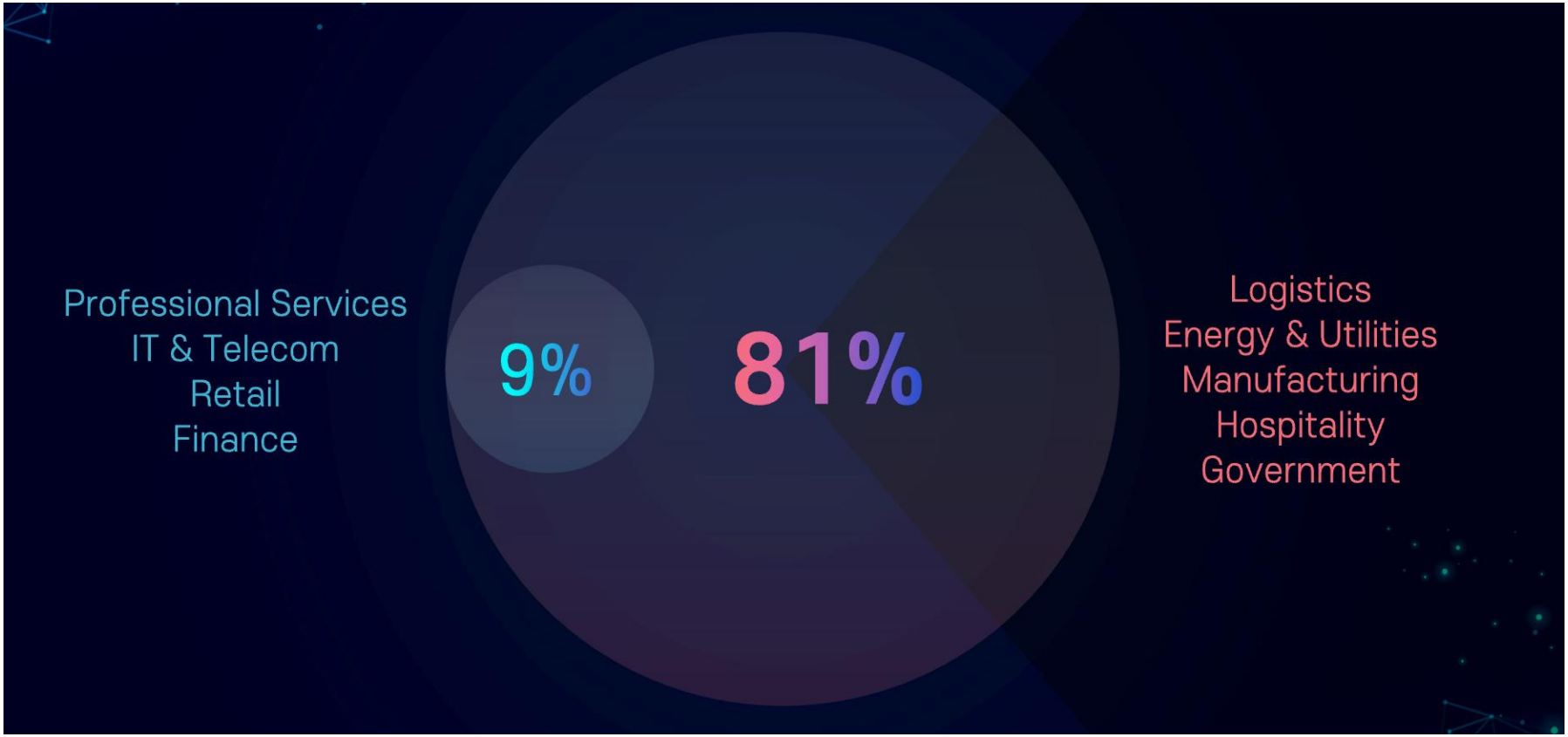


Across 63 use cases,
G.AI has the potential to generate $2.6T to $4.4T (5천6백조원) annually in value across industries.

1. 비즈니스 관점

Rate of G.AI adoption in the workplace in the USA 2023, by Industry

(Source : MIT Technology Review Insight poll, 2023)



1. 비즈니스 관점

G.AI 적용 시 일반적인 고민

보안 및 적합성

비용

부정확한 결과

1. 비즈니스 관점



1. 비즈니스 관점



2. 기술 관점

RAG 기법 진화

	Retrieval	Query	Structure
Flat Chunk	❑ **Naïve RAG** (2020) • 모델은 외부 지식을 동적으로 검색 및 활용하여 모델 파라미터에 기억을 제한하지 않음 • Pretrained Language Model 활용의 새로운 방식 제시	❑ Self-RAG (2023.10) • Self-Critiquing 통해 모델이 자체적으로 확인 • RAG 시스템을 단순 도구에서 자기 검토 능력을 갖춘 에이전트로 확장	❑ Modular RAG (2024.07) • 기존의 RAG 시스템을 레고 블록처럼 재구성 가능한 프레임워크로 변환 • 각 모듈을 독립적으로 설계하고 필요에 따라 교체하거나 결합할 수 있어, 다양한 작업 시나리오에 맞게 시스템을 최적화
	❑ Advanced RAG (2023.12) • 검색 품질 문제 해결을 위한 구조적 대응 방식 제시 • 단순 Top-K 검색을 넘어 문서 재정렬(Reranking), 의미 기반 벡터 검색(Dense Retrieval), 다단계 추론 기반 검색(Multi-hop Retrieval) 등 고도화된 기술 도입	❑ Corrective RAG (2024.01) • 모델이 스스로 검색 결과의 품질을 평가 (Correct, Incorrect, Ambiguous) • 품질이 낮을 경우 추가 검색(웹검색)으로 전환하여 생성의 정확도 개선 ❑ Adaptive-RAG (2024.03) • 질문의 유형과 난이도에 따라 적절한 검색 및 생성 전략을 스스로 조절 • 질의 복잡도를 분류하여 복잡도 따른 질의 처리 가능	❑ Agentic RAG (2025.01) • 에이전트 기반으로 실시간으로 검색 전략을 조정하여 최신 정보 활용하는 동적 검색 전략 관리 시스템 • 다중 에이전트 협업 통해 복잡한 작업 처리 능력 향상 및 실시간성 강화
Graph	❑ Graph RAG (2024.04) • Naïve RAG의 Flat Chunk면 충분하다는 가정 재점검. 지식그래프/온톨로지 관점 결합해 관계를 명시적으로 색인 • 인덱싱이 핵심이지만 Retrieval + Query까지 작용 • 관계 기반으로 Multi-hop 질의에서 유효. 실무에서는 벡터 검색과의 하이브리드 라우팅으로 사용		

2. 기술 관점

(2020) Proposal from Naïve RAG

"While pre-trained language models store vast knowledge in their parameters, they lack the ability to easily update, explain, or selectively access that knowledge. To overcome these limitations, we propose retrieval-augmented generation (RAG)—a hybrid model that combines a parametric generator with a non-parametric retriever, enabling scalable, updatable, and interpretable access to external knowledge."

파라미터 기반 LLM 한계 인식

- 대형 사전학습 언어모델은 지식(factual knowledge)을 모델 파라미터에 내부에 저장할 수 있음
- 그러나 이러한 구조는 근본적 한계를 가짐 :
  - 지식의 수정 및 확장이 어려움 → 모델 자체의 재학습 필요
  - 출처를 알 수 없어, 응답의 근거를 확인하거나 설명하는 것이 불가능
  - 환각 현상 → 존재하지 않는 정보도 생성 가능

지식 집약적 Task 취약

- 단순 자연어 처리 작업은 가능하지만,
- 외부 지식이 필수적인 타스크에서는 성능 하락 (eg, Domain-based QA, Fact verification 등)
- 도메인별 정확한 정보, 최신 지식 반영에는 매우 제한적임

"We explore a setting where both PARAMETRIC and NON-PARAMETRIC memory components are pre-trained and pre-loaded with extensive knowledge."

- **모델을 다시 학습하지 않아도 외부 문서 통해 지식 확장 가능**
- **검색 문서를 근거로 제시할 수 있는 설명력 확보**
- **지식 업데이트 시 문서 인덱스만 추가/교체하면 됨 (모델 재학습 불필요)**

Which One is Right for You?

(2023) Full Fine-tuning, PEFT, Prompt Engineering, and RAG



※ PEFT : Parametr-Efficient Fine-Tuning, 모델의 일부 파라미터만 조정 (LoRA, Prefix Tuning, IA^3 등)

Souce : https://www.linkedin.com/pulse/full-fine-tuning-peft-prompt-engineering-rag-which-one-right-you/

Enterprise

2024


CIO

홈 • 인공지능 • 칼럼 | 상용 RAG의 현주소와 도전 과제

By Paul Yu-Chun Chang, Bernhard Pflugfelder

칼럼 | 상용 RAG의 현주소와 도전 과제

뉴스
2024.07.25 • 7분

인공지능

RAG는 생성형 AI의 실용성을 강화하는 유력한 기술로 평가되지만,
실제 기업 환경에서는 데이터 정합성·정제 작업이 큰 도전 과제로 작용하고 있다.
특히, 비정형 문서와 정보 체계의 비표준성이 도입의 가장 큰 걸림돌이다.

상용 도입 효과

- 정확성 향상 : 정보 누락 없이 정확한 응답 생성 가능
- 사실 기반 응답 : 생성된 답변이 실제 문서에 근거를 두고 있음
- 사용자 맥락 반영 : 사용자 질문에 대해 보다 정밀하고 유의미한 응답 제공
- 도메인 특화 응용 가능 : 의료, 금융 등 산업에서도 활용 가능성 높음
- LLM 보완 기술 : RAG는 단독 기술 이라기 보다, LLM을 실용적으로 만들기 위한 현실적인 보완 수단

기술적 도전 과제

- 사내 문서 체계의 비일관성 : 많은 기업들이 문서가 정형화 되지 않고, 중복/노후/불필요한 문서가 혼재되어 있음
  ➔ 단순히 RAG 기술 도입이 아니라,
  데이터 관리 체계의 전반적인 재정비가 선행되어야 함

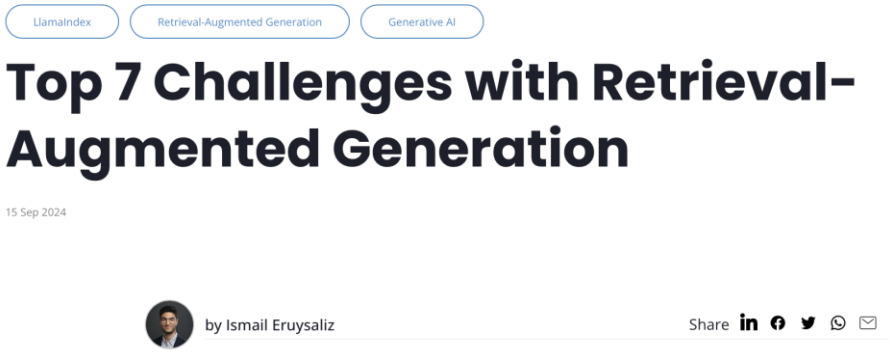
- 비정형 데이터 처리 난이도 : RAG 기반이 되는 벡터DB를 구성하기에 데이터 전처리 허들이 큼
  ➔ 데이터 정제, 정렬, 분류에 예상 보다 많은 리소스와 시간이 소요됨

Source : https://www.cio.com/article/3537629/칼럼-상용-rag의-현주소와-도전-과제.html

Enterprise

2024

VALPROVIA


LlamaIndex | Retrieval-Augmented Generation | Generative AI

Top 7 Challenges with Retrieval-Augmented Generation

15 Sep 2024

by Ismail Eruysaliz

Share

RAG 시스템은 정확한 응답 생성을 위해 효과적이지만,
구축 과정에서는 정보 누락, 출력 포맷 오류, PDF 추출 실패 등 다양한 실무적 문제가 발생한다. **따라서 RAG 도입 시에는 전처리부터 후처리까지 전반적인 시스템 품질 관리가 핵심 과제로 떠오르고 있다.**

Challenges	현상	제안하는 방안
1. 정보 누락	필요한 정보가 DB에 존재하지 않아 LLM이 부정확한 답 생성	프롬프트를 조정하여 "정보 없음"을 명시적으로 유도
2. 검색된 문맥에서 답 추출 실패	문서에 정답이 있지만 LLM이 올바르게 추출하지 못함	데이터 전처리: 중복 제거, 불필요 내용 제거, 포맷 정리 등
3. 출력 형식 오류	LLM이 지정된 포맷에 맞춰 출력하지 못함	추가적인 문서 전처리 및 Output parser 도입
4. 불완전한 응답	일부 정보만 포함된 불완전한 답변 생성	명시적 스키마 또는 output validator 사용 제안
5. 대규모 데이터 처리 부담	데이터 수집/적재량이 많으면 파이프라인 부담 (병목현상)	분산처리, 배치 설계, 사전 필터링 구조 등
6. 코드 실행의 보안 위협	LLM이 실행 가능한 코드를 반환할 때, 시스템 보안에 위협 가능	실행 제한 설정(권한, 환경 등), HITL 적용, 로그 모니터링
7. PDF 정보 추출 한계	PDF 내 복잡한 표, 이미지, 레이아웃 등으로 파싱 실패	다양한 파서 통한 실험, 사내 문서 구조 정규화 필요

Source : https://www.valprovia.com/en/blog/top-7-challenges-with-retrieval-augmented-generation

Enterprise – Use Case

2025

인공지능

글로벌 신용 평가 기업 '무디스'의 AI 에이전트로의 여정

무디스(Moody's)의 AI & Quantum Computing 매니징 디렉터(managing director) 세르지오 가고(Sergio Gago)를 만나 글로벌 신용평가 기관인 무디스의 생성형 AI에서부터 AI 에이전트 도입에 이르는 여정에 대해 들어봤다.

신동윤 수석 에디터 2025년 3월 29일

1. Denial – 새로운 기술에 대한 거부 및 부정
2. Experimentation – PoC 수행 통한 실험
3. Fear & Capacity – 위험 관리에 대한 두려움
4. Shy Chatbots – 초기 내부 적용 단계
5. First Consumer Use Case – 초기 고객 적용 단계
6. Next Generation : Agent as Co-Workers – 차세대 에이전트와의 협업

- 기술의 위험성, 환각현상, 데이터의 부정확성, 사실 검증과 결과를 뒷받침할 논리적 증거 부족 등
- 이해관계자들에게 시스템 안전성에 대한 확신을 줄 수 있도록 설득하는 과정
  - 데이터 정확성, 데이터 유출 방지
  - 규제 당국과 법률팀에게 지적재산권 보호가 충분히 이루어질 수 있도록
  - 격리된 환경(Sandbox)에서의 테스트 수행
- 이러함 위험을 극복하기 위해 RAG를 적극적으로 도입
  - RAG는 기업이 보유한 데이터를 활용해 보다 정확한 답변을 생성할 수 있도록 돕는 기술
  - "무디스 또한 RAG 기술을 광범위하게 활용하고 있으며, 앞으로 AI가 발전하는데 기본적인 역할을 하게 될 것이다"

Source : https://www.technologyreview.kr/글로벌-신용-평가-기업-무디스의-ai-에이젼트으로의/

LLM Wiki

2026-04-02

LLM Wiki

A pattern for building personal knowledge bases using LLMs.

This is an idea file, it is designed to be copy pasted to your own LLM Agent (e.g. OpenAI Codex, Claude Code, OpenCode / Pi, or etc.). Its goal is to communicate the high level idea, but your agent will build out the specifics in collaboration with you.

The core idea

Most people's experience with LLMs and documents looks like RAG: you upload a collection of files, the LLM retrieves relevant chunks at query time, and generates an answer. This works, but the LLM is rediscovering knowledge from scratch on every question. There's no accumulation. Ask a subtle question that requires synthesizing five documents, and the LLM has to find and piece together the relevant fragments every time. Nothing is built up. NotebookLM, ChatGPT file uploads, and most RAG systems work this way.

The idea here is different. Instead of just retrieving from raw documents at query time, the LLM **incrementally builds and maintains a persistent wiki** — a structured, interlinked collection of markdown files that sits between you and the raw sources. When you add a new source, the LLM doesn't just index it for later retrieval. It reads it, extracts the key information, and integrates it into the existing wiki — updating entity pages, revising topic summaries, noting where new data contradicts old claims, strengthening or challenging the evolving synthesis. The knowledge is compiled once and then *kept current*, not re-derived on every query.

This is the key difference: **the wiki is a persistent, compounding artifact.** The cross-references are already there. The contradictions have already been flagged. The synthesis already reflects everything you've read. The wiki keeps getting richer with every source you add and every question you ask.

LLM Wiki, 핵심 운영 사이클 :

- **Ingest** : 새로운 소스가 들어오면 LLM이 이를 분석하여 관련된 위키 페이지를 동시에 업데이트하고 인덱스 갱신
- **Query** : 사용자의 다중 복잡 질문에 답변하며, 생성된 유용한 분석 및 종합 결과는 새로운 위키 페이지로 자동 누적
- Lint : 시스템의 무결성 위해 LLM이 스스로 링크 오류를 찾고, 모순된 주장을 감지하며, 누락된 개념을 채워 넣는 보수 작업 수행

역할 분담 :

- Human : 유용한 원본 자료를 수집하고 질문하는 역할만
- LLM Agent : knowledge base를 요약/분류/업데이트 하는 유지보수 작업 전담

Source : https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f

RAG is Dead...??

2026-04

RAG Is Dead: Karpathy's LLM Wiki Replaces RAG in 2026.



Safoura jolfaei Follow 7 min read · Apr 12, 2026

- **RAG 한계점** : RAG는 질문이 들어올 때마다 매번 0에서부터 다시 검색을 시작하며 지식을 누적하거나 종합하지 못하는 한계가 있다고 비판
- **제로 인프라 지식 축적** : 임베딩 모델이나 Vector DB 같은 복잡한 인프라 없이도, LLM이 마크다운 파일 구조 안에서 지식을 복리로 축적 할 수 있음을 강조

RAG Isn't Dead. But Something Is. Karpathy's LLM Wiki Explained



Sathish Raju Follow 10 min read · Apr 17, 2026

- **자극적 프레임 반박** : Gist 공개 후 개발자 커뮤니티에 "RAG is Dead" 선언이 많았지만, 이는 다소 과장된 프레임이라고 지적
- **상호 보완적 관계** : LLM Wiki는 수천~수만 페이지가 넘는 대규모 기업 아카이브 검색 위한 RAG를 완전히 대체하는 것이 아님. 수백 개 문서 규모에서 깊이 있는 맥락 연결이 필요한 개인 및 팀의 지식 관리 레이어로써 RAG 위에 얹어지는 도구

Source : https://medium.com/ai-research-to-reality/rag-is-dead-karpathys-llm-wiki-replaces-rag-in-2026-ce93e44633db, https://medium.com/@sathishkraju/rag-isnt-dead-but-something-is-karpathy-s-llm-wiki-explained-512e3393801b

Enterprise

2026-04

The retrieval rebuild: Why hybrid retrieval intent tripled as enterprise RAG programs hit the scale wall

Sean Michael Kerner

1:06 pm, PT, April 29, 2026

- Vector DB 중심으로 RAG 확장 되었으나, 검색 정확도 이슈 → Hybrid = Dense + Sparse + Reranking
- Hybrid 도입 의향 : 약 30% - 한 분기 만에 3배 증가
  - **검색 최적화** 위한 투자 : 대규모 운영 안정성 확보 위한 벡터 레이어 도입 필요
  - **Production RAG** 부재 : 초기 구조가 스케일을 고려하지 못해 재구축 필요
  - **평가 기준** : Response Correctness, Retrieval Accuracy, Answer Relevance
    - Response Correctness : 그 답변이 실제로 맞는가
    - Retrieval Accuracy : 검색 단계에서 맞는 문서를 가져왔는가
    - Answer Relevance : 생성된 답변이 질문에 제대로 대응하는가

Source : https://venturebeat.com/data/the-retrieval-rebuild-why-hybrid-retrieval-intent-tripled-as-enterprise-rag-programs-hit-the-scale-wall

Is RAG Dead in 2026

YES

죽었다/대체된다

❑ **1세대 방식, Native RAG 겨냥**

- Anthropic, Google 등의 100만 토큰급 Long Context Window가 등장
- Agentic AI가 확산되며 LLM이 스스로 도구를 호출하여 그때그때 가져오는 방식 (Memory, Context 기반)
- 임베딩은 본질적으로 손실 압축이라 미묘한 차이를 구분하지 못하는 등의 구조적 한계

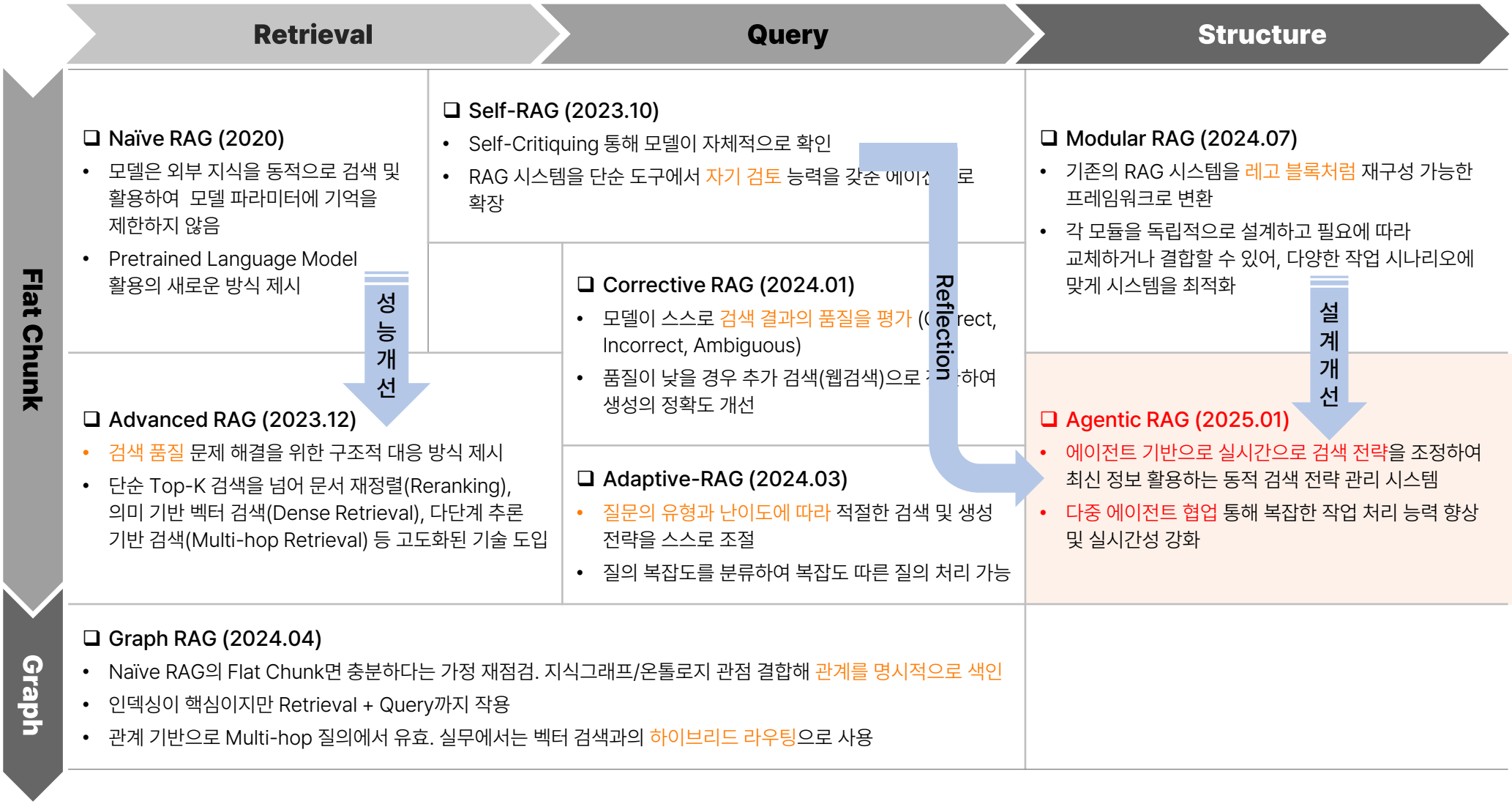
NO

안 죽었다/진화한다

❑ **1세대 이후 방식으로의 진화에 초점**

- 기업이 보유한 대부분의 데이터는 어떤 파운데이션 모델에도 학습되지 않음
- 매번 전체를 컨텍스트에 넣는 것은 비용, 지연시간 면에서 비현실적 – 검색은 여전히 유효
- Long Context는 검색의 필요성을 없애기 보다 비용 구조만 바꿀 뿐이라는 반박

RAG 확장 전략


Retrieval
Query
Structure
Flat Chunk
Graph
❑ Naïve RAG (2020)
• 모델은 외부 지식을 동적으로 검색 및 활용하여 모델 파라미터에 기억을 제한하지 않음
• Pretrained Language Model 활용의 새로운 방식 제시
성능개선
❑ Advanced RAG (2023.12)
• 검색 품질 문제 해결을 위한 구조적 대응 방식 제시
• 단순 Top-K 검색을 넘어 문서 재정렬(Reranking), 의미 기반 벡터 검색(Dense Retrieval), 다단계 추론 기반 검색(Multi-hop Retrieval) 등 고도화된 기술 도입
❑ Self-RAG (2023.10)
• Self-Critiquing 통해 모델이 자체적으로 확인
• RAG 시스템을 단순 도구에서 자기 검토 능력을 갖춘 에이전트로 확장
❑ Corrective RAG (2024.01)
• 모델이 스스로 검색 결과의 품질을 평가 (Correct, Incorrect, Ambiguous)
• 품질이 낮을 경우 추가 검색(웹검색)으로 지식 보강하여 생성의 정확도 개선
Reflection
❑ Adaptive-RAG (2024.03)
• 질문의 유형과 난이도에 따라 적절한 검색 및 생성 전략을 스스로 조절
• 질의 복잡도를 분류하여 복잡도 따른 질의 처리 가능
❑ Modular RAG (2024.07)
• 기존의 RAG 시스템을 레고 블록처럼 재구성 가능한 프레임워크로 변환
• 각 모듈을 독립적으로 설계하고 필요에 따라 교체하거나 결합할 수 있어, 다양한 작업 시나리오에 맞게 시스템을 최적화
설계개선
❑ Agentic RAG (2025.01)
• 에이전트 기반으로 실시간으로 검색 전략을 조정하여 최신 정보 활용하는 동적 검색 전략 관리 시스템
• 다중 에이전트 협업 통해 복잡한 작업 처리 능력 향상 및 실시간성 강화
❑ Graph RAG (2024.04)
• Naïve RAG의 Flat Chunk면 충분하다는 가정 재점검. 지식그래프/온톨로지 관점 결합해 관계를 명시적으로 색인
• 인덱싱이 핵심이지만 Retrieval + Query까지 작용
• 관계 기반으로 Multi-hop 질의에서 유효. 실무에서는 벡터 검색과의 하이브리드 라우팅으로 사용

1. RAG 개요

SUMMARY

SUMMARY

2. RAG 필요성

비즈니스의 '정확한 답변' 요구를 충족시키기 위한 기술적 진화의 결과물

RAG 필요성

A 비즈니스 관점

"정확한 답변"을 위한 내부 지식 활용의 중요성

- 기업들은 방대한 내부 문서를 보유하고 있지만, LLM은 해당 데이터 기반으로 사전 학습 되지 않음
- 비즈니스 적용 위해 **기업 내부의 지식에 기반**한 시스템이 필요 ➔ RAG 핵심 가치

RAG는 사내 문서를 연결하여 목적에 맞는 정확한 답변을 생성하는 현실적인 방법

B 기술 관점

"정확도와 효율성"을 모두 잡기 위한 구조적 진화

- 기존 LLM은 모든 지식을 사전에 학습해야 하며, 새로운 정보가 생기면 모델 재학습이 필요한 구조
- RAG는 검색 기반 지식 보강 방식 통해 **검색 및 정답의 정확성 확보**
  - 검색 기반 증강 : 사전에 인코딩된 문서에서 관련 내용을 찾아와 응답 생성에 반영
  - 모듈화된 구조 : 검색기(Retriever)와 생성기(Generator) 라는 독립적인 구조
  - 효율적인 확장성 : 대규모 지식 업데이트가 아닌 **필요한 문서, 또는 검색 기술 개선 통해 대응** 가능

SUMMARY

RAG 확장 전략

	Retrieval	Query	Structure
Flat Chunk	❑ Naïve RAG (2020) • 모델은 외부 지식을 동적으로 검색 및 활용하여 모델 파라미터에 기억을 제한하지 않음 • Pretrained Language Model 활용의 새로운 방식 제시	❑ Self-RAG (2023.10) • Self-Critiquing 통해 모델이 자체적으로 확인 • RAG 시스템을 단순 도구에서 자기 검토 능력을 갖춘 에이전트로 확장 ❑ Corrective RAG (2024.01) • 모델이 스스로 검색 결과의 품질을 평가 (Correct, Incorrect, Ambiguous) • 품질이 낮을 경우 추가 검색(웹검색)으로 재검색하여 생성의 정확도 개선	❑ Modular RAG (2024.07) • 기존의 RAG 시스템을 레고 블록처럼 재구성 가능한 프레임워크로 변환 • 각 모듈을 독립적으로 설계하고 필요에 따라 교체하거나 결합할 수 있어, 다양한 작업 시나리오에 맞게 시스템을 최적화
	❑ Advanced RAG (2023.12) • 검색 품질 문제 해결을 위한 구조적 대응 방식 제시 • 단순 Top-K 검색을 넘어 문서 재정렬(Reranking), 의미 기반 벡터 검색(Dense Retrieval), 다단계 추론 기반 검색(Multi-hop Retrieval) 등 고도화된 기술 도입	❑ Adaptive-RAG (2024.03) • 질문의 유형과 난이도에 따라 적절한 검색 및 생성 전략을 스스로 조절 • 질의 복잡도를 분류하여 복잡도 따른 질의 처리 가능	❑ Agentic RAG (2025.01) • 에이전트 기반으로 실시간으로 검색 전략을 조정하여 최신 정보 활용하는 동적 검색 전략 관리 시스템 • 다중 에이전트 협업 통해 복잡한 작업 처리 능력 향상 및 실시간성 강화
Graph	❑ Graph RAG (2024.04) • Naïve RAG의 Flat Chunk면 충분하다는 가정 재점검. 지식그래프/온톨로지 관점 결합해 관계를 명시적으로 색인 • 인덱싱이 핵심이지만 Retrieval + Query까지 작용 • 관계 기반으로 Multi-hop 질의에서 유효. 실무에서는 벡터 검색과의 하이브리드 라우팅으로 사용		

성능개선

Reflection

설계개선

AI Campus 생성형AI

2. RAG A to Z : 방법론에서 평가 지표까지

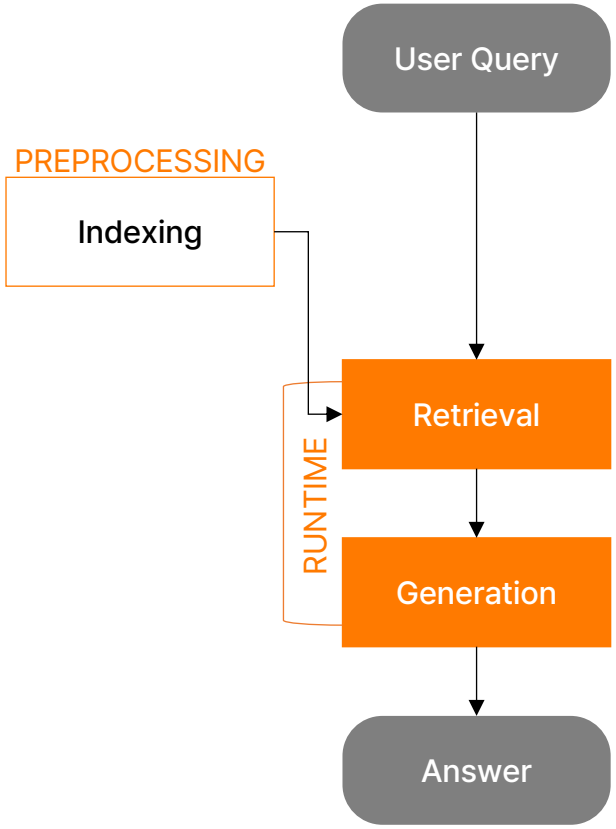
- RAG 방법론
- RAG 프로세스 : PREPROCESSING, RUNTIME
- RAG 평가

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

1. RAG 방법론

가장 기본적인 형태의 RAG 방법론



- 지식 베이스를 만드는 단계
- 원본 데이터 정제 → 텍스트 추출 → Chunk 분할 → 벡터 변환 (→ Vector DB 저장)

Pre-processing | Indexing

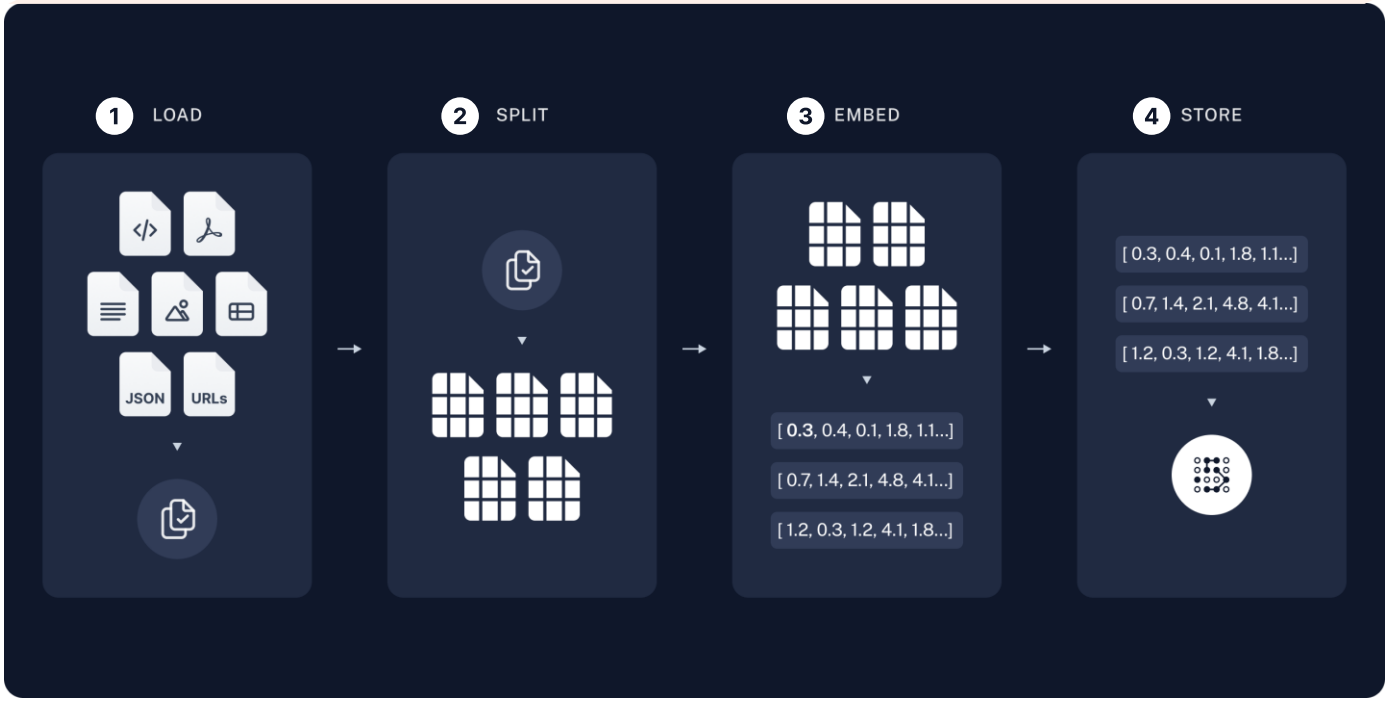
- 사용자 질문 기반으로 문서 검색
- 질문 벡터화 → 유사도 기반 상위K개의 청크 검색

- 검색된 K개의 청크와 사용자 질문을 결합하여 LLM에 Prompt로 전달
- LLM은 입력 받은 Context 기반으로 최종 응답 생성

2. RAG 프로세스

Key Point

PREPROCESSING RUNTIME



❶ Document Loader
: 필요 문서를 로드하고 초기 처리하는 과정.
마치 학생이 공부하기 전에 필요한 책들을
책장에서 골라오는 과정과 같음

❷ Text Splitter
: 로드된 문서를 처리 가능한 작은 단위로 분할

❸ Embedding
: 각 문서 또는 문서의 일부를 벡터 형태로
변환하여 문서의 의미를 수치화 함

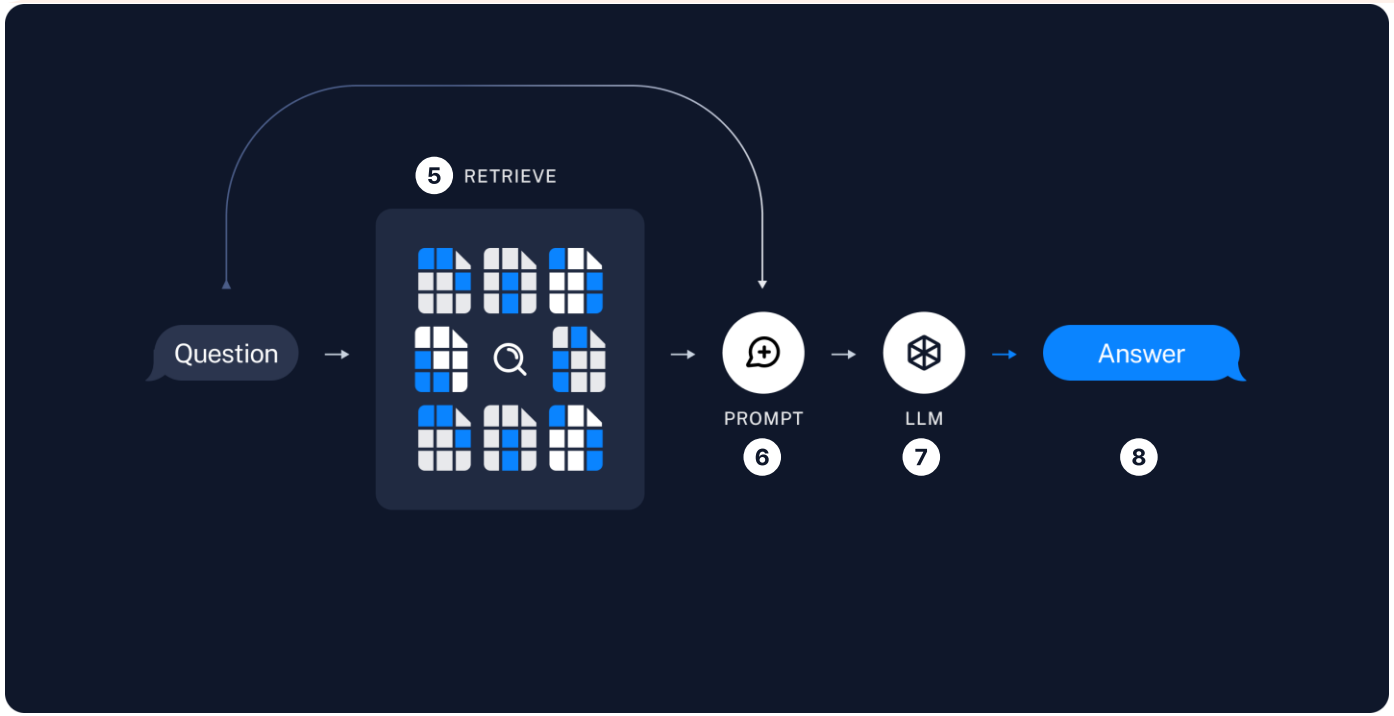
❹ Vector Database
: 임베딩된 벡터들을 DB에 저장

Source : https://velog.io/@jjlee6496/RAG란

2. RAG 프로세스

Key Point

PREPROCESSING | RUNTIME



❺ Retriever
: 질문이 주어지면, 이와 관련된 벡터를 벡터DB에서 검색함. 질문에 가장 잘 맞는 책의 챕터를 찾는 것과 유사함

❻ Prompt
: 검색된 정보를 바탕으로 언어 모델을 위한 질문을 구성함. 이는 정보를 바탕으로 어떻게 질문할지 결정하는 과정임

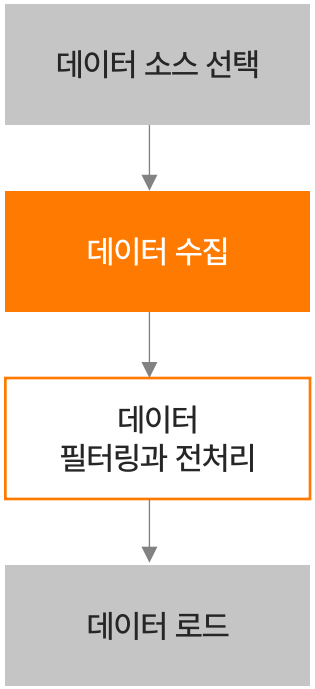
❼ LLM
: 구성된 프롬프트를 사용하여 언어 모델이 답변을 생성함

❽ Chain
: 앞선 모든 과정을 하나의 파이프라인으로 묶어주는 체인을 생성함

Source : https://velog.io/@jjlee6496/RAG란

2-(1). Document Loader

- RAG 시스템이 필요로 하는 데이터를 외부 소스로부터 효율적으로 수집하고 준비하는 단계
- LangChain의 다양한 로더를 활용하여 필요 데이터를 수집하고 정제함
- https://docs.langchain.com/oss/python/integrations/document_loaders



- 어떤 종류의 데이터가 필요한지, 그 데이터를 어디서 어떻게 수집할지 확인
- 데이터 소스는 내부 문서, DB, 웹사이트, API, 공개데이터셋 등 다양할 수 있음

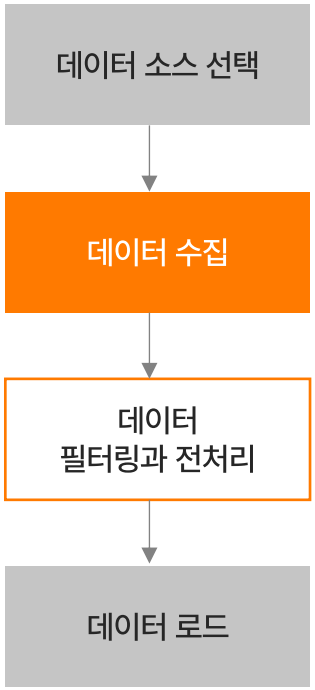
- 선택된 데이터 소스에서 필요한 데이터를 수집하는 과정
- API 호출, 웹 스크래핑, DB 쿼리 등을 포함할 수 있고, 필요 시 인증이나 접근 권한 설정이 추가적으로 요구될 수 있음
- (참고) LangChain Document Loader

- 수집된 데이터 중에서 RAG 시스템에 필요한 정보만을 추출하고 정제하는 과정
- 불필요한 포맷팅을 제거하거나 답변에 포함되지 말아야 하는 정보 제거

- 전처리된 데이터를 RAG 시스템 내부적으로 사용할 수 있는 형식으로 변환하고 로드
- 보통 메모리 내 데이터 구조로의 변환을 포함하고, 필요에 따라 DB나 파일시스템에 저장할 수도 있음

2-(1). Document Loader

- https://docs.langchain.com/oss/python/integrations/document_loaders




```
from langchain_community.document_loaders import PyMuPDFloader

loader = PyMuPDFloader("data/file.pdf")
docs = loader.load()
```

2-(1). Document Loader

Document Loader	Description	Package/API
PyPDF	Uses `pypdf` to load and parse PDFs	Package
Unstructured	Uses Unstructured's open source library to load PDFs	Package
Amazon Textract	Uses AWS API to load PDFs	API
MathPix	Uses MathPix to load PDFs	Package
PDFPlumber	Load PDF files using PDFPlumber	Package
PyPDFDirectry	Loa	
PyPDFium2	Loa	
PyMuPDF	Loa	
PyMuPDF4LLM	Loa Pyl	
PDFMiner	Loa	
Upstage Document Parse Loader	Loa Up	
Docling	Loa	

Document Loader	Description	Package/API
Web	Uses urllib and BeautifulSoup to load and parse HTML web pages	Package
Unstructured	Uses Unstructured to load and parse web pages	Package
RecursiveURL	Recursively scrapes all child links from a root URL	Package
Sitemap	Scrapes all pages on a given sitemap	Package
Firecrawl	API service that can be deployed locally, hosted version has free credits.	API
Docling	Uses Docling to load and parse web pages	Package
Hyperbrowser	Platform for running and scaling headless browsers, can be used to scrape/crawl any site	API
AgentQL	Web interaction and structured data extraction from any web page using an AgentQL query or a Natural Language prompt	API

- Document Loader 선택 시 고려사항 :

  - 텍스트를 원형 그대로 잘 가져오는가?
    - 한글 인코딩
    - 특수문자 처리

  - 메타데이터는 어디까지 처리 하는가?
    - page_content : 문서 내용
    - page : 페이지 번호
    - 표, 차트, 문서 내에서 속성 정보

  - 문서를 읽는 속도는 얼마나 빠른가?
    - 다량의 페이지를 포함하는 문서에서 처리 속도

Source : https://python.langchain.com/docs/integrations/document_loaders/

2-(2). Text Splitter

- 크고 복잡한 문서를 LLM이 받아들일 수 있는 효율적인 작은 규모의 조각으로 나누는 작업
- 나중에 사용자가 입력한 질문에 대하여 보다 효율적인 정보만 압축/선별하여 가져오기 위함
- https://docs.langchain.com/oss/python/integrations/splitters

문서 구조 파악

- 다양한 형식의 문서 내에서 문서 구조를 파악함
- 문서의 헤더, 푸터, 페이지번호, 섹션제목, 테이블 구조 등을 식별

단위 선정

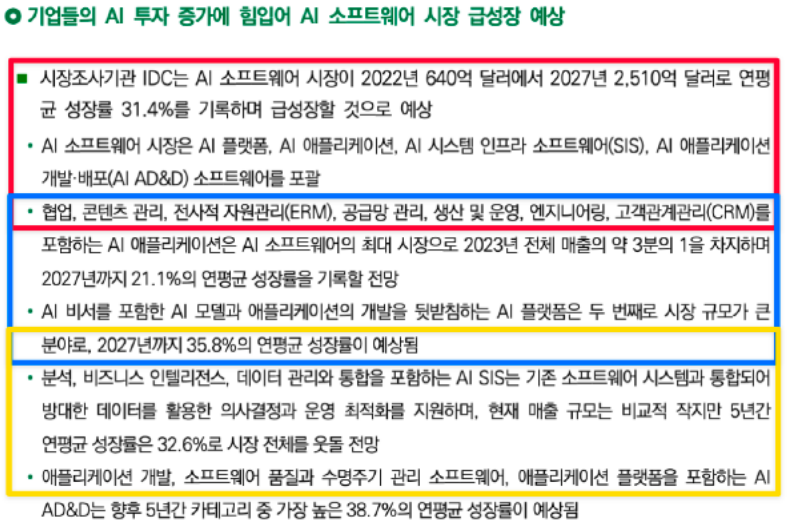
- 문서를 어떤 단위로 나눌지 결정 (페이지별, 섹션별, 문단별 등)
- 단위는 문서의 내용과 목적에 따라 정의하는 것이 필요

단위 크기 선정 (Chunk Size)

- 문서를 몇 개의 토큰 단위로 나눌 것인지 정함

Chunk Overlap

- 기계적으로 토큰 단위로 단위를 나누면 앞뒤 문맥을 파악하는 것이 어려움
- 분할된 끝 부분에서 맥락이 이어질 수 있도록 일부를 겹쳐서(overlap) 분할하는 것이 필요
- Trade-off : 문맥 보존 ↔ 저장량/중복검색 증가 – 보통 청크 크기의 10~20%

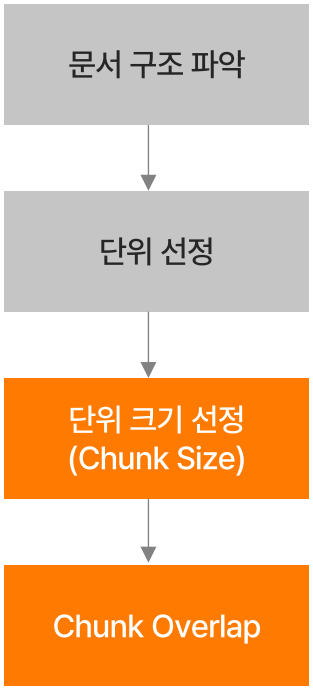


기업들의 AI 투자 증가에 힘입어 AI 소프트웨어 시장 급성장 예상

- 시장조사기관 IDC는 AI 소프트웨어 시장이 2022년 640억 달러에서 2027년 2,510억 달러로 연평균 성장률 31.4%를 기록하며 급성장할 것으로 예상
- AI 소프트웨어 시장은 AI 플랫폼, AI 애플리케이션, AI 시스템 인프라 소프트웨어(SIS), AI 애플리케이션 개발·배포(AI AD&D) 소프트웨어를 포괄
- 협업, 콘텐츠 관리, 전사적 자원관리(ERM), 공급망 관리, 생산 및 운영, 엔지니어링, 고객관계관리(CRM)를 포함하는 AI 애플리케이션은 AI 소프트웨어의 최대 시장으로 2023년 전체 매출의 약 3분의 1을 차지하며 2027년까지 21.1%의 연평균 성장률을 기록할 전망
- AI 비서를 포함한 AI 모델과 애플리케이션의 개발을 뒷받침하는 AI 플랫폼은 두 번째로 시장 규모가 큰 분야로, 2027년까지 35.8%의 연평균 성장률이 예상됨
- 분석, 비즈니스 인텔리전스, 데이터 관리와 통합을 포함하는 AI SIS는 기존 소프트웨어 시스템과 통합되어 방대한 데이터를 활용한 의사결정과 운영 최적화를 지원하며, 현재 매출 규모는 비교적 작지만 5년간 연평균 성장률은 32.6%로 시장 전체를 웃돌 전망
- 애플리케이션 개발, 소프트웨어 품질과 수명주기 관리 소프트웨어, 애플리케이션 플랫폼을 포함하는 AI AD&D는 향후 5년간 카테고리 중 가장 높은 38.7%의 연평균 성장률이 예상됨

2-(2). Text Splitter

- https://docs.langchain.com/oss/python/integrations/splitters




```
from langchain_community.document_loaders import PyMuPDFLoader

loader = PyMuPDFLoader("data/file.pdf")
docs = loader.load()
```

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=100)
splitters = text_splitter.split_documents(docs)
```

[참고] Token vs Chunk

Token

- 정의 : 문장을 단어 또는 서브워드로 쪼개어 나눈 조각 ➔ 단어 조각
- 예시 :
  - 단어 단위 토큰화 : “I love Orange” → [“I”, “love”, “Orange”]
  - Subword 토큰화 : ”unbelievable” → [“un”, “believ”, “able”]
- 특징 :
  - 모델 입력의 기본 단위
  - 모델 학습 시 토큰ID로 변환되어 사용됨
  - 텍스트 길이 측정 시 문장이 몇 토큰인지로 판단함 (GPT, BERT 등)

Chunk

- 정의 : 긴 텍스트를 일정한 크기의 토큰으로 나눈 블록 ➔ 토큰들의 집합, 여러 토큰을 하나의 청크로 구성
- 역할 : 모델이 한 번에 처리할 수 있는 길이를 초과하는 경우 여러 블록으로 분할
- 방식 :
  - 고정 길이 청크 : (e.g.) 512 토큰 씩 잘라서 분할
  - Sliding Window 방식 : 일부 겹치게 잘라서 문맥 유지
- 예시 :
  - 긴 기사글 1200 토큰 → Chunk1 (512 토큰) + Chunk2 (512 토큰) + Chunk3 (176 토큰)
  - 이 때, 각 청크 내부는 [Token1, Token2, … , TokenN] 형태로 구성

[참고] Chunking

텍스트를 조각낸 토큰을 모델이 감당할 수 있게 덩어리로 묶는 과정

"This is an example text for demonstration."

1. tokenizer : WordPiece tokenizer 적용하면
   → ["This", "is", "an", "example", "text", "for", "demonstration"]

2. Chunking
   : chunk_size = 4, chunk_overlap = 2

   ❑ Chunk1 : ["This", "is", "an", "example"]
   ❑ Chunk2 : ["an", "example", "text", "for"]
   ❑ Chunk3 : ["text", "for", "demonstration"]

TIP Tokenizer → ← Chunking

2가지 방식 모두 유효하며, 목적에 따라 전략적으로 선택

	Tokenizer → Chunking	Chunking → Tokenizer
프로세스	전체 문서를 토크나이즈 한 후, 토큰 기준으로 청크 분할	전체 문서를 문자/문장/단락 기준으로 청크 분할 후, 각 청크에 토크나이저 적용
목적	**모델 입력 제한 기준으로 분할** → 모델 처리 효율 중심	**의미 단위 기반으로 분할** → 문서 의미 구조 중심
예시	GPT, BERT 기반 문장 처리	검색 시스템, 문서 요약, RAG 등

예시:

"Deep learning is widely used. It powers many applications like speech recognition and image processing."

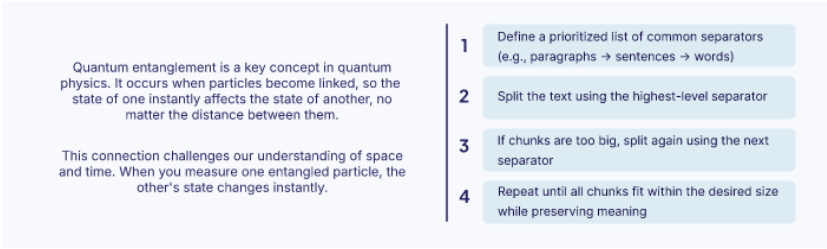
- Tokenizer → Chunking
  - 전체 문서 토큰화 → 토큰 수 기준으로 512 단위로 자름 (의미 단위는 무시 될 수 있음)

- Chunking → Tokenizer
  - "Deep Learning is widely used.", "It powers many applications ..." 로 먼저 나눈 후
  - 각각 토큰화 → 문장 단위 의미 유지

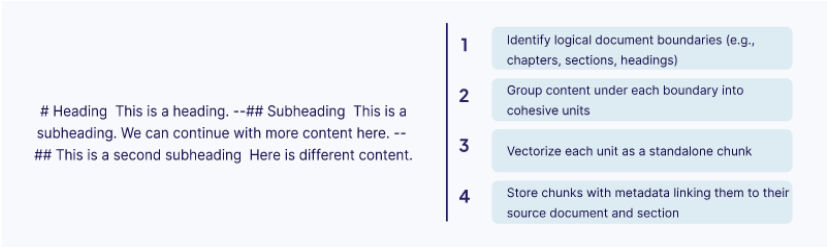
2-(2). Text Splitter

Chunk Strategy > A. Simple Chunking Technique

- **Fixed-Size Chunking** (or Token Chunking) :
  - 텍스트 전체를 미리 정해진 크기(토큰 개수 또는 문자수)로 분할
    - 장점 : 구현이 쉽고 빠름, 단순한 문서 처리에 적합
    - 단점 : 의미적 경계(문단, 문장 등)를 무시할 수 있음, 정보 손실이나 부정확성 발생
  - Chunk Overlap : 청크 끝부분 일부를 다음 청크 시작 부분에 포함



- **Recursive Chunking** :
  - 문서를 여러 수준의 구분자를 가진 계층적 방식으로 나누는 기법
  - 처음에는 큰 단위(문단, 단락)으로 나누고, 만약 그 청크가 설정한 최대 크기를 넘는다면 더 작은 단위(문장, 단어 등)로 반복하여 나누는 방식
    - 장점 : 문서 구조를 어느 정도 보존하면서 나눌 수 있음 . 의미 단위가 파괴될 가능성 적음
    - 단점 : 전체 처리 시간과 storage 요구량 증가

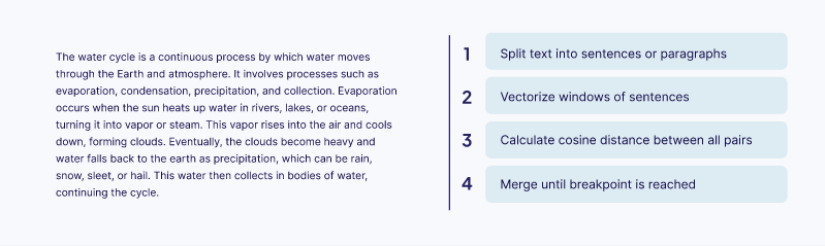


- Document-based Chunking :
  - 문서 전체를 하나의 큰 청크로 취급
  - 문서가 짧고 길이 균일할 때 적합 (공지, 메모, 요약 리포트, FAQ 등)
    - 장점 : 문맥 보존, 짧은 문서에 적합
    - 단점 : 정밀 검색 부족(세부 구절, 문장 단위 매칭 어려움)

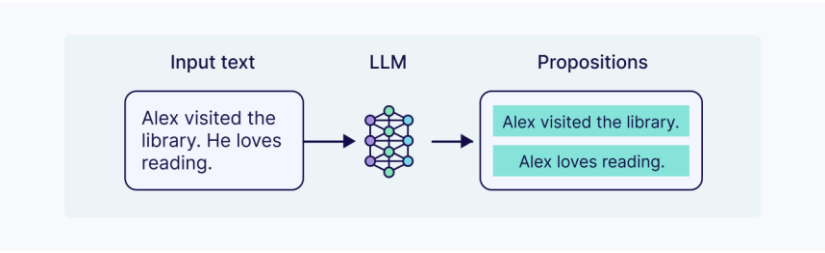
Source : https://weaviate.io/blog/chunking-strategies-for-rag?utm_source=linkedin&utm_medium=dw_social&utm_campaign=chunking&utm_content=268045841

2-(2). Text Splitter

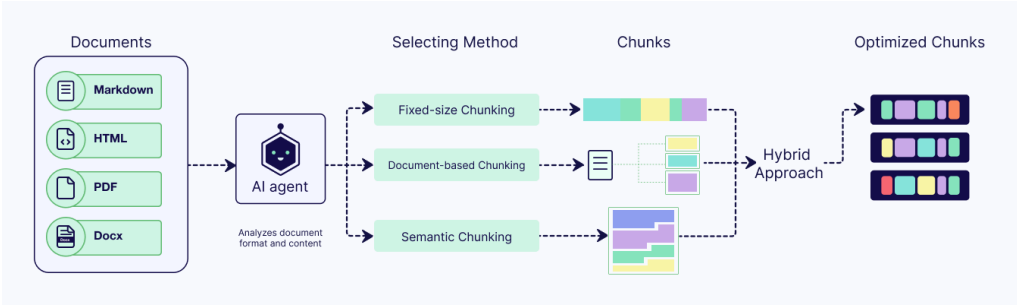
Chunk Strategy > B. Advanced Chunking Technique (1/2)



- **Semantic Chunking** :
  - 텍스트를 의미(Semantic) 단위에 따라 분할
  - 단순히 길이(토큰수, 문장수)가 아니라, 내용적 관련성이나 주제 전환을 기준으로 나뉨
    - 장점 : 문맥 보존, 검색 효율성↑, 정확성 향상
    - 단점 : 비용 증가, 불확실성(의미 구분이 항상 정확하지 않고, 경계가 잘못 잡힐 수 있음)



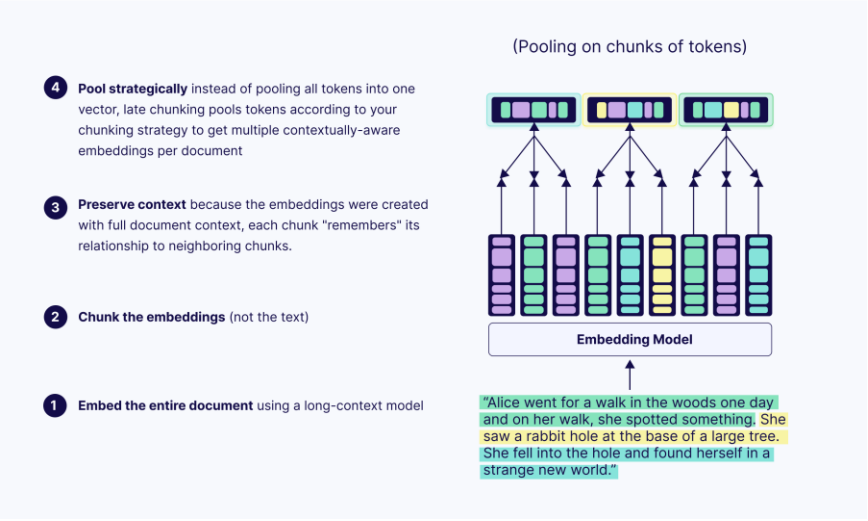
- LLM-based Chunking :
  - LLM 활용하여 텍스트를 의미 단위로 분할
    - 장점 : 가장 유연한 분할 가능, 다양한 도메인 대응
    - 단점 : 비용 부담, 규칙 기반(Fixed, Recursive) 보다 속도 저하, 일관성 한계(LLM)
  - 다양한 종류의 문서에 청크 전략 수립이 가능한 경우 → Agentic Chunking



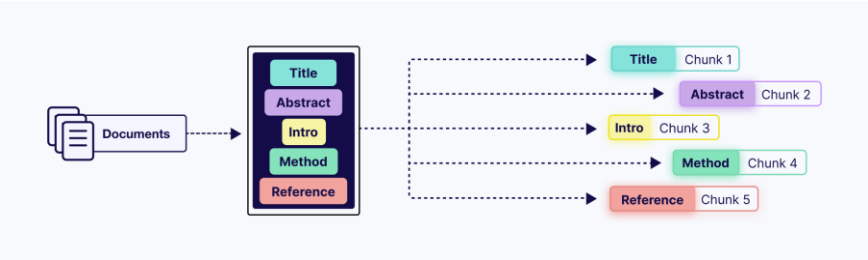
Source : https://weaviate.io/blog/chunking-strategies-for-rag?utm_source=linkedin&utm_medium=dw_social&utm_campaign=chunking&utm_content=268045841

2-(2). Text Splitter

Chunk Strategy > B. Advanced Chunking Technique (2/2)



- Late Chunking :
  - 텍스트를 사전에 잘라두지 않고, 원문 전체를 그대로 임베딩해 두었다가,
  - 사용자 질의가 들어오면 그 질문과 연관된 부분을 찾아 해당 시점에 chunking
    → 검색 이후(Chunk Late)에 필요한 부분만 분할하는 접근방식
    - 장점 : 문맥 손실 최소화, 유연한 검색, 문서 구조 보존
    - 단점 : 실시간 처리 비용↑, 구현 난이도 높음, 인덱싱 효율 저하
  - 정확성 > 속도가 더 중요한 도메인에서 검토 가능



- **Hierarchical Chunking** :
  - 문서에서 상위 청크(섹션)와 하위 청크(문단)를 동시에 생성하고, 검색 시 계층적으로 활용
    - 장점 : 문맥 유지 + 세밀한 검색, 복잡 문서 구조에 효과적
    - 단점 : Storage/계산량 증가, 어느 수준에서 분할할지 튜닝 필요
  - RAG의 정밀도와 재현율을 동시에 확보하고 싶을 때

Source : https://weaviate.io/blog/chunking-strategies-for-rag?utm_source=linkedin&utm_medium=dw_social&utm_campaign=chunking&utm_content=268045841

2-(2). Text Splitter

Chunk – 메타데이터 태깅

```
{
  "content" : "검색에서 찾은 청크 입니다. ...",
  "metadata" : {
      "chunk_id" : "doc123_0007",
      "document_id" : "doc123",
      "source" : "https://.../파일명.pdf",
      "file_name" : "파일명.pdf",
      "title" : "문서 제목",
      "page": 7,
      "section": "10.1",
      "section_title" : "섹션 제목",
      "updated_at": "YYYY-MM-DD",
      "access_tag" : ["all"],
      "document_type": "매뉴얼",
      ...
  }
}
```

※ 그 외 필요 항목은 프로젝트 내부 목적에 맞게 정의/추가 반영

TIP 데이터 전처리, 고려사항

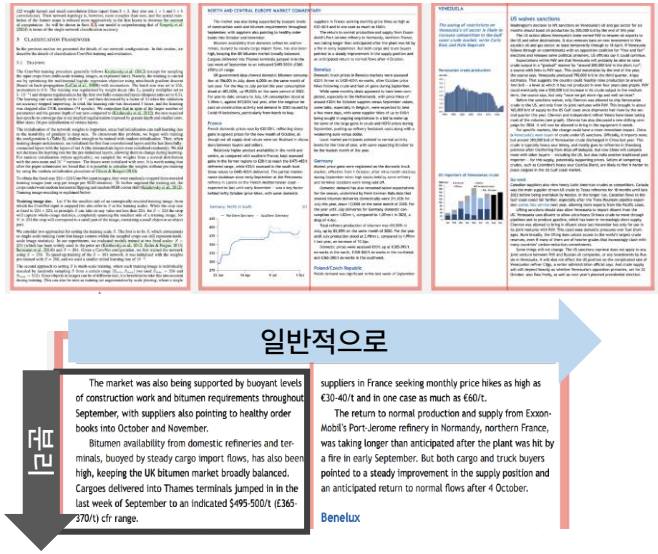
❑ Split & Chunking

- Baseline : 페이지 단위 → 결과 기반으로 분할 전략 확정
  - 표, 문단이 페이지 경계에 잘리는지 사전 확인
  - 페이지에 걸쳐 주요 키워드가 걸쳐 있는지 사전 확인
- Heading 기준으로 청킹 시에는 상위 섹션 경로를 메타데이터로 저장
  - 출처 추적, 검색 정확도 향상 효과
  - Heading이 불명확하거나 평면적 문서(스캔, 표지형)는 효과 제한적
- 표는 캡션까지 묶어서 처리. 각주는 분리

❑ 정제 (노이즈 제거)

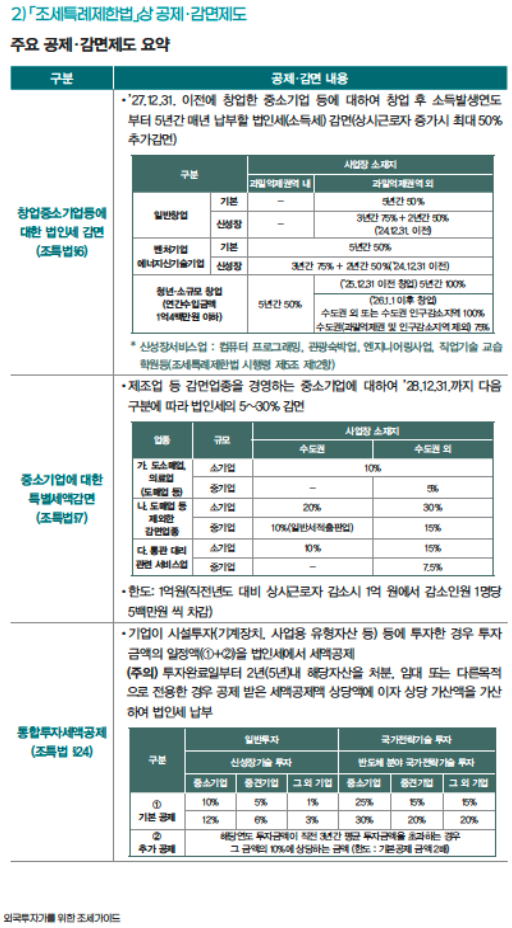
- 줄바꿈 하이픈 복원, 유니코드 정규화, 글자 깨짐 확인
- 영역 Crop
  - 여백, Header/Footer 정보가 검색에 노출될 수 있음
  - 해당 정보는 주로 문서 제목이 반영되어 있어 반복적으로 검색 될 수 있음
  - 불필요하다면 사전 제거
- 차트/표 잔여 텍스트가 본문 노이즈가 될 경우
  - 글자수 필터링 (e.g., 한줄 N자 미만 제거)

❑ Parsing – 구조/읽기 순서



- 다단/복잡 레이아웃은 읽기 순서에 맞춰 분리
  - Pdfplumber(pdfminer.six 기반)로 문자 좌표(bbox) 추출 후 순서 재구성
  - 다단은 단(column) 경계 잡고 튜닝 필요
- 표 데이터가 핵심이면 전용 도구로 별도 추출 (feat. OCR, VLM, etc.)
- 파싱 후 품질 검증 필수
  - 추출 실패, 빈 텍스트, 깨진 문자 비율 점검

TIP 복잡한 표, 파싱 전략



2) 「조세특례제한법」상 공제·감면제도

주요 공제·감면제도 요약

구분	공제·감면 내용
창업중소기업등에 대한 법인세 감면 (조특법§6)	• '27.12.31. 이전에 창업한 중소기업 등에 대하여 창업 후 소득발생연도부터 5년간 매년 납부할 법인세(소득세) 감면(상시근로자 증가시 최대 50% 추가감면)
중소기업에 대한 특별세액감면 (조특법§7)	• 제조업 등 감면업종을 경영하는 중소기업에 대하여 '28.12.31.까지 다음 구분에 따라 법인세의 5~30% 감면
통합투자세액공제 (조특법 §24)	• 기업이 시설투자(기계장치, 사업용 유형자산 등) 등에 투자한 경우 투자금액의 일정액(①+②)을 법인세에서 세액공제 (주의) 투자완료일부터 2년(5년)내 해당자산을 처분, 임대 또는 다른목적으로 전용한 경우 공제 받은 세액공제액 상당액에 이자 상당 가산액을 가산하여 법인세 납부

16 외국투자기를 위한 조세가이드

- 표 파싱이 왜 어려운가?
  - 구조 파괴 : 단순 텍스트 추출 시 행과 열의 관계가 무너져 의미 전달 불가능
  - 병합 셀 문제 : 상하/좌우로 병합된 셀은 빈 데이터로 인식되어 데이터 왜곡 발생
  - 비정형 문서 : 스캔된 이미지 형태의 문서는 일반적인 파서로 접근 불가

- 주요 파싱 방법론

구분	PdfPlumber	OCR + Clustering	VLM
원리	PDF 텍스트 레이어 좌표 추출	이미지 내 텍스트 좌표군 클러스터링	시각적 추론 통한 구조화 (JSON)
장점	처리 속도 빠름, 비용 없음	스캔 이미지 적용 가능	중첩/복잡한 표에 압도적 성능
단점	텍스트 레이어 없는 파일 불가	좌표 오차로 인한 열 뒤섞임 발생	API 비용 발생, 속도 상대적 느림

  - 텍스트 레이어, Text Layer : PDF는 2개 층으로 겹쳐져 있음
    - 이미지 층 : 사람 눈에 보이는 글자와 그림 (모습)
    - 텍스트 층 : 눈에는 보이지 않지만, 머신이 글자를 인식하고 드래그 할 수 있게 해주는 데이터 층
    - 스캔 문서는 텍스트 층이 없어 드레그가 안되지만, PDF는 해당 레이어가 있어 글자를 복사할 수 있음
  - 좌표, Coordinate : 머신은 각 글자가 문서의 어디에 위치하는지를 숫자(x, y)로 파악
    - X좌표 : 문서 왼쪽 끝에서부터 얼마나 떨어져 있는지 (가로 위치)
    - Y좌표 : 문서 위쪽 끝에서부터 얼마나 떨어져 있는지 (세로 위치)
    - Boundary Box : 글자를 감싸는 사각형의 네 꼭지점 좌표

- 단순 표 : 비용 효율적인 PdfPlumber 활용 (후처리로 빈 셀 보정 필요)
- 스캔 표 : OCR 등으로 텍스트와 좌표를 추출한 뒤 Y좌표 기준으로 행 재구성
- 중첩 표 : VLM에 JSON 스키마를 프롬프트로 전달하여 의미론적 파싱 수행

Source : https://policy.nl.go.kr/search/searchDetail.do?rec_key=SH2_PLC20260365888

TIP 복잡한 문서 구조

법령, 규정, 계약

계층적 구조

- 1조, Article
- ①항, Paragraph
- 1호, Subparagraph

계층 단절 : 항, 호 단위 청크는 어느 조 소속인지 사라져 의미가 불완전

▢ **제18조(개인정보의 목적 외 이용 · 제공 제한)** ① 개인정보처리자는 개인정보를 제15조제1항에 따른 범위를 초과하여 이용
제공하여서는 아니 된다.

② 제1항에도 불구하고 개인정보처리자는 다음 각 호의 어느 하나에 해당하는 경우에는 정보주체 또는 제3자의 이익을 부
용도로 이용하거나 이를 제3자에게 제공할 수 있다. 다만, 제5호부터 제9호까지의 경우는 공공기관의 경우로 한정한다.

1. 정보주체로부터 별도의 동의를 받은 경우
2. 다른 법률에 특별한 규정이 있는 경우
3. 정보주체 또는 그 법정대리인이 의사표시를 할 수 없는 상태에 있거나 주소불명 등으로 사전 동의를 받을 수 없는 경우

교차 참조 대상 조문 문맥 파악

"제15조제1항에 따라"만 검색되고 해당 항의 내용이 빠짐

단서/예외 본문 내용과 하나의 청크로 유지되도록

해당 청크가 다른 청크로 구분되면 해당 제약사항을 고려하지 않은 답변 생성

▢ **제2조(정의)** 이 법에서 사용하는 용어의 뜻은 다음과 같다. <개정 2014. 3. 24.>

1. "개인정보"란 살아 있는 개인에 관한 정보로서 성명, 주민등록번호 및 영상 등을 통하여 개인을 알아볼 수 있는 정보(해당 정보만으로는 특정 개인을 알아볼 수 없더라도 다른 정보와 쉽게 결합하여 알아볼 수 있는 것을 포함한다)를 말한다.
2. "처리"란 개인정보의 수집, 생성, 연계, 연동, 기록, 저장, 보유, 가공, 편집, 검색, 출력, 정정(訂正), 복구, 이용, 제공, 공개, 파기(破棄), 그 밖에 이와 유사한 행위를 말한다.
3. "정보주체"란 처리되는 정보에 의하여 알아볼 수 있는 사람으로서 그 정보의 주체가 되는 사람을 말한다.
4. "개인정보파일"이란 개인정보를 쉽게 검색할 수 있도록 일정한 규칙에 따라 체계적으로 배열하거나 구성한 개인정보의 집합물(集合

시점 질의 시점에 유효한 버전인지 확인

개정 버전을 구분하지 못하면, 답변 자체가 달라짐

정의 참조 용어 정의로 연결

▢ **제50조(조정절차 등)** ① 제43조부터 제49조까지의 규정에서 정한 것 외에 분쟁의 조정방법, 조정절차 및 조정업무의 처리 등에 필요한 사항은 대통령령으로 정한다.

② 분쟁조정위원회의 운영 및 분쟁조정 절차에 관하여 이 법에서 규정하지 아니한 사항에 대하여는 「민사조정법」을 준용한다.

준용/외부 참조 다른 문서에서 확인해야 할 수도 있음

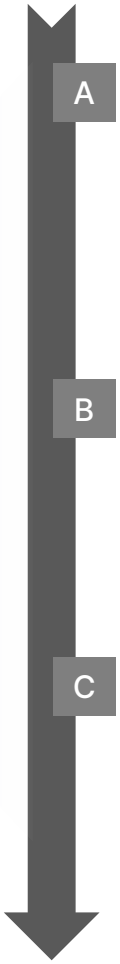
← 해당 청크 만으로는 부족

Source : https://www.law.go.kr/lsEfInfoP.do?lsiSeq=195062#

TIP 복잡한 문서 구조, 파싱 전략

파싱 만큼 중요한 것 - 검색 시 **"맥락 확장"**

- 상위 조항(parent) 경로 – 소속 조/항의 맥락
- 참조 대상 조항 – "제N조에 따라" 위한 N조 본문
- 본문에 걸린 단서, 예외 – 규칙과 한 묶음으로
- 적용 시점 유효 버전 – 개정 이력 중 질의 시점 기준



책을 분류/정리 하는 것. 몇 층/몇 번 서가에 꽂을지 번호 규칙대로 배치 → 구조를 잡는 단계

A 규칙 기반 계층 파싱

- 정규식 기반 트리화 (제N조, ①, 1., 가.)
- 상위 경로를 메타 데이터로 저장
- 장점 : 번호 체계가 규칙적이면 정확
- 단점 : 불규칙 체계에는 취약, 참조는 해결하지 못함

각 책을 읽고 "이 책은 저 책을 인용하고 있다"는 메모를 붙이는 일 → 관계를 찾아내는 단계

B LLM 기반 구조화

- 문서 구조 스키마 제공 → LLM이 구조와 관계를 JSON으로 추출
- 장점 : 암묵적 참조 처리 가능 (e.g., "앞 항", "이전 조항" 등)
- 단점 : 비용/속도, 환각

메모들을 모아 어느 책이 어느 책과 연결되는 관계를 그리는 일 → 관계 저장/활용 단계

C 참조 그래프 (feat. GraphRAG)

- Node = 조항
- Edge = 관계 (참조, 준용, 정의, 개정, 예외 등)
- 장점 : 흩어진 조항, 정의, 예외 간의 관계를 multi-hop 질의로 처리 가능
- 단점 : 그래프 구축, 유지 비용 큼

2-(3). Embedding

- Text Split 단계에서 생성된 문서 단위들을 기계가 이해할 수 있는 수치적 형태로 변환하는 과정
- 사용자가 입력한 질문(Query)에 대하여 DB에 저장된 문서 조각/단락(Chunk)을 검색하여 가져올 때 유사도 계산 시 활용
- https://docs.langchain.com/oss/python/integrations/embeddings

Embedding 필요성

1. 의미 이해

복잡하고 다양한 의미를 내포하고 있는 자연 언어를 정량화된 형태로 변환함으로써 컴퓨터가 문서의 내용과 의미를 더 잘 이해하고 처리할 수 있음

2. 정보 검색 향상

수치화된 벡터 형태로의 변환은 문서 간의 유사성을 계산하는 데 필수적임. 이는 관련 문서를 검색하거나, 질문에 가장 적합한 문서를 찾는 작업을 용이하게 함

(예시) Q : IDC가 예측한 AI소프트웨어 시장의 연평균 성장율은 어떻게 되나요?

시장조사기관 IDC는 AI 소프트웨어 시장이 2022년 640억 달러에서 2027년 2,510억 달러로 연평균 성장률 31.4%를 기록하며 급성장할 것으로 예상 ①
AI 소프트웨어 시장은 AI 플랫폼, AI 애플리케이션, AI 시스템 인프라 소프트웨어(SIS), AI 애플리케이션 개발·배포(AI AD&D) 소프트웨어를 포괄

협업, 콘텐츠 관리, 전사적 자원관리(ERM), 공급망 관리, 생산 및 운영, 엔지니어링, 고객관계관리(CRM)를 포함하는 AI 애플리케이션은 AI 소프트웨어의 최대 시장으로 2023년 전체 매출의 약 3분의 1을 차지하며 2027년까지 21.1%의 연평균 성장률을 기록할 전망 ②
AI 비서를 포함한 AI 모델과 애플리케이션의 개발을 뒷받침하는 AI 플랫폼은 두 번째로 시장 규모가 큰 분야로, 2027년까지 35.8%의 연평균 성장률이 예상됨

분석, 비즈니스 인텔리전스, 데이터 관리와 통합을 포함하는 AI SIS는 기존 소프트웨어 시스템과 통합되어 방대한 데이터를 활용한 의사결정과 운영 최적화를 지원하며, 현재 매출 규모는 비교적 작지만 5년간 연평균 성장률은 32.6%로 시장 전체를 웃돌 전망 ③
애플리케이션 개발, 소프트웨어 품질과 수명주기 관리 소프트웨어, 애플리케이션 플랫폼을 포함하는 AI AD&D는 향후 5년간 카테고리 중 가장 높은 38.7%의 연평균 성장률이 예상됨

① 단락 : [0.1, 0.5, 0.9, ..., 0.1, 0.3]
② 단락 : [0.7, 0.1, 0.3, ..., 0.4, 0.2]
③ 단락 : [0.9, 0.4, 0.5, ..., 0.4, 0.2]
질문 : [0.1, 0.5, 0.9, ..., 0.6, 0.8]

유사도 계산 (단락 – 질문)
① 단락 : 80%
② 단락 : 30%
③ 단락 : 20%

2-(3). Embedding

- https://docs.langchain.com/oss/python/integrations/embeddings

```
# Setup : Environment (.env)
OPEN_API_KEY=“---”


# API
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings()
```

[참고] Embedding

One-Hot Encoding

	request	feature	how	issue	credit
request →	1	0	0	0	0
feature →	0	1	0	0	0
how →	0	0	1	0	0
issue →	0	0	0	1	0
credit →	0	0	0	0	1

- 거대한 문서 모음집이 있다고 가정
- 해당 모음집의 모든 단어들에 대하여 숫자 인덱스를 부여
- 단어 수에 따라 embedding vector 차원이 지나치게 커지는 경향이 있고, embedding vector 내 절대 다수의 원소가 “0”값을 가진 sparse 구성

Learned Embedding

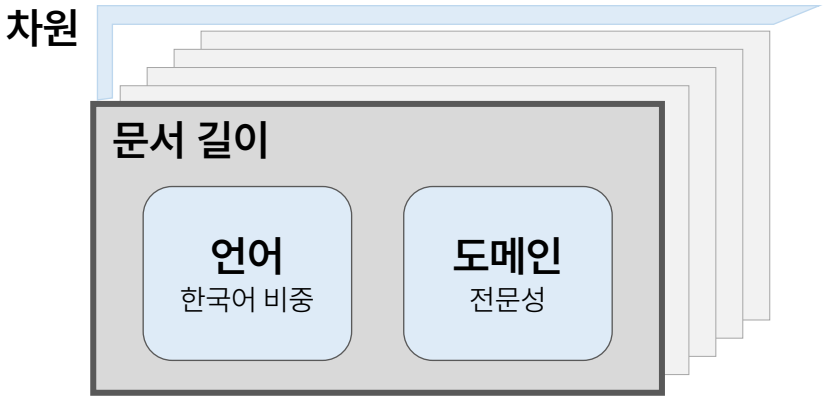
request →	0.007	-0.039	-0.002	-0.024	0.007
feature →	-0.014	-0.011	0.014	-0.017	0.025
how →	0.01	0.004	0.006	-0.037	0.056
issue →	-0.015	-0.035	-0.009	-0.012	-0.015
credit →	-0.012	-0.027	-0.001	-0.051	0.064

- Neural network 모델로 학습하여 생성
- 학습 과정에서 문맥 상 의미적으로 유사한 단어들의 embedding vector들 간의 거리는 **가깝게**, 그렇지 않은 단어들 간의 거리는 **멀게** 만드는 방식으로 의미적 관계 학습
- 낮은 차원 (e.g., 384~1,536) 에서 모든 원소가 밀도 있게 활용 가능

2-(3). Embedding

Embedding 모델 선택 – 비용을 넘어서는 의사결정 필요

- 한 번 선택되면 전체 인덱스 구조와 운영 비용에 영향
- 만약 임베딩 모델이 중간 교체된다면, 전체 파일에 재임베딩 및 인덱스 재구성 하는 비용 부담



Embedding 모델 선택 시 주요 고려사항 :

- 언어 : 한국어 비중 (다국어)
- 도메인 : 제조, 반도체, 통신 등 도메인 전문성이 반영된 전문 용어
- 문서 길이 : 모델 최대 입력 토큰 고려
- 차원 : 차원, 인덱스 규모 고려
- 비용 : 전처리 뿐만 아니라 사용자 쿼리 시에도 적용

모델 선정 후 **반드시 테스트** 통해 확정 필요 !!

[참고] MTEB Leaderboard

(2026.06) Multilingual(Korean, English), Domain(Web, News, Blog)

MTEB Rank	Model	Active Params (B)	Embedding Dim	Max Tokens	Zero-shot %	Retrieval
1	geevec-ai/geevec-embeddings-1.0		4,096		NA	83.49
2	Octen/Octen-Embedding-8B	7.567	4,096	32,768	98	75.36
3	google/gemini-embedding-001		3,072	2,048	98	76.52
4	tencent/KaLM-Embedding-Gemma3-12B-2511	11.766	3,840	32,768	75	73.48
6	microsoft/harrier-oss-v1-27b	27.009	5,376	131,072	84	72.38
7	Qwen/Qwen3-Embedding-8B	7.567	4,096	32,768	98	72.76
8	geevec-ai/geevec-embeddings-1.0-lite	0.349	4,096	32,768	NA	72.80
9	nvidia/llama-embed-nemotron-8b	7.505	4,096	32,768	98	71.30
10	Qwen/Qwen3-Embedding-4B	4.022	2,560	32,768	98	70.10
17	jinaai/jina-embeddings-v4	3.935	2,048	32,768	98	67.22
24	google/embeddinggemma-300m	0.308	768	2,048	98	65.17
32	Qwen/Qwen3-Embedding-0.6B	0.596	1,024	32,768	98	60.46
52	openai/text-embedding-3-large		3,072	8,191	NA	56.67
79	BAAI/bge-m3	0.568	1,024	8,191	98	53.63
90	openai/text-embedding-3-small		1,536	8,191	NA	48.00

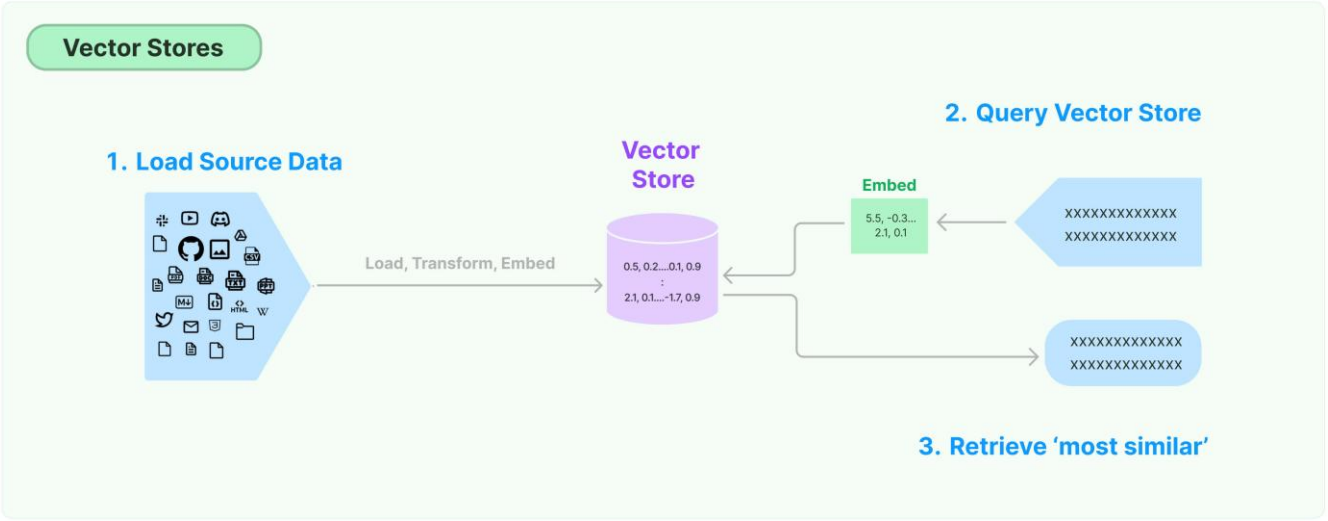
※ Remark

- Zero-shot : What portion of the benchmark a model has not been trained on.
- Retrieval : Find the most relevant documents for a query out of a large corpus.
- **Leaderboard는 대상을 검토하기 위한 시작으로 보고, 최종 임베딩 모델은 다루는 데이터셋에 적용하여 최적의 모델을 선정해야 함**

Source : https://huggingface.co/spaces/mteb/leaderboard

2-(4). Vector Database

- 이전 단계에서 생성된 임베딩 벡터들을 효율적으로 저장하고 관리하는 과정
- 향후 검색 과정에서 벡터들을 빠르게 조회하고, 관련 문서를 신속하게 찾아내는데 필수적
- https://docs.langchain.com/oss/python/integrations/vectorstores



벡터스토어는...

RAG 시스템의 검색 기능과 직접적으로 연결되어 있고,
전체 시스템의 응답시간과 정확성에 큰 영향을 미침

1. 빠른 검색 속도: Embedding Vector를 효과적으로 저장하고 indexing하여, 대량의 데이터 중에서도 관련 정보를 빠르게 검색할 수 있음

2. Scalability: 데이터가 지속적으로 증가함에 따라 이를 수용할 수 있도록 확장성을 보장하며, 시스템의 성능 저하 없이 대규모 데이터를 관리할 수 있도록 함

Source : https://jayant017.medium.com/rag-using-langchain-part-3-vector-stores-and-retrievers-a75f4d14cbf3

2-(4). Vector Database

- https://docs.langchain.com/oss/python/integrations/vectorstores

```
from langchain_community.vectorstores import FAISS

vectorstore = FAISS.from_documents(documents=docs, embedding=embeddings)
```

2-(4a). Database

Vector DB는 단순한 수치 저장소가 아니라 “vector + meta”를 함께 저장하고 검색까지 가능한 특화된 구조

RDB, Table

COLUMNS

Primary Key

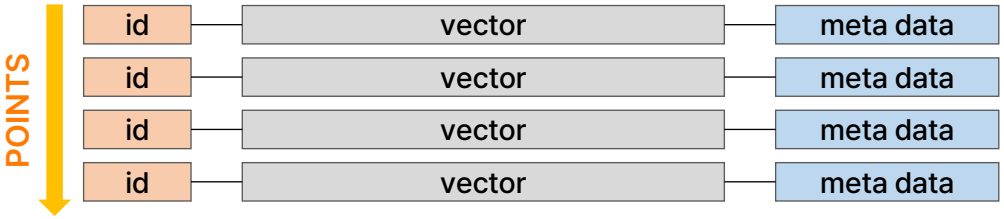
ROWS

ID	COL 1	COL 2	COL 3
ID	Field Data	Field Data	Field Data
ID	Field Data	Field Data	Field Data
ID	Field Data	Field Data	Field Data

VectorDB, Collection

DIMENSIONS

POINTS



구성 요소	내용	RDB 기준 비교
Collection	• 벡터들을 그룹화한 저장 단위 • 주로 하나의 데이터셋 또는 주제 단위로 관리	Table
Point	• 벡터 하나의 데이터 단위 • ID – Vector – Payload로 구성	Row
ID	• 각 Point를 식별하는 고유값 • 주로 UUID로 구성	Primary Key
Vector	• 다차원 수치 임베딩 벡터 • 유사도 검색의 핵심 데이터	Field Data
Meta data	• 벡터에 부가적으로 붙는 설명 정보 • 필터링용 메타정보	Column (메타정보)

※ 기본 구성 요소는 Vector DB 마다 유사하나 명칭이 조금씩 상이함
- Chroma : Collection – Document - Metadata
- Qdrant : Collection – Point – Payload
- Pinecone : Namespace – Vector – Metadata
- Weaviate : Class – Object – Properties

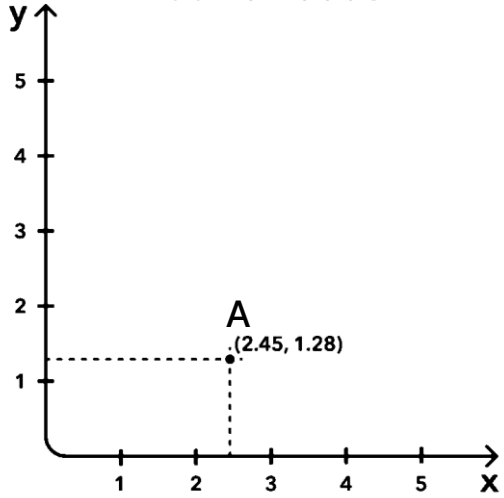
[참고] Embedding Vector

임베딩 모델에 의해 생성된 벡터

2-Dimension Vector

$A = [2.45, 1.28]$

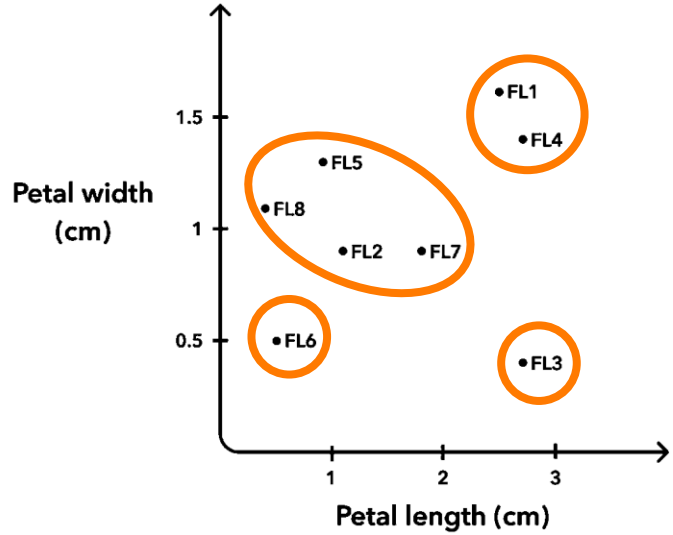
2개의 숫자로 하나의 벡터가 구성되며,
각각은 다른 의미의 정보를 담음



IRIS Dataset

ID	Petal length (cm)	Petal width (cm)	Vector
FL1	2.5	1.6	[2.5, 1.6]
FL2	1.1	0.9	[1.1, 0.9]
FL3	2.7	0.4	[2.7, 0.4]
FL4	2.7	1.4	[2.7, 1.4]
FL5	0.9	1.3	[0.9, 1.3]
FL6	0.5	0.5	[0.5, 0.5]
FL7	1.8	0.9	[1.8, 0.9]
FL8	0.4	1.1	[0.4, 1.1]

Plotting vectors
→ similar to each other



Source : https://www.contentful.com/blog/what-are-vector-databases/

[참고] 텍스트를 머신이 인식할 수 있도록 바꿔야 한다

Token

- 정의 : 문장을 단어 또는 서브워드로 쪼개어 나눈 조각 ➔ 단어 조각
- 역할 : 텍스트를 처리 가능하도록 최소 단위로 분할
- 비유 : 책을 단어별로 잘라놓은 것
- 예시 : “I love Orange” → [“I”, “love”, “Orange”]

Embedding

- 정의 : 각 토큰을 고정된 벡터 공간으로 변환한 수치 표현 ➔ 단어별 의미 점수표 (정적인 의미 공간)
- 역할 : 토큰 → 숫자 벡터로 바꾸어 신경망이 학습 가능하게 함
- 비유 : 단어에 의미를 입힌 숫자ID 카드
- 예시 : “love” → [0.12, -0.87, 0.55, ..., 0.01]

Encoding

- 정의 : 토큰의 위치/문맥 등 포함된 정보까지 반영한 고차원 표현 ➔ 문맥에 따라 달라지는 점수표 (동적인 문장 속 의미)
- 역할 : 전체 문장의 구조적, 문맥적 의미를 포함
- 비유 : 문장 속에서 역할이 반영된 단어
- 예시 : "love"가 문장마다 다르게 인코딩됨
  → "love" (in "I love orange") → [0.51, -0.42, 0.98, ..., 0.20]
  → "love" (in "They love pizza") → [0.63, -0.33, 0.85, ..., 0.31]

Q1) Token은 숫자인가요? → Token은 문자 조각, 숫자는 Embedding 입니다.
Q2) Embedding과 Encoding은 같은가요? → Embedding은 의미 벡터, Encoding은 문맥까지 반영한 표현 입니다
Q3) 같은 Token이라도 Embedding과 Encoding 차원은 다른가요? → 보통 다를 수 있습니다. 모델 구조에 따라 상이 (BERT: 768, RoBERTa-Larget : 1024)

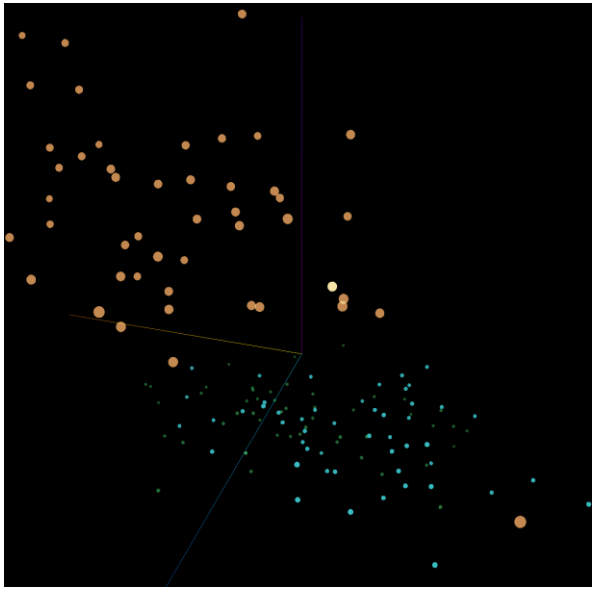
[참고] Dimension? 차원이 벡터 표현에 미치는 영향

벡터 공간에서 "축"은 데이터를 설명하는 특징으로 데이터의 복잡도와 표현력을 의미함

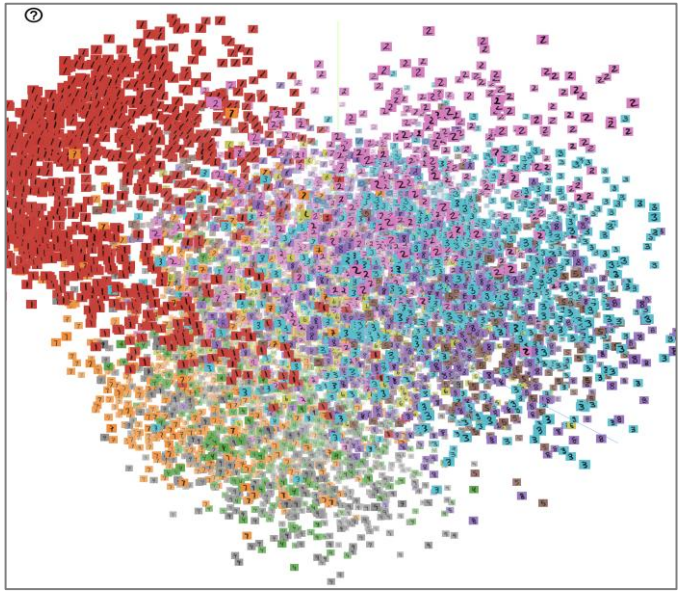
- Low Dimension : 간단한 패턴만 표현 가능, 거리 기반으로 명확하게 유사도 판단 가능. 그러나 정보 손실 우려
- High Dimension : 복잡한 의미나 추상적 개념 표현 가능. 그러나 모든 벡터가 비슷해 보이는 현상 발생 가능

→ 거리 기반 유사도 계산의 신뢰도 저하, 차원의 저주

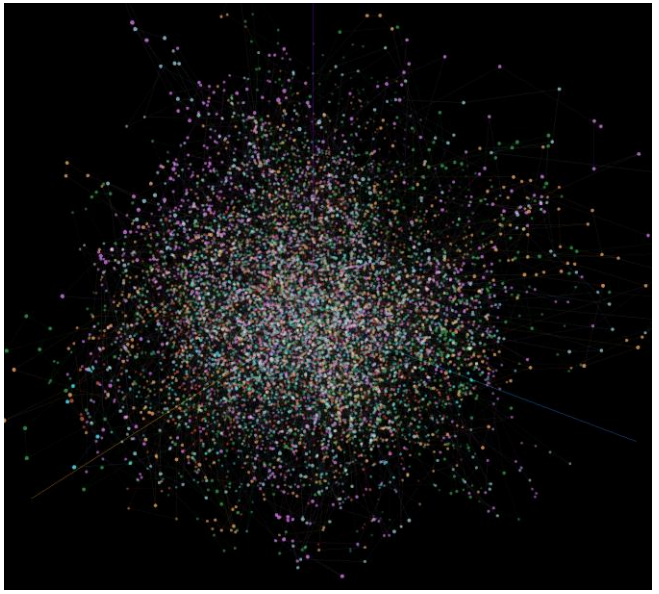
Data : IRIS
Points : 150
Labels : 3
Dimension : 4



Data : MNIST
Points : 10,000
Labels : 10
Dimension : 784



Data : GNMT Interlingua
Points : 9978
Labels : 3 (srctar)
Dimension : 1024



※ 차원의 저주, Curse of Dimensionality
차원이 높아질수록 거리 계산이 신뢰성을 잃고
데이터가 퍼져 유사도 판단이 어려워지는 현상

Source : https://projector.tensorflow.org

[참고] 적절한 벡터 차원 수?!

차원 수는 데이터의 복잡성, 벡터DB 사용 목적에 따라 다르게 설정됨

- 차원 : 128 | 512 | 768 | 1024 | 1536 | 2048 | ...
- 적절한 차원 수는 "표현력 vs 효율성" 균형 고려

구분	일반적인 차원 범위	내용
추천 시스템	64 ~ 128	희소 데이터, 간단한 추론 로직
간단한 텍스트 유사도	128 ~ 512	문데이터셋 크기(작은 데이터셋), 실시간 처리(빠른 검색용), 경량 모델
이미지 검색/인식	256 ~ 2048	시각 특징 포착, 효율성을 위한 차원 축소 고려
일반 Q&A, RAG	768 ~ 1024	풍부한 의미론적 문맥 필요, BERT/GPT 계열 모델 사용, 질의 응답
복합 문서 (코드, 테이블, 수식 등)	1024 ~ 1536	복잡한 구조 표현, 많은 특징에 대한 인코딩 필요
멀티모달 (e.g. 텍스트+이미지)	1024 ~ 2048+	여러 모달리티 특성 반영, 다양한 모델 특성 고려

[참고] 차원 수가 '2의 제곱수' 형태인 이유?

- 하드웨어 최적화 : 딥러닝 연산 장치는 2, 4, 8, ... 단위로 연산 블록을 병렬로 처리 → 128, 256처럼 2의 제곱 단위 차원이 연산 최적화에 유리
- 모델 구조상 고정된 출력 차원 : 많은 임베딩 모델은 사전 정의된 차원수를 출력 (e.g. BERT 768차원, GPT3 12288차원 ← Head 당 128차원)
- 벡터DB 및 검색 구조 상 정렬 최적화 : 고정된 크기는 메모리 정렬과 CPU 캐시 효율성 확보에 도움

2-(4b). Vector Index

빠른 근사 검색을 위해...

비교 대상을 처음부터 줄이는 것이 목적

➔ Vector Index는 **모든 것을 비교하지 않아도** 되게 만드는 구조적 장치



- 차원 : 384, 768, 1536, 3072 등
- 저차원이면 : 서로 다른 대상이 겹쳐 보임
- 고차원이면 : 의미적으로 다른 것들은 더 멀어지고, 비슷한 것들은 더 가까워짐

- **차원이 크다는 것은, 대상을 더 많은 관점에서 자세히 표현할 수 있음**
- **그러나 Vector 검색은 차원이 높아질수록 계산 비용이 급격히 증가함**
- **모든 벡터를 하나씩 비교하는 방식(Full Scan)은 데이터가 커질수록 현실적으로 사용 불가능**

- 벡터 거리 계산은 각 차원을 하나씩 비교해서 차이를 모두 더하는 과정
- (e.g.) A와 B 벡터가 (키, 몸무게) 구성 : 키 차이, 몸무게 차이 ➔ 2번 비교
- 차원이 늘어난다는 것은 비교해야 할 체크리스트가 길어지는 것

2-(4b). Vector Index

수많은 벡터 데이터 중에서 가장 유사한 벡터를 빠르게 찾기 위한 전략 !!!

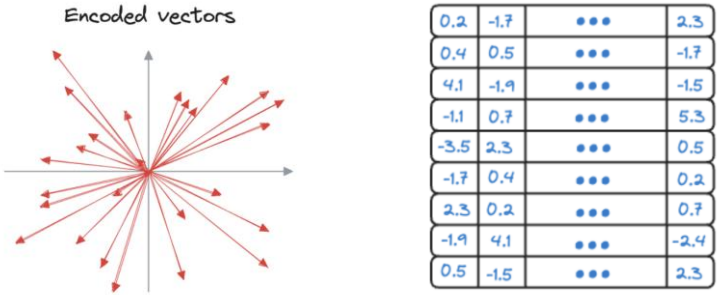
구분	Flat Brute-Force	IVF Inverted File Index	HNSW Hierarchical Navigable Small World	PQ Product Quantization
주요 특징	• 모든 벡터와 직접 거리 계산 (Exhaustive Search)	• 벡터를 클러스터링 한 후에 • 해당 클러스터 내에서만 탐색	• 그래프 기반 탐색 • 다단계 계층적 그래프 구조	• 벡터를 압축하여 저장 • 거리 계산은 근사값
거리 계산	• Nearest Neighbor	Approximate Nearest Neighbor (ANN) 정확도 보다는 근접한 결과를 빠르게 찾기 위한 방법. 근사이기 때문에 정답과 아주 약간의 오차 허용		
적합한 경우	• 작은 데이터셋 • 소규모 벡터 테스트 시 유용	• 중~대규모 데이터셋	• 실시간 검색 • 정밀도-속도 균형 고려	• 모바일, 임베디드 환경
장점	• 정확도 100% • 정확한 최근접 이웃	• 속도 빠름 • 검색 공간 축소	• 빠른 검색 • 정밀도 높음	• 빠른 근사 검색 • 저장공간 ↓
단점	• 속도 느림 • 대규모 처리 비효율	• 정밀도 ↓ • nprobe 조절하여 속도/정확도 균형 조정 가능	• 인덱스 구축에 메모리 소모 • 벡터 삽입 느림	• 정확도 ↓ • 압축 따른 정보 손실
대표 예시	• FAISS (feat. Scikit-learn)	• Pinecone • FAISS • Milvus	• Qdrant • Weaviate • Vespa.ai	• FAISS • Annoy (Spotify)

검색 속도와 정확도 간의 Trade-off 조절

※ nprobe ≈ Number of probes. 검색 시 탐색할 클러스터 개수
- nprobe 값이 크면 : 더 많은 클러스터 탐색 → 정확도 ↑, 속도↓
→ 정확도 vs 속도 trade-off 조절 지표

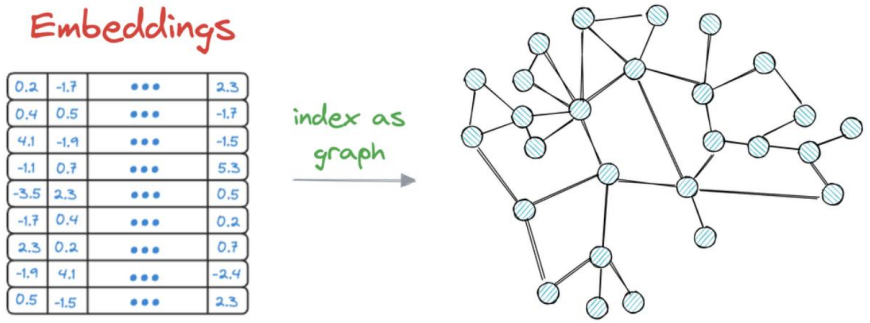
2-(4b). Vector Index

Flat – it involves no indexing strategy, and stores the vectors as they are, i.e., in a 'flat' data structure.



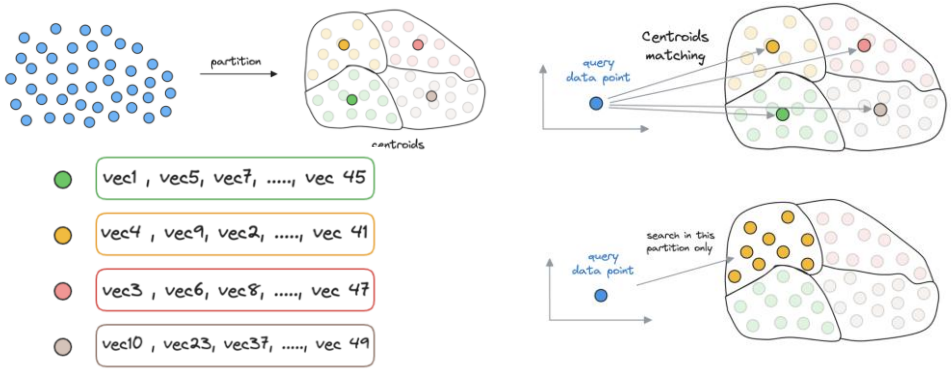
0.2	-1.7	•••	2.3
0.4	0.5	•••	-1.7
4.1	-1.9	•••	-1.5
-1.1	0.7	•••	5.3
-3.5	2.3	•••	0.5
-1.7	0.4	•••	0.2
2.3	0.2	•••	0.7
-1.9	4.1	•••	-2.4
0.5	-1.5	•••	2.3

HNSW – to construct a graph structure, where each node represents a vector, and edges connect nodes based on their similarity.

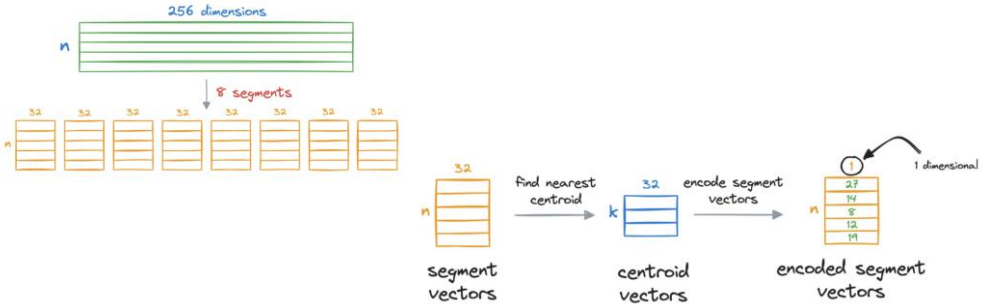


0.2	-1.7	•••	2.3
0.4	0.5	•••	-1.7
4.1	-1.9	•••	-1.5
-1.1	0.7	•••	5.3
-3.5	2.3	•••	0.5
-1.7	0.4	•••	0.2
2.3	0.2	•••	0.7
-1.9	4.1	•••	-2.4
0.5	-1.5	•••	2.3

IVF – First, compare the input query across all centroid, and then compare it to the vectors in the obtained partition.



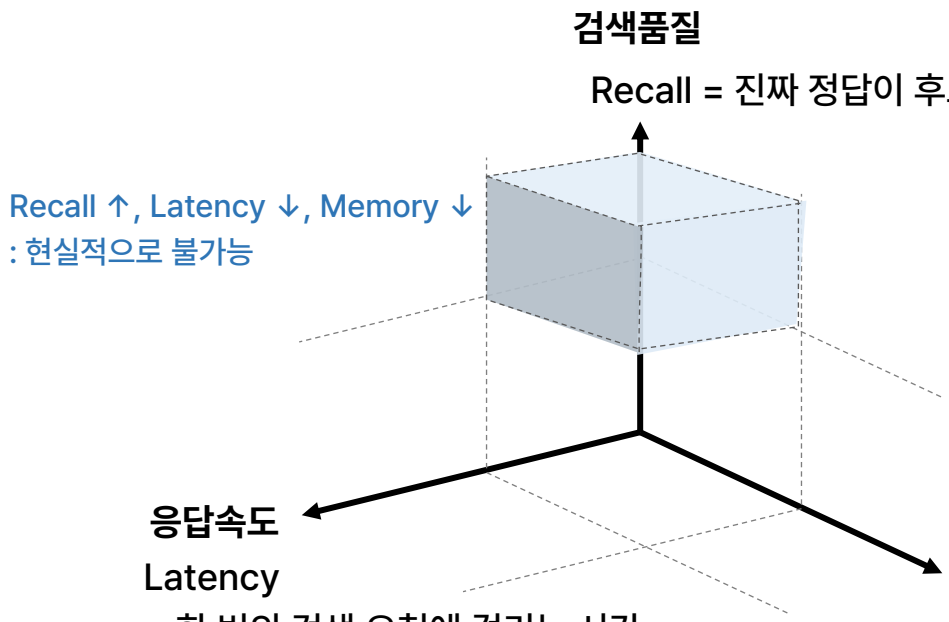
Product Quantization – Increasing the number of centroids and segments will increase the precision of the ANN search, but it will also increase the run-time of search algorithm.



Source : https://www.dailydoseofds.com/a-beginner-friendly-and-comprehensive-deep-dive-on-vector-databases/

[참고] Recall – Latency – Memory

3축을 동시에 최적할 수 없다! 하나를 얻으면 하나 이상을 포기해야 한다!!!



검색품질

Recall = 진짜 정답이 후보 안에 포함될 확률

높을수록 : 검색 품질 안정적

낮을수록 : 일부 정답 누락 가능 & 속도/메모리 절약 가능

Recall ↑, Latency ↓, Memory ↓

: 현실적으로 불가능

응답속도

Latency

= 한 번의 검색 요청에 걸리는 시간

낮을수록 : 실시간 서비스

높을수록 : 배치 처리, 오프라인 분석

자원비용

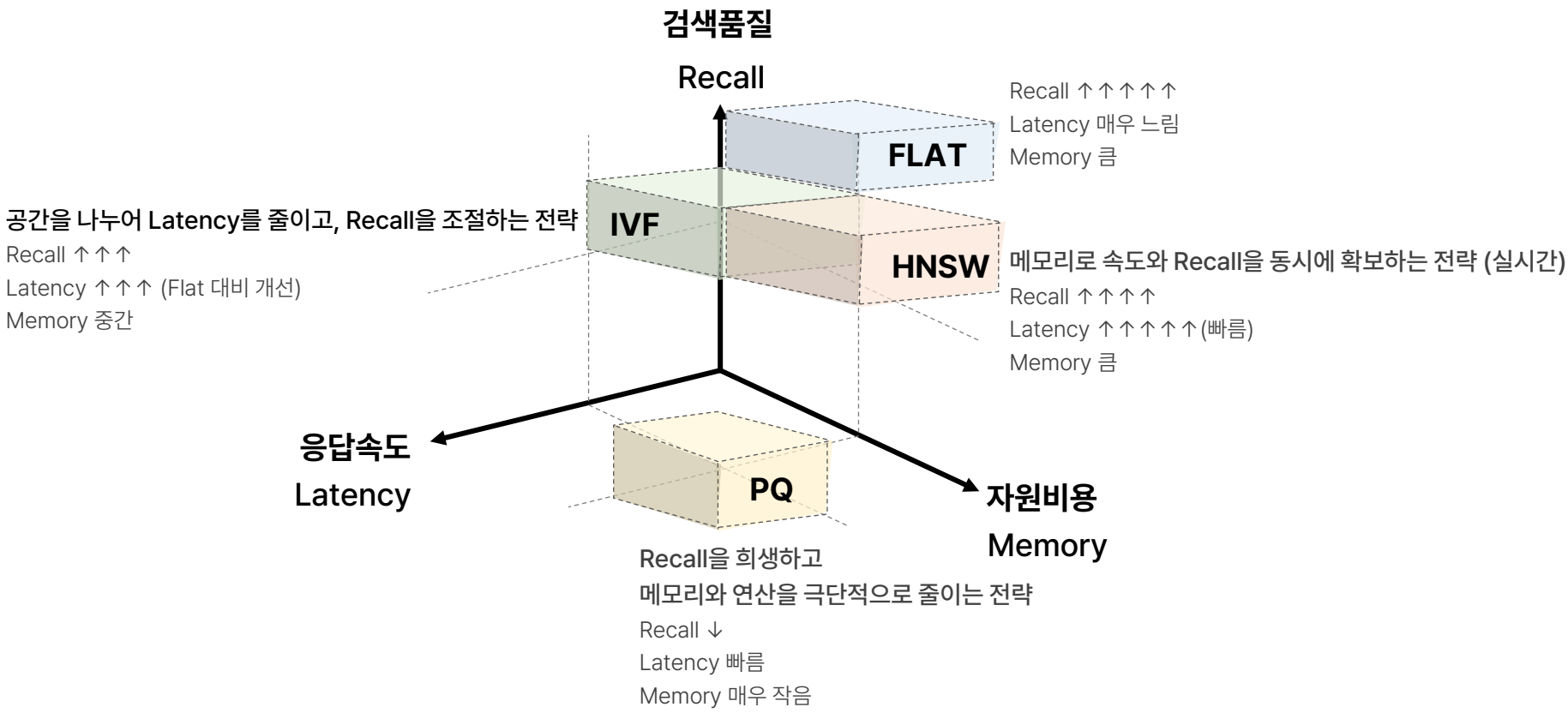
Memory = Index + 벡터를 저장하는데 필요한 메모리

적을수록 : 모바일, 엣지 환경

많을수록 : 대규모 서버

2-(4b). Vector Index

Index 선택은 서비스 요구사항 중 **무엇을 가장 중요하게 볼 것인가**에 대한 선택



2-(4c). Vector Stores

Vector DB 선택 시 고려해야 할 기능/성능 차이 비교

구분	FAISS	Milvus	Qdrant	Chroma	Pinecone
출시 년도	2017	2019	2021	2023	2021
개발사	Meta	Zilliz	Qdrant Team	ChromaDB	Pinecone Systems, Inc.
주요 목적	벡터 데이터의 고속 검색 위한 인덱싱 전용 라이브러리 (DB 아님)	대규모 서비스용 서버형 Vector DB	벡터+조건 필터 기반 하이브리드 검색	개발, 실험 단계의 벡터, 메타데이터 관리	클라우드 기반 벡터 데이터 검색 및 관리 (SaaS)
성능	GPU 기반의 고성능 벡터 검색	대규모 데이터셋에서 안정적 성능	실시간 벡터 + 조건 검색	중소규모 데이터셋에서 빠르고 간단하게 사용	대규모 클러스터 확장 가능, 실시간 검색 지원
확장성	직접 구현 필요	클러스터 기반 고확장성	Docker/K8S 기반 분산 지원	로컬 중심, 제한적	자동 확장 (Managed)
특징적 기능	다양한 인덱싱 방식 제공 (PQ, HNSW 등)	분산 아키텍처, 다양한 Index 관리	필터 조건 + 벡터 검색 결합, REST/gRPC API 지원	Python API로 쉽고 빠르게 사용 가능	운영, 인프라 관리
장점	성능, 알고리즘 통제 가능	서비스 환경 바로 적용 가능	정밀한 조건 검색, 클라우드/On-Prem 모두 지원	로컬 데이터 처리 및 관리가 간단	운영 부담 최소, 빠른 상용화
단점	배포 및 설정 복잡	인프라/운영 복잡	초대규모 확장에는 제약	대규모 서비스 부적합	SaaS 모델로 관련 비용 발생

※ FAISS는 기능적으로는 Vector Store의 일부를 담당하지만, Database로서 운영/기능 범위를 포괄하지는 않음.
그러나 중소규모 프로젝트에서는 로컬 메모리 기반의 Vector Store처럼 사용되고 있음.

[참고] WHICH Vector Stores ????

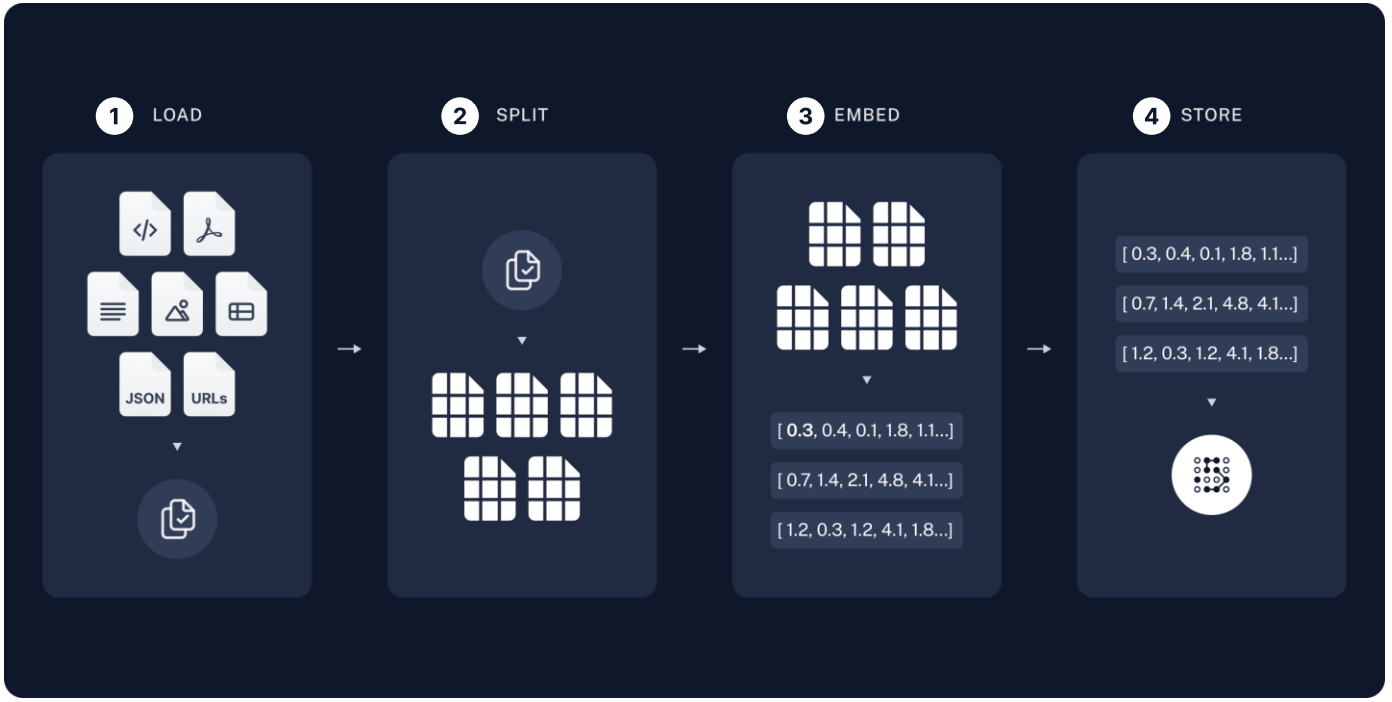
구분	후보	주요 내용
소규모 프로젝트	Chroma, FAISS	오픈소스 기반, 로컬 테스트 최적화
대규모 데이터 (수억 개 이상)	Milvus, Vespa	분산 처리 가능, GPU 지원(M), 고성능 커스터마이징 가능(V)
Edge or 경량AI	Qdrant, FAISS	리소스 효율성 확보 가능
실시간 추천/검색 시스템	Qdrant, Pinecone	초저지연 검색(Q), 실시간 삽입/삭제/업데이트 성능 우수(P)
복잡한 조건 필터링 + 검색	Qdrant, Weaviate	메타쿼리 조합 우수(Q), GraphQL 기반 조건 표현(W)
클라우드 기반 서비스	Pinecone, Milvus	SaaS형 운영 최적화(P), K8s 통한 클라우드 통합(M)
커스터마이징이 필요한 엔터프라이즈	Vespa, Milvus	세밀한 설정 및 고가용성 아키텍처를 직접 제어 가능

❑ '무엇이 최고인가' 보다 "우리 목적에 맞는 최선의 선택은 무엇인가"를 고민

❑ **어떤 목적과 환경에서 사용하는가**에 따라 적합한 VectorDB를 선정해야 함

[참고] WHICH ONE????

PREPROCESSING



❶ Document Loader 150

❷ Text Splitter 10

❸ Embedding 60+

❹ Vector Store 80

조합 : 7,200,000+

2-(5). Retriever

- 저장된 벡터DB에서 사용자의 질문과 관련된 문서를 검색하는 과정 → 정보 검색의 질을 결정하는 핵심적인 역할
- 사용자 질문에 가장 적절한 Context를 제공하여, 언어 모델이 보다 정확한 답변을 생성할 수 있도록 함
- https://docs.langchain.com/oss/python/langchain/retrieval

질문의 벡터화

- 사용자의 질문을 벡터 형태로 변환 (문서 임베딩과 같인 모델 적용해야 함)
- 변환된 질문 벡터는 후속 검색 작업의 기준점으로 사용

↓

벡터 유사성 비교

- 저장된 문서 벡터들과 질문 벡터 사이의 유사성을 계산
- Cosine Similarity, MMR(Max Marginal Relevance) 등의 수학적 방법 사용

→ (A) Hybrid Retriever
: 키워드 + 의미 검색

↓

상위 문서 선정

- 계산된 유사성 점수를 기준으로 상위 k개의 가장 관련성 높은 문서를 선정
- 선정된 문서들은 다음 단계에서 사용자의 질문에 대한 답변을 생성하는데 사용

→ (B) Reranker
: 문서 검색 재정렬

↓

문서 정보 반환

- 선정된 문서들의 정보를 다음 단계인 Prompt 생성으로 전달함
- 반환할 때에는 문서의 내용, 위치, 메타데이터 등이 포함될 수 있음

[참고] Cosine Similarity

문서 간, 또는 질문-문서 간의 유사성 평가

- 두 벡터 간의 코사인을 계산하여 유사도를 측정하는 기법
- 벡터의 크기가 아닌 방향에 초점을 맞추므로 주로 문서와 쿼리 간의 유사도를 비교하는데 사용
- (비유) 손전등의 밝기(=길이)가 다르더라도, 비추는 방향이 같으면 코사인 유사도는 높다



- 1 : 두 벡터의 방향이 완전히 같음 (최대 유사)
- 0 : 두 벡터가 직교 (관련 없음)
- -1 : 두 벡터의 방향이 완전히 반대 (최대 상이)

Embedding

↓

벡터 정규화

- 방향만 유지하고 벡터의 크기를 1로 만드는 단계
- 단위 벡터로 변환하는 과정 (normalized vector)

↓

유사도 계산

- $Cosine\ Similarity = \frac{\vec{A} * \vec{B}}{\|\vec{A}\| \|\vec{B}\|}$

공식 해석

- (분자) Dot Product, 내적
  - 선형대수에서 두 벡터 간의 곱셈 연산 중 하나, 결과는 Scalar type
  - 두 벡터 사이의 각도와 크기를 함께 고려하여 계산
  - 두 벡터가 고차원 공간에서 "얼마나 같은 방향"을 가지는지 (방향, 크기 동시 고려)
- (분모) Norm, 노름
  - 벡터의 크기(길이) 측정 → 선형대수에서는 크기 또는 거리를 측정
  - 벡터 간 크기만 비교하도록

➔ 두 벡터의 방향 비교에만 초점 (방향+크기/크기)

↓

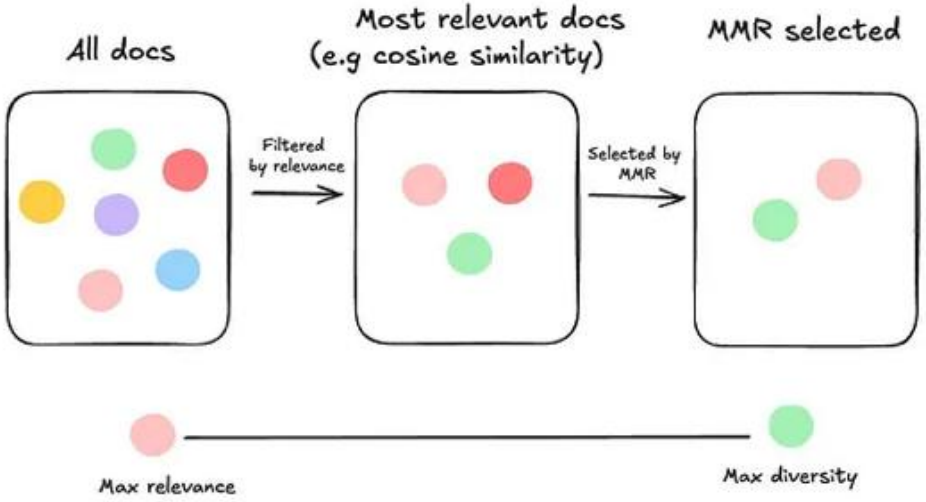
관련 문서 검색

- 코사인 유사도가 높은 상위 k개의 문서 검색

[참고] MMR

Maximal Marginal Relevance

- 이미 선택된 문서들과 새로운 문서 간의 "중복 최소화"와 "쿼리와의 유사성 최대화"를 동시에 고려하는 방법
- 검색 결과의 관련성은 유지하면서, 중복은 줄이고, 다양성은 높이는 전략



$$MMR(D_i) = \lambda \cdot Sim(D_i, Q) - (1-\lambda) \cdot maxSim(D_i, D_j)$$

- 쿼리 Q와 후보문서(D_i) 간의 유사성
- 문서가 쿼리와 얼마나 관련성이 높은지 측정

- 이미 선택된 문서(D_j)와 후보문서(D_i) 간의 유사성
- 후보 문서가 기존 문서와 얼마나 겹치는지 측정

※ λ : 유사성과 다양성 간의 가중치를 조절하는 파라미터 ($0 \leq \lambda \leq 1$)

- $\lambda = 1$: 쿼리와의 유사성만 고려 (단순 유사도 기반)
- $\lambda = 0$: 다양성만 고려 (중복성 최소화에만 집중)
- $0 < \lambda < 1$: 유사성과 다양성을 모두 고려

Source : https://medium.com/@devmessias/reducing-langchain-mmr-latency-by-90-with-better-engineering-865eef6fe767

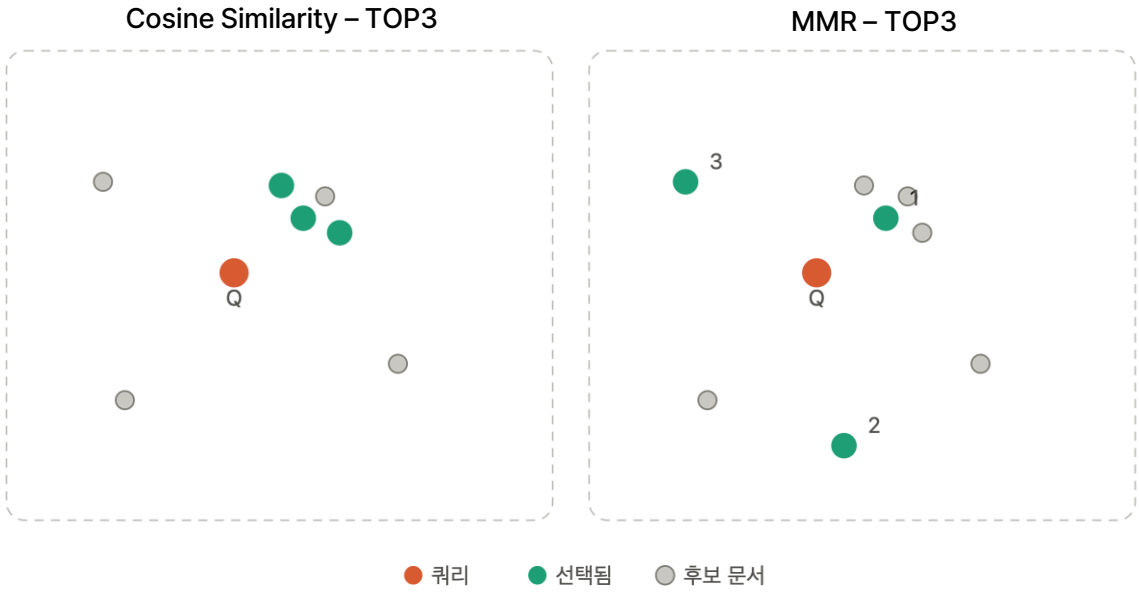
[참고] MMR, 쉽게 이해하기

Maximal Marginal Relevance

- **Marginal : '이것이 얼마나 관련 있나?' 가 아니라 "이미 가진 것에서 얼마나 새로운 걸 더하냐?"**
  - Cosine similarity : 단일 사실 질의 시
  - MMR : 여러 관점을 모아야 하는 질의 시

- **쿼리에 가까운 3개**
- **유사/중복 고려하지 않음**

→ 관련도가 높아도 각각이 더해주는 가치(marginal)는 "0"에 가까움



- **쿼리와 관련되면서 서로 다른 방향으로 퍼진 것 선택**
- 절차 :

1) 쿼리와 가장 가까운 것 하나 선택
2) 남은 후보에 대해 점수 계산
   점수 = (쿼리와 가까운 정도) – (1번과 가까운 정도)
   → 쿼리와는 가깝고,
   1)번과는 먼 문서가 높은 점수를 받음
3) 점수가 가장 높은 문서를 뽑아 추가
4) 원하는 K개가 될 때까지 2)~3)번 반복

2-(5)-A. Hybrid Retriever

가장 대표적인 조합 : Sparse + Dense Retriever

Sparse Retriever

- 문서와 질문(쿼리)를 키워드 벡터로 변환하여 처리

- 각 단어의 존재 여부만을 고려하기 때문에 계산 비용이 낮고 구현 간단
- 그러나 단어의 의미적 연관성을 고려하지 않고, 검색 결과의 품질이 키워드의 선택에 크게 의존

- TF-IDF (Term Frequency-Inverse Document Frequency)
  - 단어가 문서에 나타나는 빈도와 몇개의 문서에서 나타나는지를 반영하여 단어의 중요도를 계산
  - 자주 나타나면서도 문서 집합 전체에서 드물게 나타나는 단어가 높은 가중치를 받음

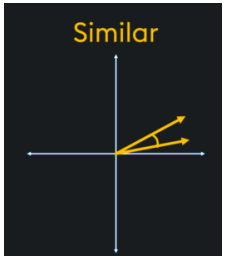
- BM25 (Best Matching 25)
  - TF-IDF 개선 모델
  - 긴 문서와 짧은 문서 간의 가중치를 조정하여, 단어 빈도의 영향을 상대적으로 조절함

Dense Retriever

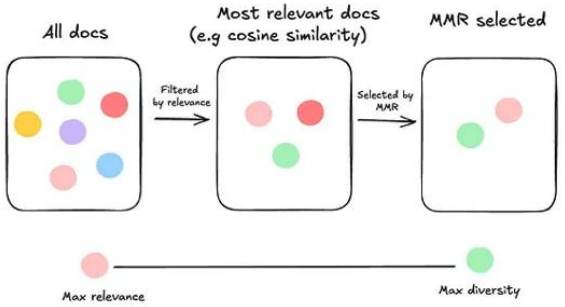
- 딥러닝 기법으로 문서와 질문(쿼리)를 고차원 벡터로 인코딩하여 문서의 의미적 내용을 보다 풍부하게 표현할 수 있음

- 키워드가 완벽히 일치하지 않더라도 의미적으로 관련된 문서 검색
- 벡터 공간에서의 거리(e.g., Cosine Similarity, MMR 등)를 사용하여 쿼리와 가장 관련성 높은 문서를 찾음
- 언어의 뉘앙스와 문맥을 이해하는데 유리하며, 복잡한 쿼리에 대해 더 정확한 검색 결과를 제공할 수 있음

Cosine Similarity

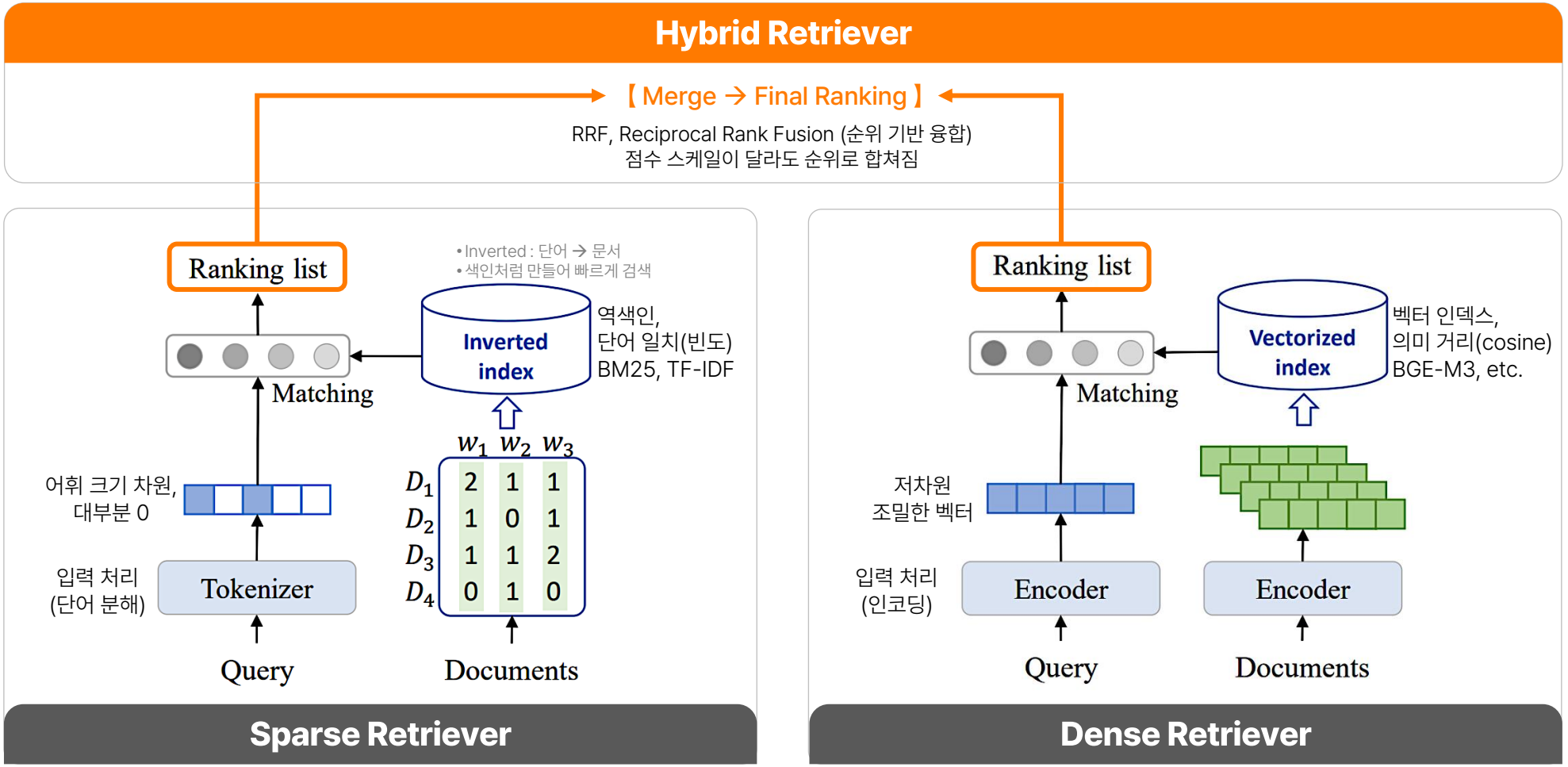


MMR



Source : https://medium.com/@devmessias/reducing-langchain-mmr-latency-by-90-with-better-engineering-865eef6fe767

2-(5)-A. Hybrid Retriever



Source : https://blog.reachsumit.com/posts/2023/09/generative-retrieval/

[참고] BM25, 너만으로 충분할까?

BM25로 먼저 Baseline 구축 → mismatch가 심하면 SPLADE 병행/검증하는 방식으로 진행

❑ BM25가 좋은 경우 :

- ○ 정확한 키워드 검색이 중요할 때
- ○ 전문 용어, 고유명사 검색
- ○ 빠른 검색 속도가 필요할 때
- ○ 컴퓨팅 자원이 제한적일 때

Query "car" ≠ Doc "vehicle"

정확한 매칭이 중요하지만
pure sparse embedding만으로는 한계

Sparse Lexical and Expansion model for First Stage Ranking

❑ SPLADE : Learned Sparse

- ○ 단어 확장 전략 – BERT MLM Head
- ○ BM25 수준의 효율성 확보 – Log-saturation, FLOPS 정규화

➔ Vocabulary mismatch 완화

긍정적 시각 :

- 수십 년간 검증된 안정적인 Baseline
- CPU만으로도 동작, 처리 속도 빠름
- 왜 이 문서가 선택되었는지 설명 가능
- 낯선 토큰도 정확 매칭으로 찾아낼 수 있음

운영 :

- 거버넌스, 유지보수 부담이 적어 최소 복잡도로 견고한 Baseline

긍정적 시각 :

- 인덱싱 시점에 쿼리/문서 양쪽을 의미적으로 관련 용어로 확장
- 벤치마크에서 BM25 보다 대체로 우수한 성능 보고
- 다국어, 짧은 텍스트에서도 우수

운영 :

- 인덱싱에 트랜스포머 추론(GPU) 필요 – 인프라 비용
- 모델 레지스트리, 데이터 리니지, 카나리 배포 등 거버넌스 요구 – 엔지니어링 성숙도가 요구되는 선택지

(※) Data Lineage : 데이터가 어디서 생성되어 어떤 과정을 거쳐 지금 형태가 됐는지 추적하는 이력 지도.
특정 검색 결과가 이상하면, 이 데이터가 원본 문서 → 어떤 전처리 → 어떤 모델을 거쳤는지 거슬러 올라가 원인을 찾을 수 있게 해 줌

(※) Canary Deployment : 새로운 시스템(SPLADE) 전체를 한번에 적용하지 않고, 일부 트래픽에만 먼저 적용해 문제가 없다고 확인한 뒤에 점진적으로 확대하는 배포 방식

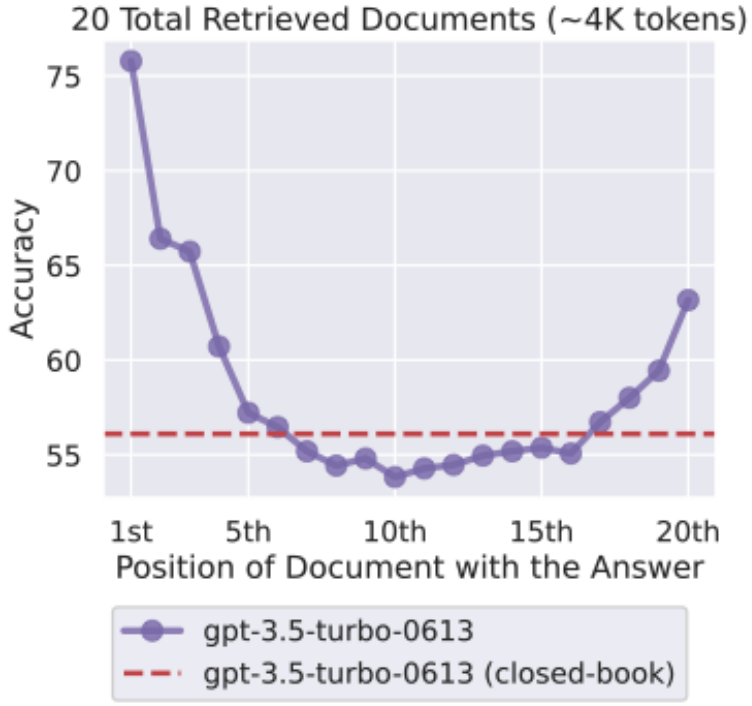
2-(5)-B. Reranker

기존의 중요도 기준으로 내림차순 정렬의 한계점 지적

- Lost in the Middle : How Language Models Use Long Contexts (2023)

Abstract

While recent language models have the ability to take long contexts as input, relatively little is known about how well they *use* longer context. We analyze the performance of language models on two tasks that require identifying relevant information in their input contexts: multi-document question answering and key-value retrieval. We find that performance can degrade significantly when changing the position of relevant information, indicating that current language models do not robustly make use of information in long input contexts. In particular, we observe that performance is often highest when relevant information occurs at the beginning or end of the input context, and significantly degrades when models must access relevant information in the middle of long contexts, even for explicitly long-context models. Our analysis provides a better understanding of how language models use their input context and provides new evaluation protocols for future long-context language models.

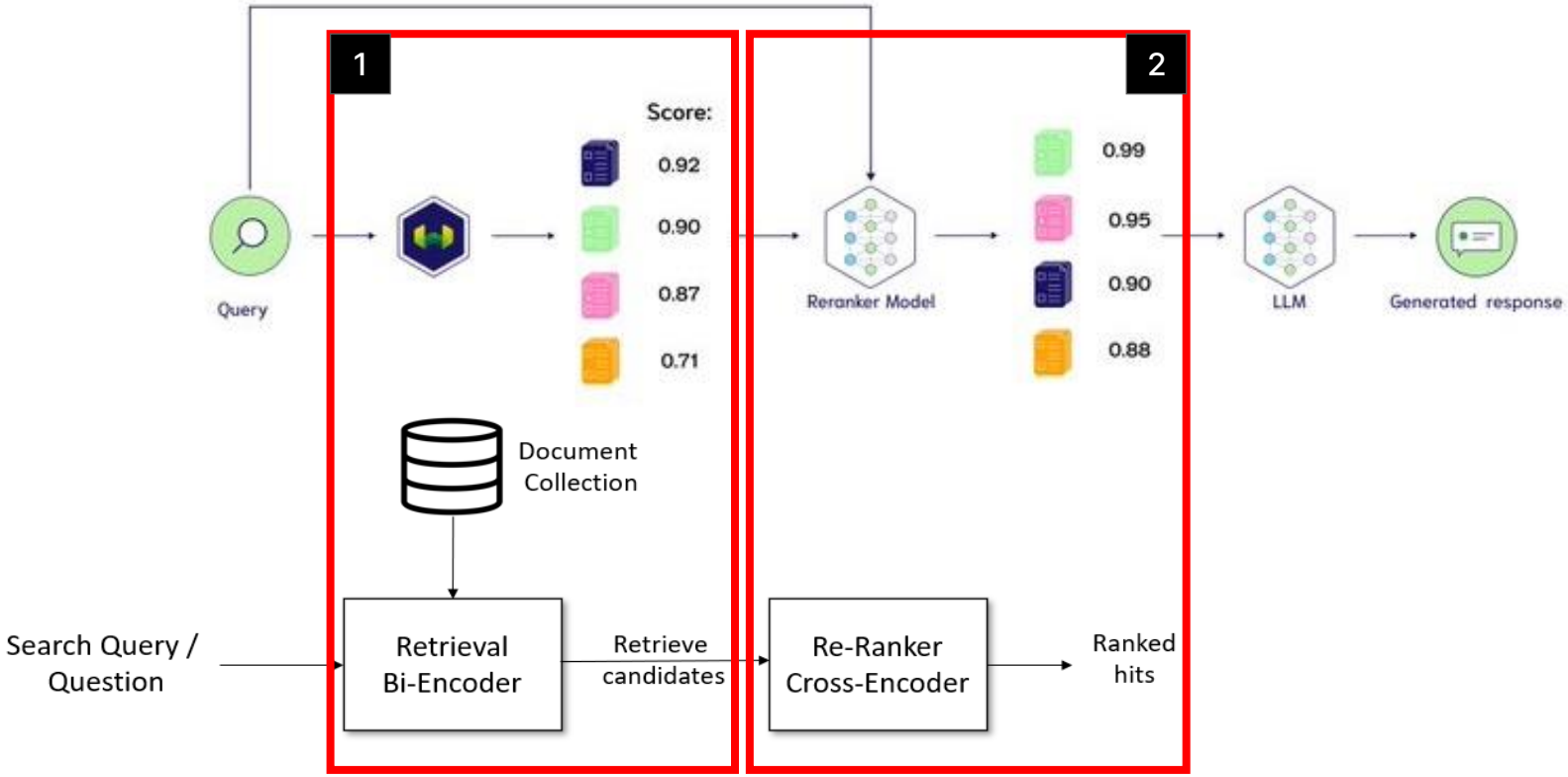


Source : *https://arxiv.org/pdf/2307.03172*

2-(5)-B. Reranker

Retriever 통해 검색된 결과의 정확도를 향상시키기 위해 두 단계로 구성된 검색 시스템 (Two-Stage Retrieval System)

1. 대규모 문서 집합에서 관련성 있는 후보 문서들을 빠르게 추출한 후,
2. Reranker는 추출된 후보 문서들을 더 정교하게 분석하여 최종적인 순위를 결정하는 방식임 (feat. Lost-in-Middle)

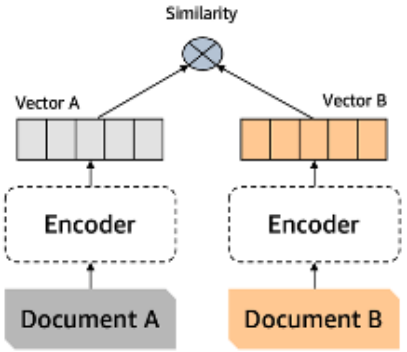


Reference: https://x.com/ecardenas300/status/1732472798272974915

2-(5)-B. Reranker

작동 방식 : "의미적 정밀도"를 더해서 사용자가 원하는 정답을 찾을 수 있도록 !!!

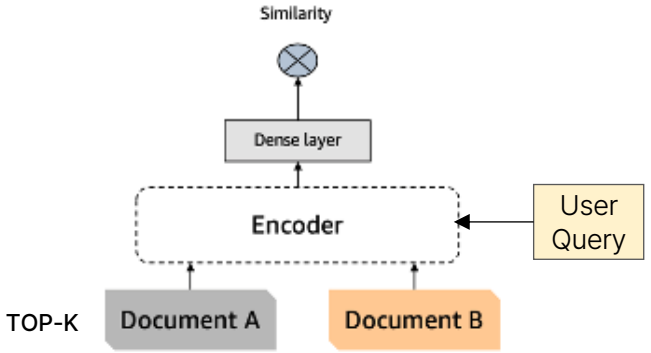
Retriever (Bi-Encoder)



1. 문서와 질문을 각각 벡터로 인코딩
2. 벡터 간의 유사도를 계산하여 관련성 측정
3. 유사도가 높은 상위 K개 문서를 선택

대규모 문서 집합에서 빠르게 관련 문서를 찾을 수 있지만,
정확도가 다소 떨어질 수 있음

Reranker (Cross-Encoder)



1. Retriever 단계에서 선택된 상위 K개 문서를 입력 받음
2. 각 문서와 사용자 질문을 함께 언어모델에 입력
3. 모델은 문서와 질문 간의 관련성을 평가 → 호출 시점에만 계산
4. 평가 결과에 따라 문서들을 재정렬

문맥을 고려한 의미 분석을 수행하여 **더 정확한 결과** 제공

2-(5)-B. Reranker

Retriever vs Reranker

구분	Retriever	Reranker
주요 목적	대용량 문서 내에서 관련 문서를 빠르게 검색	정확한 문서 순위 조정
우선 순위	속도	정확도
(예시)	카페 추천 → 유사 단어 포함한 문서 Top-N 검색	→ 내가 원하는 분위기의 카페를 1순위로 올림
Output	후보 문서 집합	후보 문서 내에서 정확한 순위와 점수
처리 방식	Embedding vector 비교 → Cosine Similarity	쿼리-문장 쌍의 의미 비교 → Relevance Score
모델 구조	Bi-Encoder - 질문과 문서를 각각 따로 임베딩 → 임베딩 벡터 비교	Cross-Encoder - 질문과 문서를 함께 입력하여 비교 → 의미간 관계 고려
주요 사용 모델	BGE, E5, Jina-Embedding 등	Cohere-Reranker, bge-reranker, Jina-Reranker 등
장점	• 처리 속도가 빠름 (병렬 임베딩 처리 가능) • 대규모 문서 검색에 필수	• 정밀도 높음 (문맥 이해 기반) • RAG 시스템에 필수
단점	• 의미 정확도가 떨어질 수 있음 • 결과 품질이 낮을 수 있음	• 상대적으로 느림 • 계산 비용 높음

[참고] MRR

Reranker 평가 지표 : Mean Reciprocal Rank, MRR

- Reciprocal : 역수 (1/num)
- Mean Reciprocal : 여러 질문에서 정답이 몇 등에 있었는지를 역수로 계산해서 평균 낸 값
- MRR : 사용자가 원하는 답(정답)이 얼마나 앞에 나왔는지를 수치로 나타낸 성능 평가 지표

- 계산방식 = 1 / 정답이 나온 순위
  - 정답이 1번째 있으면 → 1/1 = 1.0
  - 정답이 2번째 있으면 → 1/2 = 0.5
  - 정답이 5번째 있으면 → 1/5 = 0.2

  ➔ 여러 질문에 대한 Reciprocal Rank 평균값 = MRR

Q1) MRR 평가를 위해 라벨이 필요한가요? → 성능 평가를 해야 하기 때문에 정답 라벨이 필요합니다. (이 질문에 대해서 이 문서가 정답, Ground Truth)
Q2) 그렇다면 MRR 평가는 Supervised 인가요? → MRR은 학습이 아닌, 최종 결과의 정답 위치를 기준으로 평가하는 것입니다

[참고] Reranker Benchmark

Benchmark (2023.11)

※ Hit Rate : 정답이 Top-K 결과 안에 포함된 비율 (정답 포함 여부에 초점)
※ MRR, Mean Reciprocal Rank : 정답이 얼마나 위에 있는지를 평균적으로 평가한 값 (정확한 순서에 초점)

Embedding	WithoutReranker		bge-reranker-base		bge-reranker-large		Cohere-Reranker	
	Hit Rate	MRR	Hit Rate	MRR	Hit Rate	MRR	Hit Rate	MRR
OpenAI	0.876404	0.718165	0.91573	0.832584	**0.910112**	**0.855805**	**0.926966**	**0.86573**
bge-large	0.752809	0.597191	0.859551	0.805243	0.865169	0.816011	0.876404	0.822753
llm-embedder	0.814607	0.587266	0.870787	0.80309	0.876404	0.824625	0.882022	0.830243
Cohere-v2	0.780899	0.570506	0.876404	0.798127	0.876404	0.825281	0.876404	0.815543
Cohere-v3	0.825843	0.624532	0.882022	0.806086	0.882022	0.834644	0.88764	0.836049
Voyage	0.831461	0.68736	0.926966	0.837172	**0.91573**	**0.858614**	**0.91573**	**0.851217**
JinaAI-Small	0.831461	0.614045	0.91573	0.843071	**0.926966**	**0.857303**	**0.926966**	**0.868633**
JinaAI-Base	0.848315	0.68221	0.938202	0.846348	**0.938202**	**0.868539**	**0.932584**	**0.873689**
Google-PaLM	0.865169	0.719476	0.910112	0.833708	**0.910112**	**0.85309**	**0.910112**	**0.855712**

A. WithoutRanker (기본 검색만) : 기본 임베딩 성능만 사용한 경우
B. bge-reranker-base : A 보다 향상된 성능
C. bge-reranker-large : B 보다 향상되고, 특히 OpenAI 임베딩과 함께 사용할 때 MRR 0.85 수준으로 크게 향상
D. Cohere-Reranker : 적용 시 가장 우수한 성능 (※ 최고 성능 = JinaAI-Base + Cohere-Rernaker)

[참고] bge-reranker, BAAI General Embedding : 문장 간 의미 차이를 파악해서 결과 재정렬 (cosine similarity 기반)
Cohere-Reranker : 사용자의 질문과 문서 사이의 의미적 유사도를 평가해서, 더 좋은 검색 순서를 만들어줌 (relevance score 방식)

Source : https://www.llamaindex.ai/blog/boosting-rag-picking-the-best-embedding-reranker-models-42d079022e83

[참고] Reranker Benchmark

Benchmark (2024.03.08)

성능이 개선된 Embedding + Reranker 조합

Embedding	개선된 Reranker	기존 MRR	개선된 MRR	개선 정도
bge-large	**bge-m3-reranker**	**0.8227**	**0.878**	**+0.055**
Cohere-v3	**Cohere-Reranker v3**	**0.8364**	**0.872**	**+0.036**
JinaAI-Small	Jina-Reranker-v2	0.8666	0.874	+0.007
JinaAI-Base	Jina-Reranker-v2	0.8736	0.878	+0.004
JinaAI-Base	bge-m3-reranker	0.8736	0.878	+0.004
Cohere-v2	Cohere-Reranker v3	0.8280	0.839	+0.011
Voyage	Jina-Reranker-v2	0.8512	0.843	(+)Hit Rate

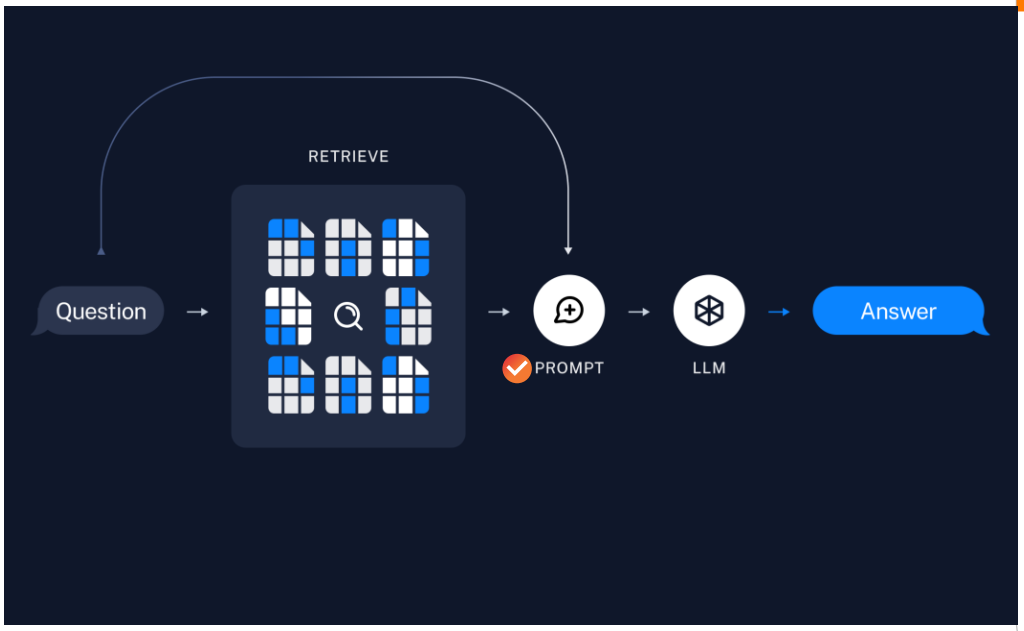
[참고]

- bge-m3-reranker
  - bge-reranker-large : Reranker 성능 중심
  - bge-m3-reranker : Embedding + Reranking + Cross-lingual 성능 통합 (M3 = Multi-task, Multi-lingual, Multi-modal)
- Cohere-Reranker : 최신 트랜드를 반영한 v3 모델 (LLM 기반의 의미 파악 최적화 + Zero/Few-shot 대응 강화 + Semantic Nuance 정교화)
- E5-Large-V2 / E5-Jina-v2 : Embedding + Reranker 겸용, 고성능 모델 (높은 성능, 빠른 처리 속도, 타스크 호환성 등)

Source : https://huggingface.co/spaces/mteb/leaderboard

2-(6). Prompt

- 검색기(Retriever)에서 검색된 문서들을 바탕으로 언어 모델이 사용할 질문이나 명령을 생성하는 과정
- 검색된 정보를 바탕으로 최종 사용자의 질문에 가장 잘 대응할 수 있는 응답을 생성하기 위한 필수적인 단계
- https://docs.langchain.com/langsmith/create-a-prompt



Prompt 필요성

1. 문맥 설정, Context : 프롬프트는 언어모델이 특정 문맥에서 작동하도록 설정하는 역할을 하는데, 이 때 모델은 제공된 정보(Context)를 바탕으로 보다 정확하고 관련성 높은 답변을 생성할 수 있음

2. 정보 통합 : 여러 문서에서 검색된 정보는 서로 다른 관점이나 내용을 포함하는데, 프롬프트를 통해 여러 정보를 통합하고, 모델이 이를 효율적으로 활용할 수 있는 형식으로 조정함

3. 응답 품질 향상 : 질문에 대한 모델의 응답 품질은 프롬프트의 구성에 크게 의존함. 잘 구성된 프롬프트는 모델이 보다 정확하고 유용한 정보를 제공함

2-(6). Prompt

- https://docs.langchain.com/langsmith/create-a-prompt

```
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate.from_template(
"""당신은 질문-답변(Question-Answer) Task를 수행하는 AI Assistant 입니다.
검색된 문맥(context)를 사용하여 질문(question)에 답하세요.
만약, 문맥(context)으로부터 답을 찾을 수 없다면 '모른다'고 말하세요.
한국어로 대답하세요.

# Question :
{question}

# Context :
{context}

# Answer : """
)
```

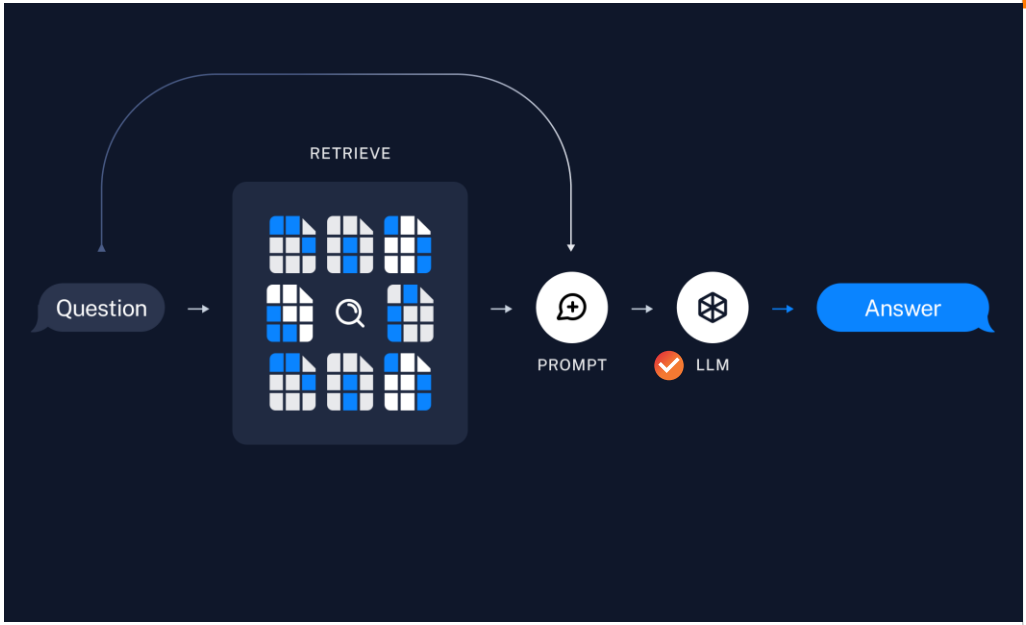
지시사항 (Instruction)

질문 (사용자 입력 질문)

문맥 (검색된 정보)

2-(7). LLM

- Prompt 단계에서 구성된 입력을 기반으로 대규모 언어 모델을 활용하여 응답을 생성하는 과정
- 언어 모델의 능력을 최대한 활용하여 사용자의 질문에 대해 정확하고 자연스러운 답변을 생성함
- https://docs.langchain.com/oss/python/langchain/models



LLM 필요성

1. 사용자 의도 이해 : 자연어 이해(NLU)와 자연어 생성(NLG) 능력이 결합되어, 보다 자연스럽고 적합한 응답을 제공할 수 있음

2. 문맥적 적응성 : LLM은 주어진 문맥을 고려하여 응답을 생성함. 또한 사전 학습된 지식 외 사용자가 제공한 정보에 기반한 답변을 문맥을 참고하여 답변하도록 함

LLM 단계는 필요 데이터와 Context를 종합하여 사용자의 질문에 대한 답변의 질과 자연스러움 반영

2-(7). LLM

- https://docs.langchain.com/oss/python/langchain/models

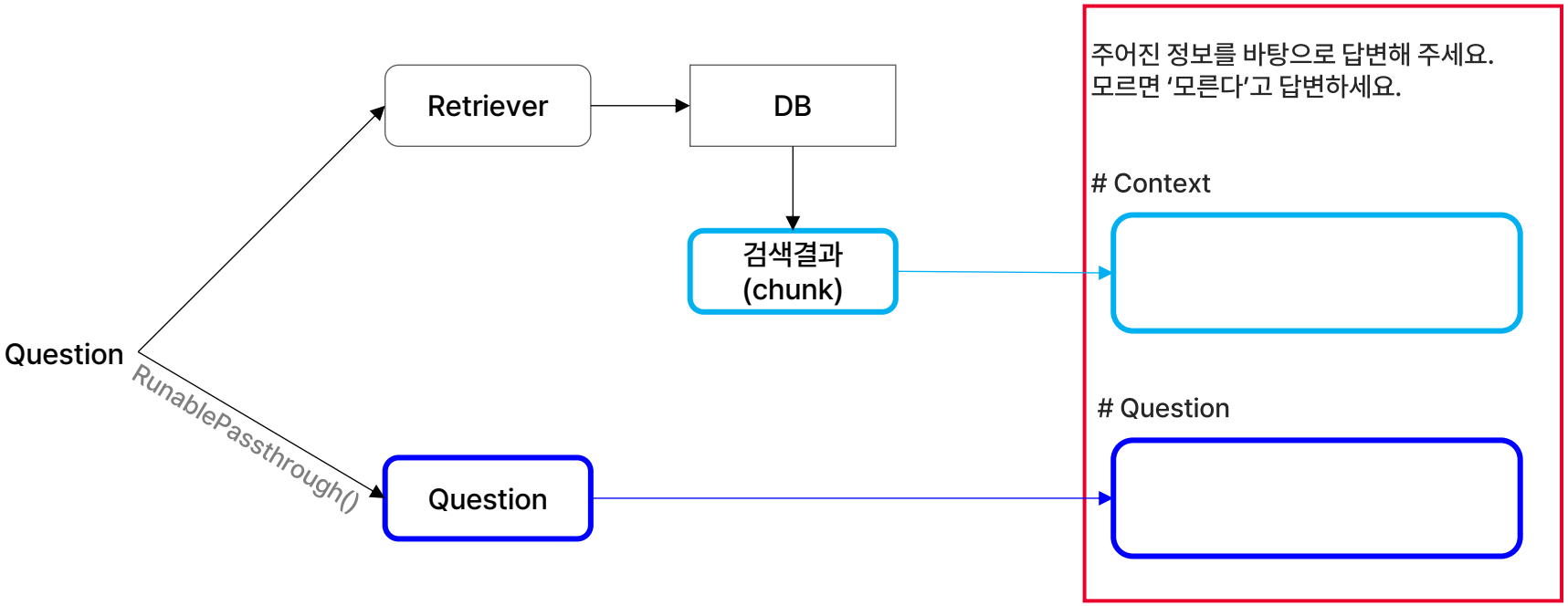
```
# Setup : API KEY


# LangChain (Pre)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model_name="gpt-4o-mini", temperature=0)


# LangChain (v1.0)
from langchain.chat_models import init_chat_model
model = init_chat_model("gpt-4o-mini")
```

2-(8). Chain

- 이전의 과정을 하나로 묶어 RAG Pipeline으로 조립하여 완성하는 단계



2-(8). Chain

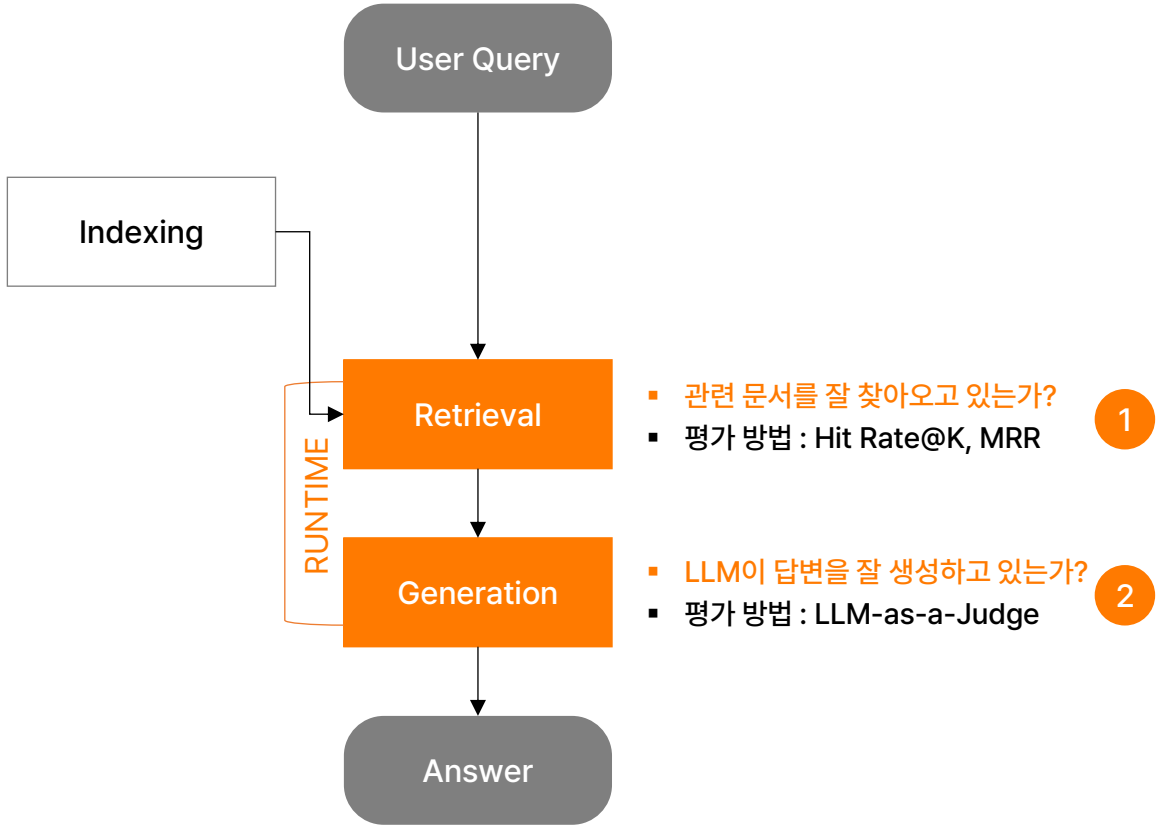

```
# Chain 생성
# chain = prompt | llm
# chain = prompt | llm | StrOutputParser()

chain = (
   {“context”: retriever, “question”: RunnablePassthrough()}
   | prompt
   | llm
   | StrOutputParser()
)

# Chain 실행
question = “A에 대해 설명해 주세요.”
response = chain.invoke(question)
```

3. RAG 평가

평가 목적 : 성능 좋은 LLM이라 하더라도, 검색 단계에서 관련 문서를 가져오지 못하면 좋은 답변을 기대할 수 없음



3-(1). 평가 방법 – Retrieve

정답 문서가 포함되어 있고 & 몇 번째 위치하는지를 함께 봐야 함

Hit Rate@K (적중률)

상위 K개의 검색 결과 안에

정답 문서가 포함되어 있는가?

$$\text{Hit Rate@K} = \frac{\text{상위}K\text{개 결과 안에 정답이 있는 질문 수}}{\text{전체 질문 수}}$$

예시 (K=3, 질문 4개) :

질문	1위	2위	3위	정답 위치	Hit?
Q1	정답	X	X	1위	O
Q2	X	정답	X	2위	O
Q3	X	X	정답	3위	O
Q4	X	X	X	K이하 없음	X

→ Hit Rate@3 = 3/4 = 0.75

➔ 단순하고 직관적
그러나 정답이 어디 위치에 있는지는 구분하지 못함

MRR, Mean Reciprocal Rank

정답 문서가

검색 결과의 몇 번째에 나오는가?

MRR :

- 각 질문에서 정답 문서의 순위를 rank_i 라고 하면
- Reciprocal Rank = 1/rank_i (역수)
- MRR : RR의 평균

예시 :

- Q1 : 1/1 = 1.0
- Q2 : 1/2 = 0.5
- Q3 : 1/3 = 0.333
- Q4 : 1/0 = 0 (정답 없음)

$$\text{MRR} = \frac{(1.0+0.5+0.333+0)}{4} = 0.458$$

➔ 정답이 얼마나 앞 순위에 나오는지를 반영

➔ 같은 Hit Rate이라도 MRR이 높으면 더 좋은 검색기

3-(1). 평가 방법 – Generation

LLM-as-a-Judge

- System Prompt 통해 LLM에게 평가자 역할 부여
- 애매하거나 관대한 평가가 되지 않도록 명확한 기준을 제공해야 함. 필요 시 해당 도메인/과업의 주요 평가 체계를 확인해야 함

```
Task: Rate the assistant's answer based on how correct and relevant it is.

Criteria:
• Correctness (1–5) – Does it align factually with the context?
• Relevance (1–5) – Does it directly address the question?

Steps:
1. Read the question and context.
2. Compare the answer with the context.
3. Assign a relevance and correctness score (1–5).

Question: {question}
Context: {context}
Answer: {answer}

Provide:
• Correctness score:
• Relevance score:
• Explanation:
```

프롬프트 구성 시 주의사항

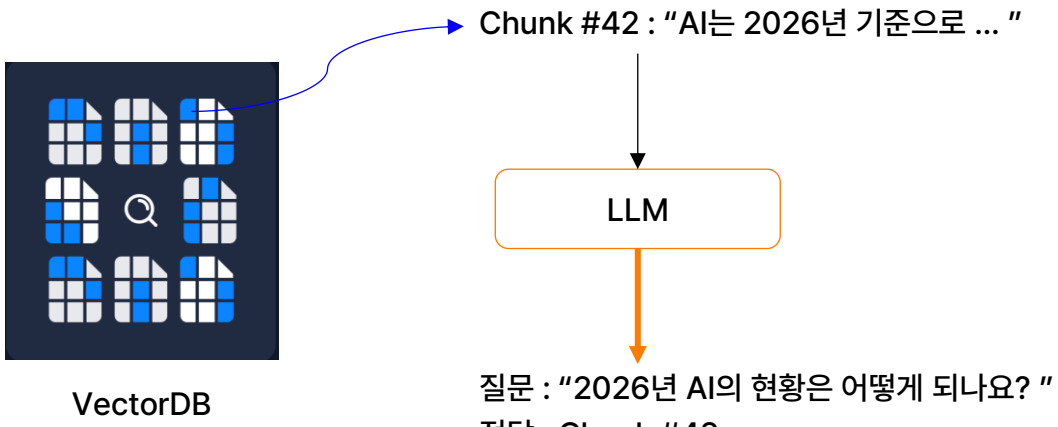
명확한 기준 : 정밀히 나눈 평가 항목 (정확성, 충실성, 일관성, 유창성 등) 제시

항목	내용
정확성, Correctness	응답이 사실과 일치하는지 평가, 사실적 오류가 없는지 확인
충실성, Faithfulness	응답이 참고 문서 정보에 기반하고 있는지 평가, Hallucination 발생 시 감점
관련성, Relevance	응답이 질문에 직접적으로 답변하고 있는지 평가, 문맥이나 질문과 상관없는 내용이 포함되면 감점
일관성, Consistency	응답 내 논리적 흐름이 자연스럽고 앞뒤가 맞는지 평가
명확성, Clarity	문장이 이해하기 쉽게 명확히 표현되었는지 평가
유창성, Fluency	문장이 자연스럽고 매끄럽게 연결되는지 평가 (표현, 단어, 말투 등)
간결성, Conciseness	불필요한 반복이나 장황한 설명 없이 간단 명료하게 핵심을 전달하는지 평가

- 출력 양식 통제 : 점수, 설명을 포함하여 Form-filling 구조 권장
- 반복 & 교차 평가 : 동일 프롬프트로 반복 수행 또는 Multi-LLM 통한 교차 평가를 수행한 결과를 비교하여 편차 최소화

3-(2). Retrieve 평가 데이터셋

아이디어 : 정답 청크를 LLM에게 주고, 해당 내용으로 답할 수 있는 질문을 생성하도록



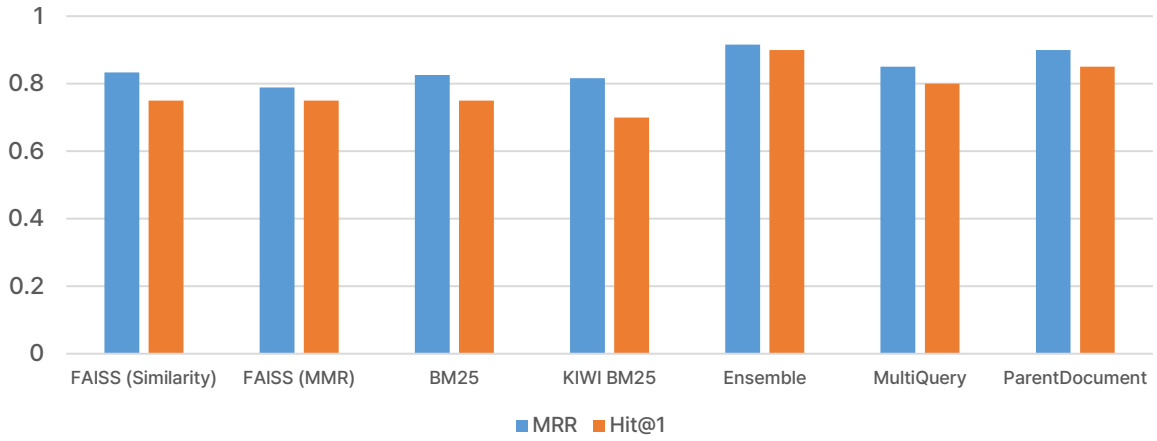
➔ 이렇게 만들어진 (질문, 정답청크ID) 쌍이 Ground Truth로 활용

3-(3). Retrieve 평가 결과 확인

내 데이터에 적합한 Retrieve는 무엇인가를 확인하는 중요한 단계 !!!

최종 평가 결과 (예시) :

Retriever	특징	Hit@1	Hit@3	Hit@5	MRR
FAISS (Similarity)	기본 벡터 유사도 검색	0.750	0.950	0.950	0.833
FAISS (MMR)	다양성 고려한 검색	0.750	0.800	0.900	0.789
BM25	키워드 기반 검색 (영문 토크나이저)	0.750	0.900	0.950	0.826
Kiwi BM25	키워드 기반 검색 (한국어 형태로 토크나이저)	0.700	0.950	0.950	0.816
Ensemble (BM25 + FAISS)	키워드 + 의미 기반 검색 결합	0.900	0.950	0.950	0.916
MultiQuery Retriever	LLM에게 질문 다각화 후 검색	0.800	0.950	0.950	0.850
ParentDocumentRetriever	작은 청크로 검색 후 큰 문서 반환	0.850	0.950	0.950	0.900



[Best] Ensemble (BM25 + FAISS)

- MRR : 0.916
- Hit@1 : 0.900

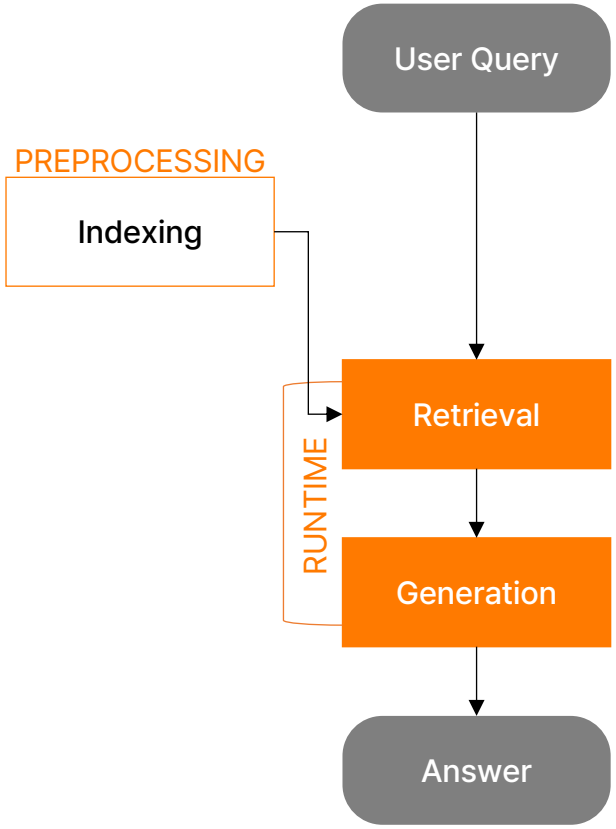
2. RAG A to Z

SUMMARY

SUMMARY

1. RAG 방법론

가장 기본적인 형태의 RAG 방법론



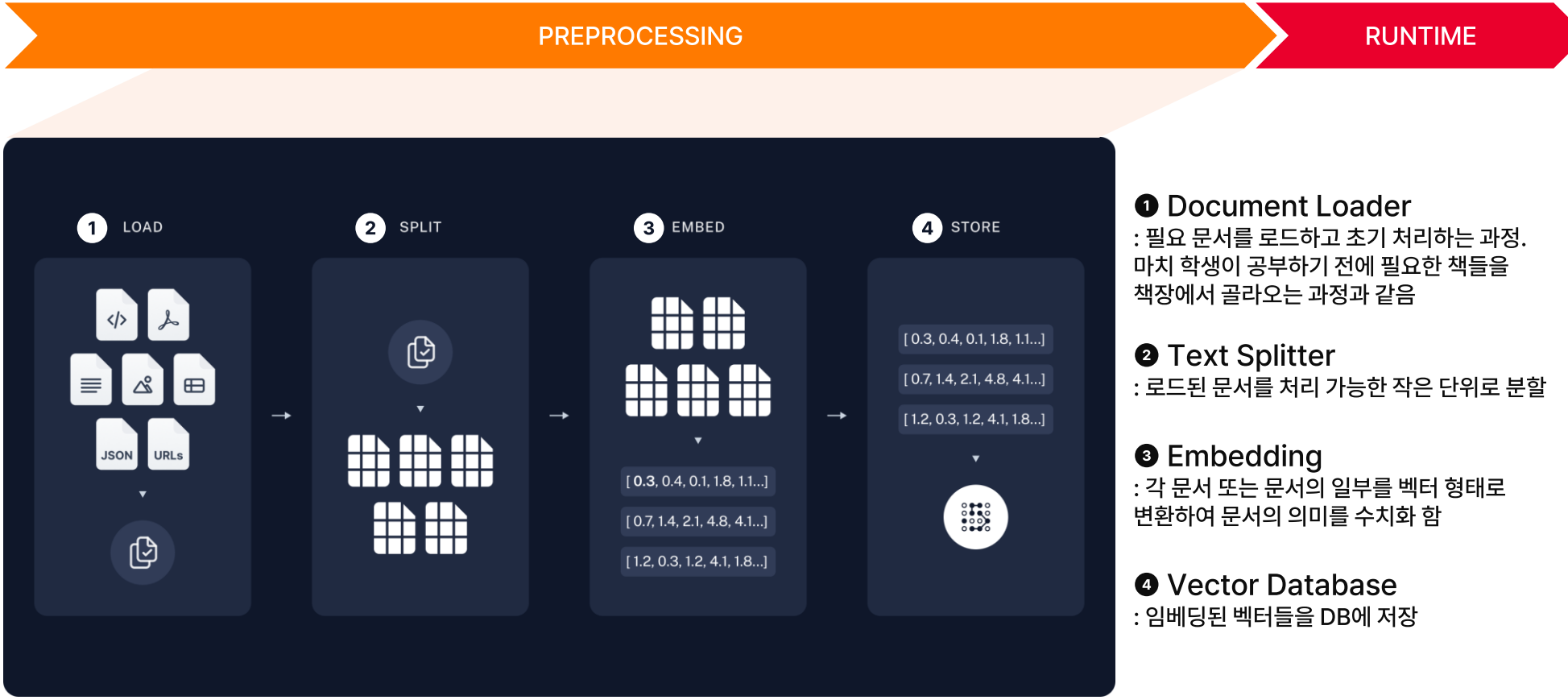
- 지식 베이스를 만드는 단계
- 원본 데이터 정제 → 텍스트 추출 → Chunk 분할 → 벡터 변환 (→ Vector DB 저장)

Pre-processing | Indexing

- 사용자 질문 기반으로 문서 검색
- 질문 벡터화 → 유사도 기반 상위K개의 청크 검색

- 검색된 K개의 청크와 사용자 질문을 결합하여 LLM에 Prompt로 전달
- LLM은 입력 받은 Context 기반으로 최종 응답 생성

2. RAG 프로세스



❶ Document Loader
: 필요 문서를 로드하고 초기 처리하는 과정.
마치 학생이 공부하기 전에 필요한 책들을
책장에서 골라오는 과정과 같음

❷ Text Splitter
: 로드된 문서를 처리 가능한 작은 단위로 분할

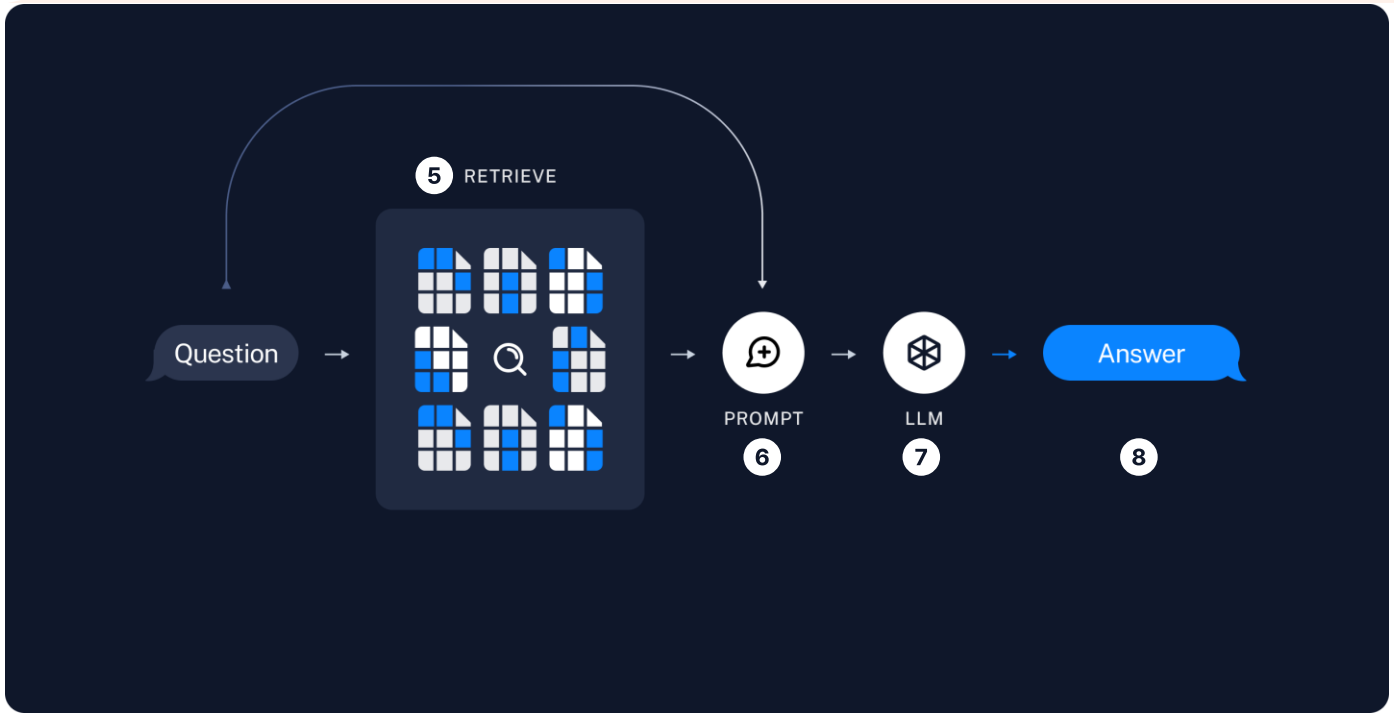
❸ Embedding
: 각 문서 또는 문서의 일부를 벡터 형태로
변환하여 문서의 의미를 수치화 함

❹ Vector Database
: 임베딩된 벡터들을 DB에 저장

Source : https://velog.io/@jjlee6496/RAG란

2. RAG 프로세스

PREPROCESSING | RUNTIME



❺ Retriever
: 질문이 주어지면, 이와 관련된 벡터를 벡터DB에서 검색함. 질문에 가장 잘 맞는 책의 챕터를 찾는 것과 유사함

❻ Prompt
: 검색된 정보를 바탕으로 언어 모델을 위한 질문을 구성함. 이는 정보를 바탕으로 어떻게 질문할지 결정하는 과정임

❼ LLM
: 구성된 프롬프트를 사용하여 언어 모델이 답변을 생성함

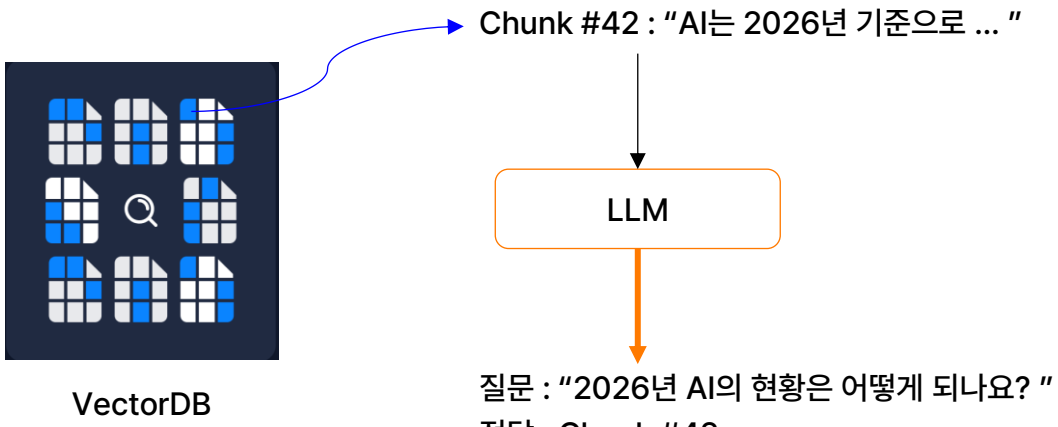
❽ Chain
: 앞선 모든 과정을 하나의 파이프라인으로 묶어주는 체인을 생성함

Source : https://velog.io/@jjlee6496/RAG란

SUMMARY

3-(2). Retrieve 평가 데이터셋

아이디어 : 정답 청크를 LLM에게 주고, 해당 내용으로 답할 수 있는 질문을 생성하도록



➔ 이렇게 만들어진 (질문, 정답청크ID) 쌍이 Ground Truth로 활용

SUMMARY

3-(1). 평가 방법 – Retrieve

정답 문서가 포함되어 있고 & 몇 번째 위치하는지를 함께 봐야 함

Hit Rate@K (적중률)

상위 K개의 검색 결과 안에

정답 문서가 포함되어 있는가?

$$\text{Hit Rate@K} = \frac{\text{상위}K\text{개 결과 안에 정답이 있는 질문 수}}{\text{전체 질문 수}}$$

예시 (K=3, 질문 4개) :

질문	1위	2위	3위	정답 위치	Hit?
Q1	정답	X	X	1위	O
Q2	X	정답	X	2위	O
Q3	X	X	정답	3위	O
Q4	X	X	X	K이하 없음	X

→ Hit Rate@3 = 3/4 = 0.75

➔ 단순하고 직관적
그러나 정답이 어디 위치에 있는지는 구분하지 못함

MRR, Mean Reciprocal Rank

정답 문서가

검색 결과의 몇 번째에 나오는가?

MRR :

- 각 질문에서 정답 문서의 순위를 rank_i 라고 하면
- Reciprocal Rank = 1/rank_i (역수)
- MRR : RR의 평균

예시 :

- Q1 : 1/1 = 1.0
- Q2 : 1/2 = 0.5
- Q3 : 1/3 = 0.333
- Q4 : 1/0 = 0 (정답 없음)

$$\text{MRR} = \frac{(1.0+0.5+0.333+0)}{4} = 0.458$$

➔ 정답이 얼마나 앞 순위에 나오는지를 반영

➔ 같은 Hit Rate이라도 MRR이 높으면 더 좋은 검색기

TIP 요약 기반 문서 검색

문서 검색을 더 효율적으로 하기 위해 '요약된 내용'을 기반으로 검색하는 전략이 필요할 수 있다 !

요약 기반 검색

- 목적 : 검색 속도와 관련도 향상
- 요약 방식 : 짧고 핵심적인 요약 → 요약문 기반 검색용
- 적용 : 벡터 저장 전 요약 단계

문서 수집 및 로딩

↓

문서 요약 생성
- LLM 기반 요약. 프롬프트로 요약 목적 명시
- 문서가 긴 경우에는 Map-Reduce, Map-Refine, Clustering Map-Reduce 등을 통해 요약본 생성

↓

요약 저장
- 원문 + 요약 같이 저장 : 메타데이터에 summary 필드
- 또는 요약만 별도 저장

↓

요약 기반 벡터 검색
- 사용자 질문 임베딩
- 요약문과 유사도 비교하여 연관 요약문 검색

↓

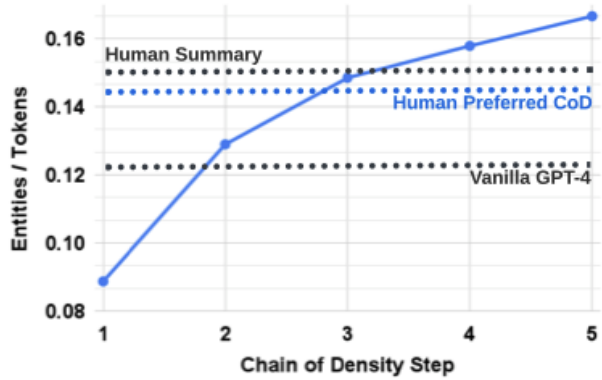
응답 생성
- 선택된 요약문 또는 원문 기반으로
- LLM이 최종 답변 생성

Chain of Density

- 목적 : 정보 밀도와 응답 품질 향상
- 요약 방식 : 점진적으로 정보 밀도를 높이는 고도화된 요약
- 적용 : 요약 생성 자체에 대한 품질 향상 전략

문서 요약 생성 단계에서 요약 품질을 고도화 할 수 있는 전략

- 처음에는 간결하게 요약하고,
- 그 다음부터 핵심 키워드를 계속 추가하면서 요약을 확장해 나가는 방식
- 핵심 키워드 누적 : 점점 더 많은 키워드가 요약문에 포함될 수 있도록



Chain 단계가 많아질수록 더 많은 정보 엔티티가 요약에 포함

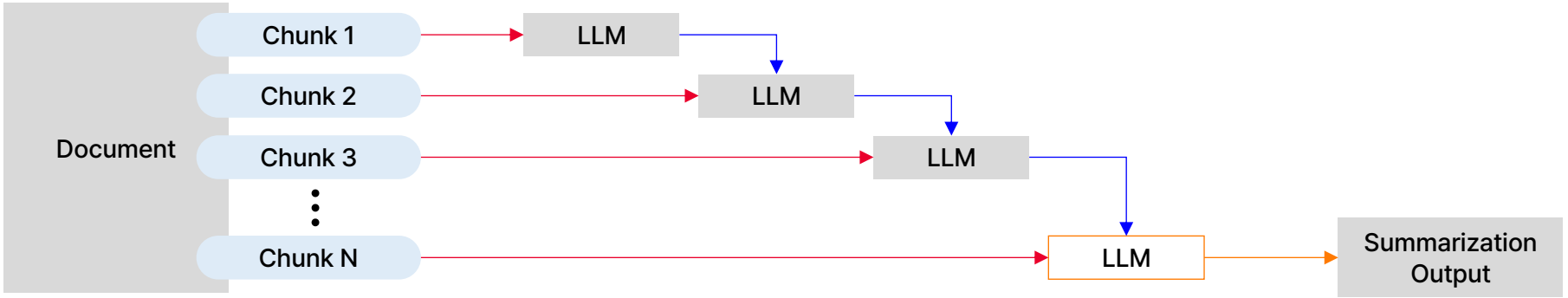
Source : https://arxiv.org/pdf/2309.04269

TIP 요약 방식

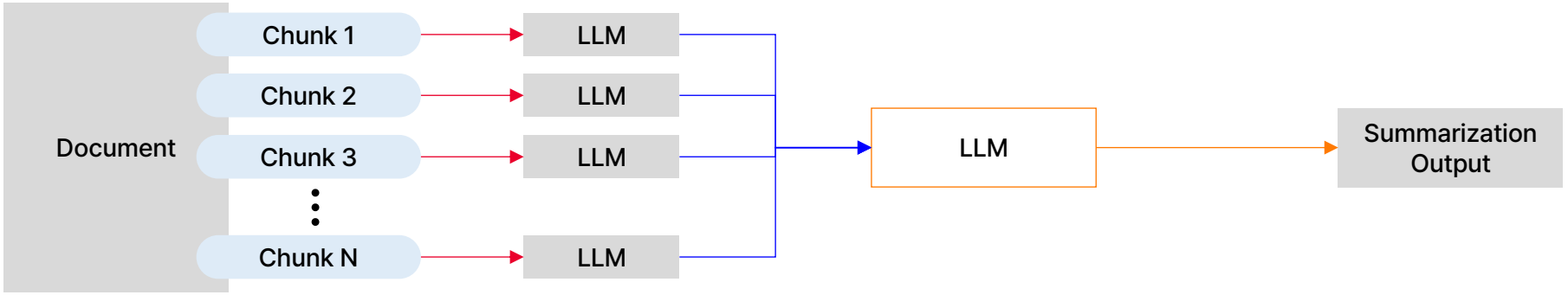
❑ Stuff (Vanilla) : (작은) 문서 전체를 LLM에 전달하여 요약하는 방식



❑ Refine : 생성된 요약들을 순차적으로 처리하여 최종 요약을 점진적으로 개선



❑ Reduce : 각 청크의 요약을 병렬처리한 후에 결합하는 방식



Source : https://arxiv.org/pdf/2309.04269

AI Campus 생성형AI

3. RAG 확장 전략

- Naïve RAG 한계점
- Advanced RAG
- Self-RAG
- Modular RAG
- Agentic RAG

※ 상세 내용은 Notion 참고

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

RAG 확장 전략

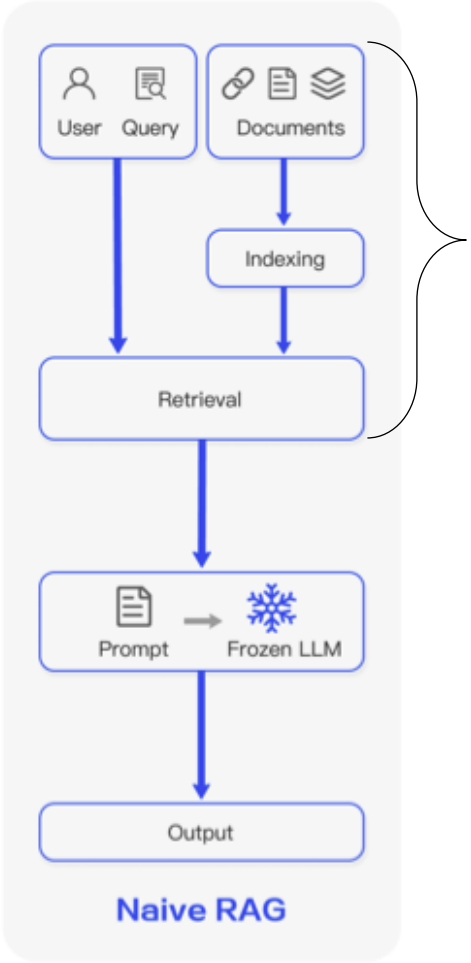
	Retrieval	Query	Structure
Flat Chunk	❑ Naïve RAG (2020) • 모델은 외부 지식을 동적으로 검색 및 활용하여 모델 파라미터에 기억을 제한하지 않음 • Pretrained Language Model 활용의 새로운 방식 제시	❑ Self-RAG (2023.10) • Self-Critiquing 통해 모델이 자체적으로 확인 • RAG 시스템을 단순 도구에서 자기 검토 능력을 갖춘 에이전트 확장	❑ Modular RAG (2024.07) • 기존의 RAG 시스템을 레고 블록처럼 재구성 가능한 프레임워크로 변환 • 각 모듈을 독립적으로 설계하고 필요에 따라 교체하거나 결합할 수 있어, 다양한 작업 시나리오에 맞게 시스템을 최적화
	❑ Advanced RAG (2023.12) • 검색 품질 문제 해결을 위한 구조적 대응 방식 제시 • 단순 Top-K 검색을 넘어 문서 재정렬(Reranking), 의미 기반 벡터 검색(Dense Retrieval), 다단계 추론 기반 검색(Multi-hop Retrieval) 등 고도화된 기술 도입	❑ Corrective RAG (2024.01) • 모델이 스스로 검색 결과의 품질을 평가 (Co ct, Incorrect, Ambiguous) • 품질이 낮을 경우 추가 검색(웹검색)으로 전환 여 생성의 정확도 개선 ❑ Adaptive-RAG (2024.03) • 질문의 유형과 난이도에 따라 적절한 검색 및 생성 전략을 스스로 조절 • 질의 복잡도를 분류하여 복잡도 따른 질의 처리 가능	❑ Agentic RAG (2025.01) 에이전트 기반으로 실시간으로 검색 전략을 조정하여 최신 정보 활용하는 동적 검색 전략 관리 시스템 • 다중 에이전트 협업 통해 복잡한 작업 처리 능력 향상 및 실시간성 강화
Graph	❑ Graph RAG (2024.04) • Naïve RAG의 Flat Chunk면 충분하다는 가정 재점검. 지식그래프/온톨로지 관점 결합해 관계를 명시적으로 색인 • 인덱싱이 핵심이지만 Retrieval + Query까지 작용 • 관계 기반으로 Multi-hop 질의에서 유효. 실무에서는 벡터 검색과의 하이브리드 라우팅으로 사용		

성능개선

Reflection

설계개선

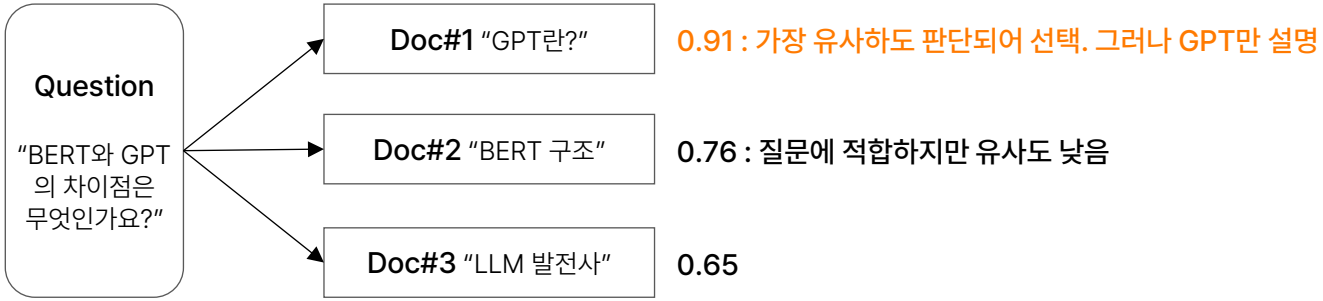
1. Naïve RAG 한계점



Query에 대한 얕은 이해 :

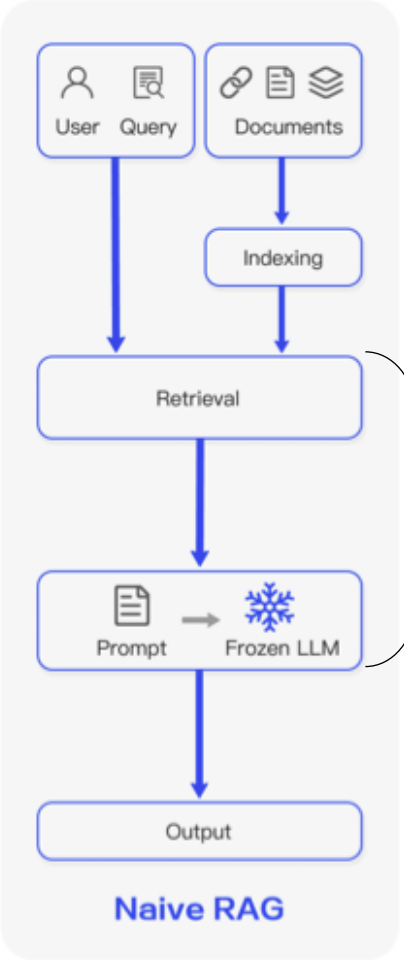
- 쿼리와 문서 청크 사이의 의미론적 유사성이 항상 일치하는 것은 아님
  - (e.g.) 질문 의도를 반영하지 못한 경우:
  - BERT와 GPT의 차이를 묻는 질문에 대해 임베딩 기반 검색 통해 GPT 설명만 포함된 문서가 높은 유사도를 받을 수 있음
  - 질문의 일부 주제와는 유사하지만, 질문 전체의 의미(비교)는 담고 있지 않음
    → 의미론적 유사성 ≠ 의도 적합성

- 검색을 위해 유사도 계산에만 의존하는 것은 쿼리와 문서 간의 관계 파악에 부족



Source : https://arxiv.org/pdf/2312.10997

1. Naïve RAG 한계점



정보 과잉이 부르는 **Hallucination :**

- 검색된 모든 청크를 LLM에 직접 공급하는 것이 항상 유익한 것은 아님
  - LLM은 맥락 안에서 중요한 내용을 구분해서 답을 생성해야 하는데,
  - 불필요한 정보까지 모두 전달되면 오히려 정답 품질을 저해할 수 있음

- 중복되고 노이즈가 많은 정보는 LLM이 핵심 정보를 식별하는데 방해가 됨
  - LLM이 정작 중요한 정보가 무엇인지 구별하지 못하여,
  - Hallucination 위험이 높아질 수 있음

Source : https://arxiv.org/pdf/2312.10997

1. Naïve RAG 한계점

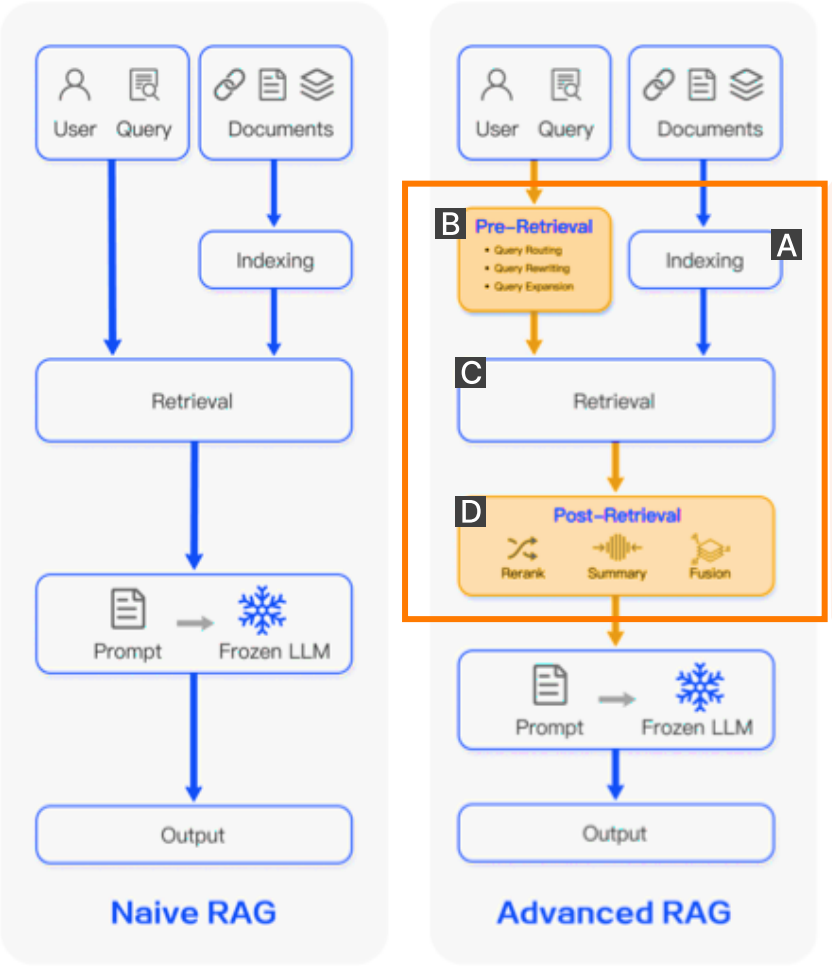
단순하고 빠르지만, 복잡한 질의 대응에는 문맥 부족, 검색 정확도 낮음, 확장성 한계가 뚜렷한 구조

구분	한계점	주요 내용
검색 정확도	청크 기반 검색의 한계	• 문서가 단순한 고정 청크로 나뉨 • 문맥이 끊기거나, 질문과 관련 없는 청크가 검색 될 수 있음
답변 품질	불충분한 문맥으로 부정확한 응답	• 관련 청크가 검색 되더라도, • LLM이 질문에 필요한 전체 맥락을 갖지 못할 수 있음
유연성	다양한 질의 처리 어려움	• 단일 쿼리 방식만 지원 : 1개 질문 → 단일 벡터 검색 • 다의어, 추론 질문, 복잡한 정보 요구에는 약함
비용	불필요한 임베딩 호출	• 변경되지 않는 문서라도 매번 새로 임베딩하거나 캐싱을 고려하지 않으면 비효율적
프롬프트 구조	프롬프트 품질이 고정적	• 문서 요약, CoT, Reasoning 등을 포함한 고급 프롬프트 구조가 적용되지 않음
확장성	멀티소스, 멀티모달 확장 어려움	• 단일 벡터DB, 단일 임베딩 모델에 종속 • 다양한 형태의 입력을 처리하지 못함 (PDF + DB + 웹검색 등)
평가 피드백	정답 여부에 대한 피드백 루프 없음	• 사용자로부터 응답 정확도 피드백을 반영하거나 개선하는 구조 없음

2. Advanced RAG

Key Point

"검색 품질"을 개선하기 위한 단계별 핵심 전략



Retrieval 이전/이후 단계 정교화 :

A. Indexing
- 문서 데이터를 벡터 형태로 변환하여 검색에 활용할 수 있도록 준비하는 단계
- Hierarchical Structure
- Semantic Chunking

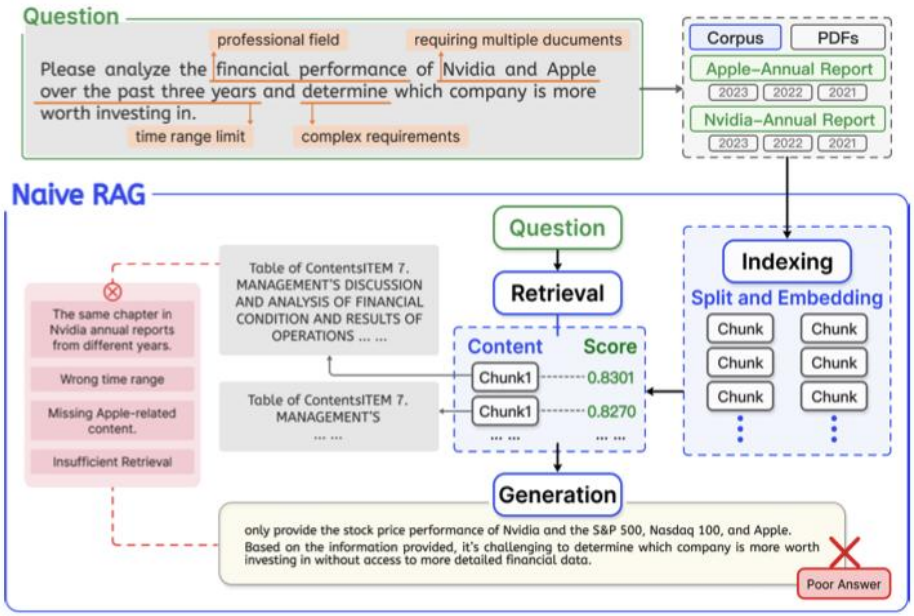
B. Pre-Retrieval
- 검색하기 전에 질문을 더 좋은 쿼리로 바꾸거나, 라우팅/확장 통해 검색 정확도를 높이는 단계
- Query Rewrite, Query Routing, Query Expansion
- Query Transformation

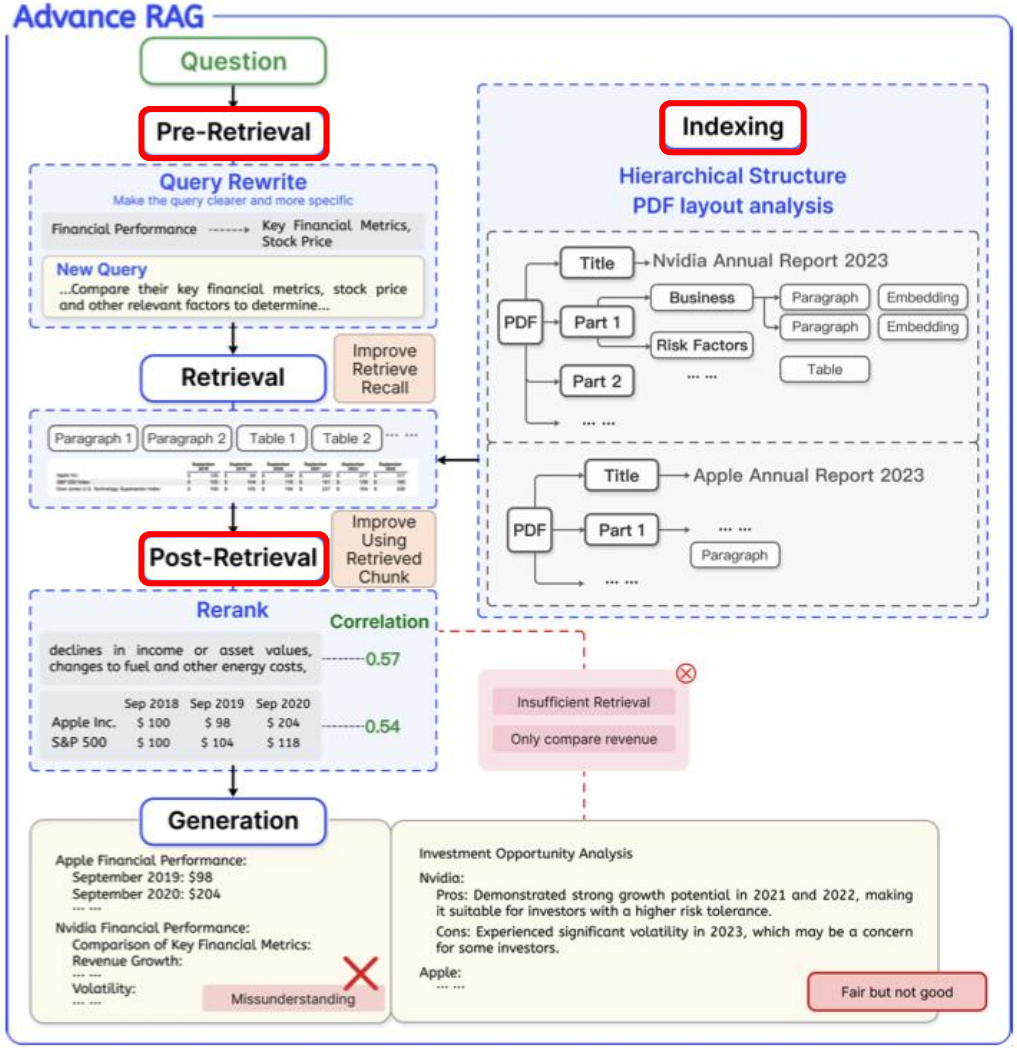
C. Retrieval
- 벡터 유사도 기반으로 문서를 검색하는 단계
- Hybrid Retrieval : Keyword + Semantic Search
- Multi-hop Retrieval : 여러 문서를 단계적으로 연결하여 복합 질문에 대응

D. Post-Retrieval
- 검색된 결과를 정제하고 최적화하여 LLM 입력 품질을 높이는 단계
- Reranker, Reorder

Source : https://arxiv.org/pdf/2312.10997

[참고] Naïve vs Advanced RAG





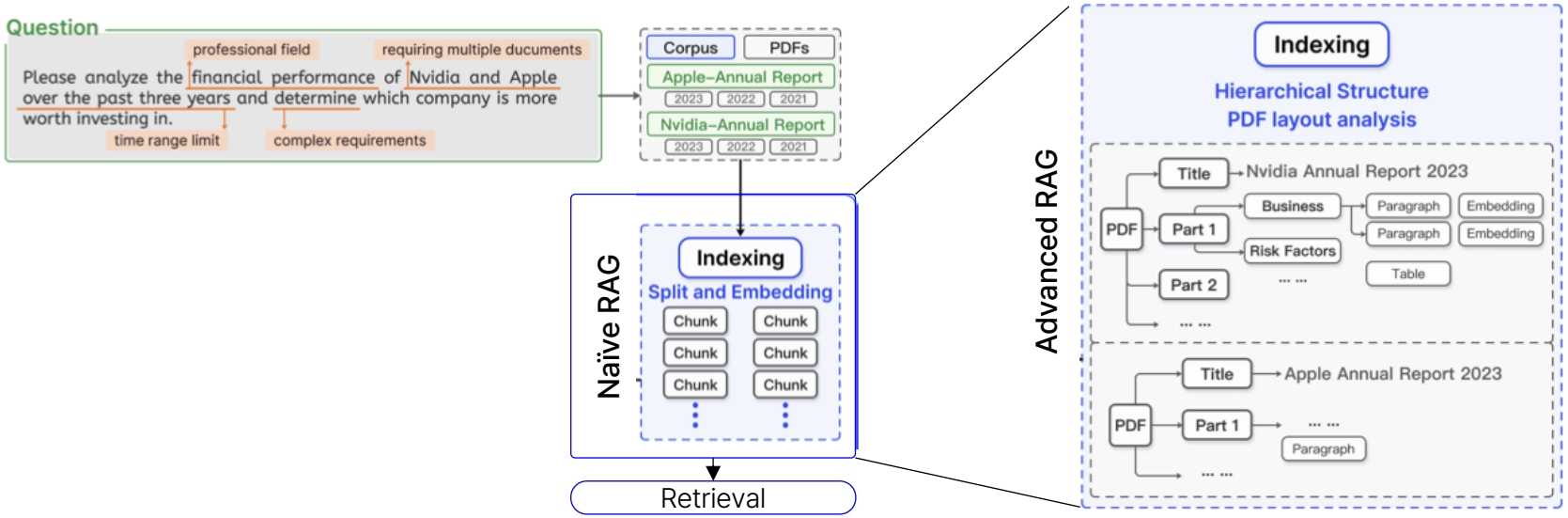
➔ 더 정밀하고 신뢰도 높은 답변 생성

- 질문 이해 및 질의 최적화 : Pre-Retrieval > Query Rewrite
- 문서 구조 활용 : Indexing > Hierarchical Structure
- 관련성 높은 정보 우선 : Post-Retrieval > Rerank

Source : https://arxiv.org/pdf/2407.21059v1

2-A. Indexing

Hierarchical Structure : 검색 정확도의 시작



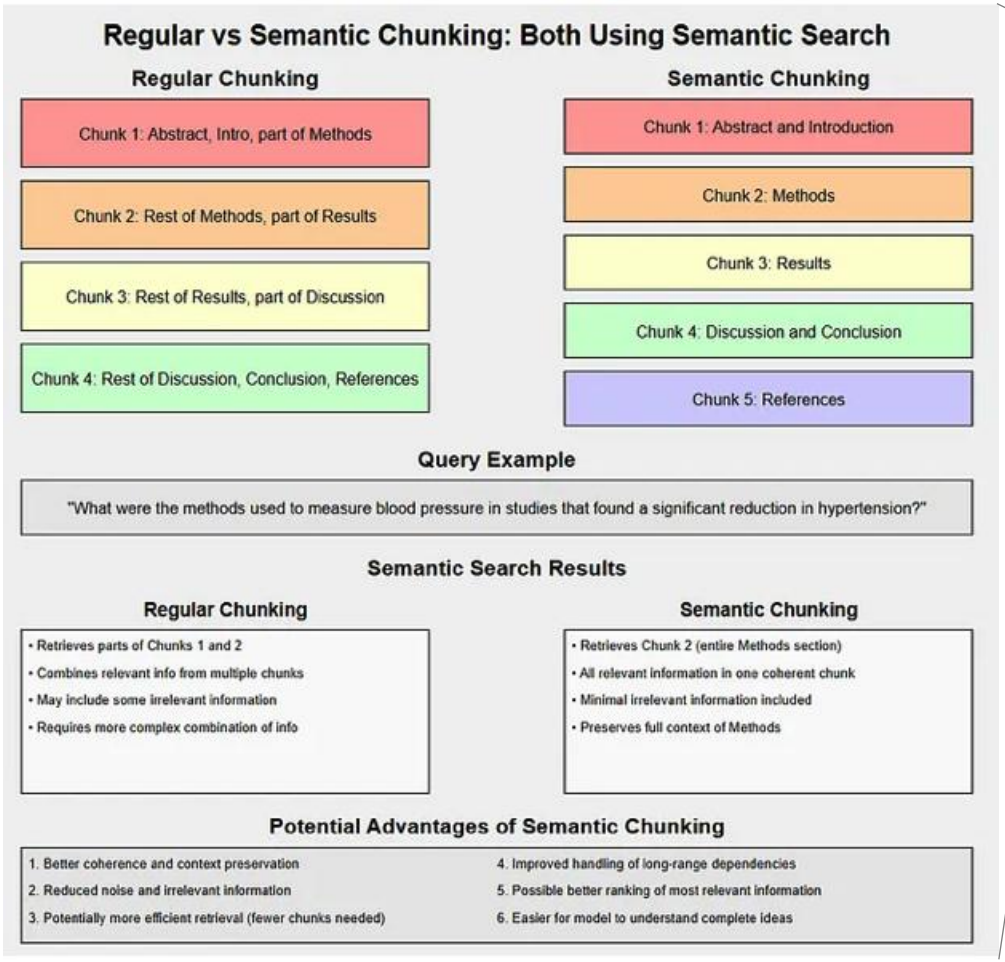
- Naïve RAG : 문서를 일정한 크기로 자르고 (고정 청크), 각 청크를 벡터로 변환하여 저장
- Advanced : 문서의 구조와 의미를 파악하여 문서 내부의 계층적 관계를 반영한 인덱싱 수행

- Hierarchical Structure : 문서를 의미 있는 트리 구조로 정리하는 방식
  - 계층적 구조 통해 검색 범위를 좁히는 대신, 검색 depth를 늘림 → 검색 정확도와 속도 향상
    (eg) 제목 → 소제목 → 문단 → 문장

- (Retrieval) Multi-hop : 노드 간의 관계를 활용 → 검색 정확도 향상
- (DB) RDB + VenctorDB 검색에도 유용

2-A. Indexing

Semantic Chunking : 의미상 유사한 단락을 기준으로 청킹 → 의미가 밀접한 정보 덩어리로 임베딩



- Query : 고혈압 연구에서 혈압 측정에 사용된 방법은 무엇인가요?
- Regular Chunking
  - 필요 정보가 Chunk 1+2 걸쳐져 있음
  - 정답 뿐만 아니라 노이즈도 포함 될 수 있음
- Semantic Chunking
  - Chunk 2에서 확인하면 됨
  - 해당 청크로 정답을 깔끔하게 할 수 있음

  → 더 빠르고, 덜 복잡하고, 더 정확하게~!

- Hierarchical Structure
  - 구조 : 문서의 물리적/논리적 구조 분석
  - 정보 : 문서가 어떻게 구성되어 있는지를 트리 형태로 파악
- Semantic Chunking
  - 구조 : 문장/문단을 의미 기반 임베딩하여 검색
  - 정보 : 문장 간 의미 유사도로 벡터 공간에 매핑

Source : https://medium.com/@nirdiamant21/semantic-chunking-improving-ai-information-retrieval-2f468be2d707

2-B. Pre-Retrieval

검색하기 전에 질문을 더 좋은 쿼리로 바꾸거나, 라우팅/확장 통해 검색 정확도를 높이는 단계

구분	목적	비유적 표현
Query Rewriting	사용자의 질문을 LLM이 이해할 수 있도록 더 적절한 검색 질의로 다시 표현	이 질문, 더 똑똑하게 바꿔볼까?
Query Routing	질문이 어떤 데이터소스 또는 인덱스로 가야 하는지 결정하는 전략	어디서 찾아야 할까?
Query Expansion	원래 질문에는 없지만, 관련된 키워드를 추가하여 검색Recall을 높이는 전략	이 단어 외에 연관된 건 없을까?
Query Transformation	질문 유형 자체를 변형하여 검색 최적화를 도모하는 전략	비교/인과 등 질문의 형태를 바꾸어야 할까?

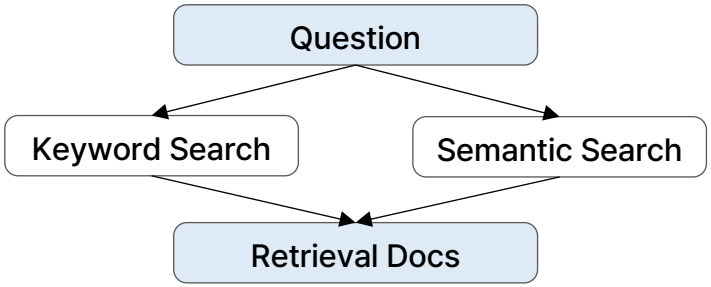
DETAILS :

- Query Rewriting
  - BERT와 GPT 중 뭐가 더 좋아요? → “Compare the architecture, training method, and use cases of BERT and GPT models.”
  - LLM을 활용한 재작성, 이전 질문 기반으로 사용자 맥락 반영
- Query Routing
  - Nvidia의 2023년 ESG 전략은? → [(VectorDB : 연차보고서) + (RDB : ESG 수치테이블)] → ESG 내용 (수치+문서)
  - MultiQueryRetriever, RouterRetreiver 등
- Query Expansion
  - 전기차 충전소 인프라 문제는? → 확장 키워드: “EV”, “charging station”, “infrastructure”, “grid”, “Tesla”, “range anxiety” 등
  - Prompt Engineering (e.g., 이 질문에 연관된 키워드 5개 생성해서...)
- Query Transformation
  - 삼성전자와 애플의 3년간 실적을 비교해 주세요 → Sub-Query 변환 (①삼성전자 2021-2023 실적, ②애플 2021-2023 실적, ③두 회사의 차이점)
  - Query Decomposition(Chain-of-Thought, Multi-hop 등)

2-C. Retrieval

정확한 검색 위한 새로운 시도

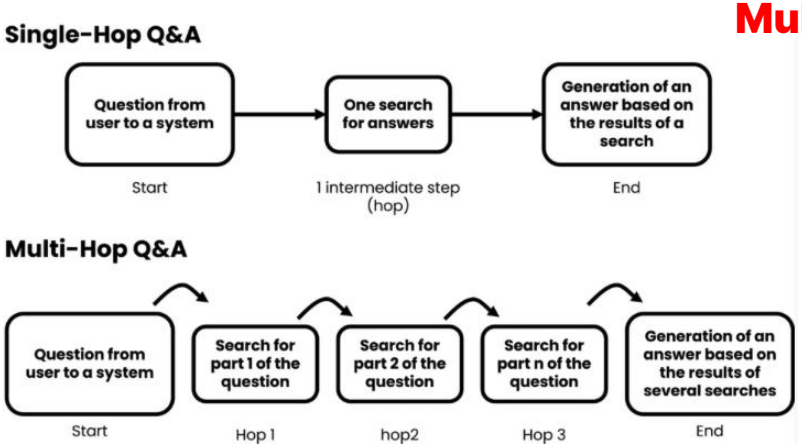
Hybrid Search



- Keyword + Semantic Search
  - 같은 질문을 여러 관점으로 검색
  - Keyword : 정확한 단어 매칭 기반
  - Semantic : 의미와 문맥을 이해하여 관련 정보 검색

➔ 다양한 검색 방식의 결합으로 검색 정밀도와 커버리지 향상

Multi-hop Search



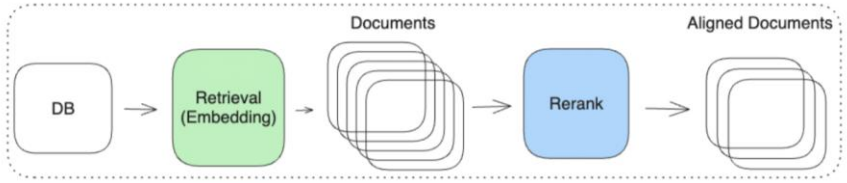
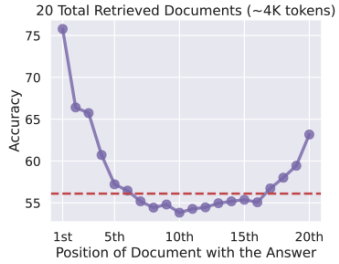
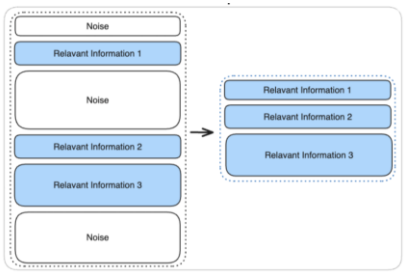
- 단계적 검색 (1-hop → 2-hop → ...)
  - 질문에 필요한 복수 개의 정보 조각을 순차적으로 검색 및 조합
  - Question → 문서1 검색 → 그 결과 기반으로 문서2 검색 → 종합

➔ 복합적인 질문에 대해 여러 문서를 검색/연결하여 종합적인 정보 제공

Source : https://ambersearch.de/en/what-is-multi-hop-qa/

2-D. Post-Retrieval

검색된 결과를 정제하고 최적화하여 LLM 입력 품질을 높이는 단계

구분	WHAT	WHY & HOW
Reranker	• 어떤 문서가 가장 중요한가? • 검색 결과 문서를 다시 평가하여 재정렬 (Two-Stage Retrieval) 	• 검색 결과가 불완전하거나 순서 오류일 수 있음 • [문서 ↔ 질문] 유사도 기반으로 검색
Context Reorder	• 어떤 순서로 보여줄까 • 선택된 문서들을 질문 흐름에 맞게 재배열 	• LLM은 앞/뒤에 있는 정보에 더 집중하는 경향성 • 논리흐름, 질문 관련도
Compressor	• 이 중에서 꼭 필요한 정보만 뽑자 • 문서 내용을 요약하거나 핵심 문장만 추출하여 LLM 입력 최적화 	• Context Window 제한, 중복 제거 • 요약/추출/선택 기반 압축

2. Advanced RAG 한계점

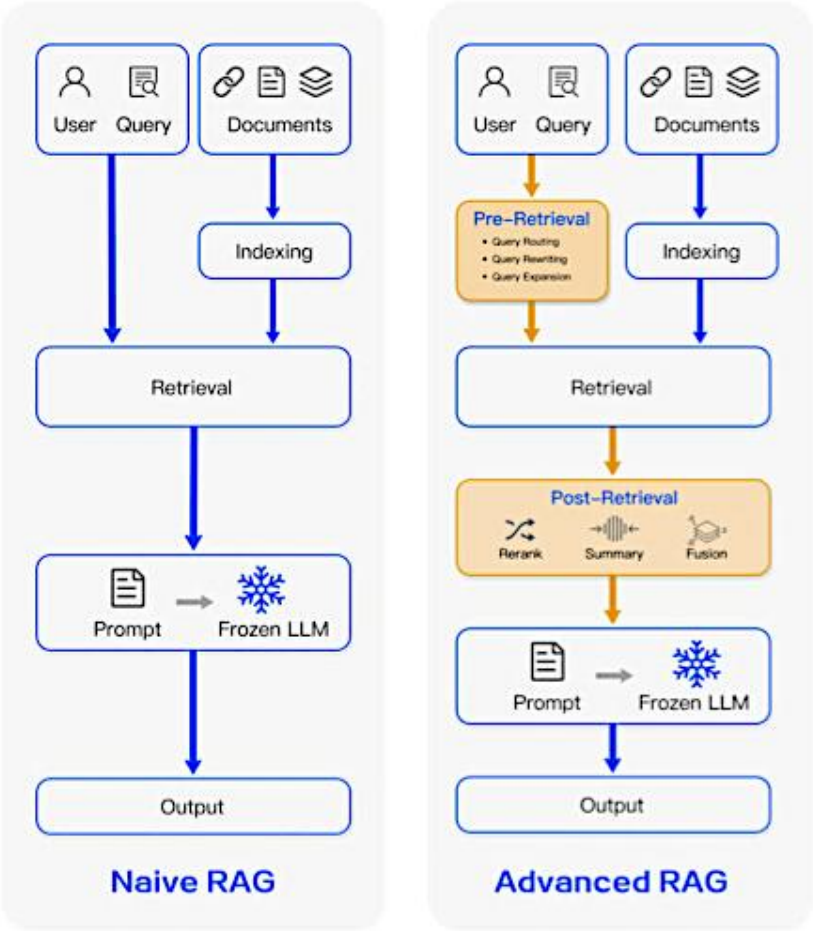
Naïve RAG의 한계 중 개선된 것, 여전히 남은 것

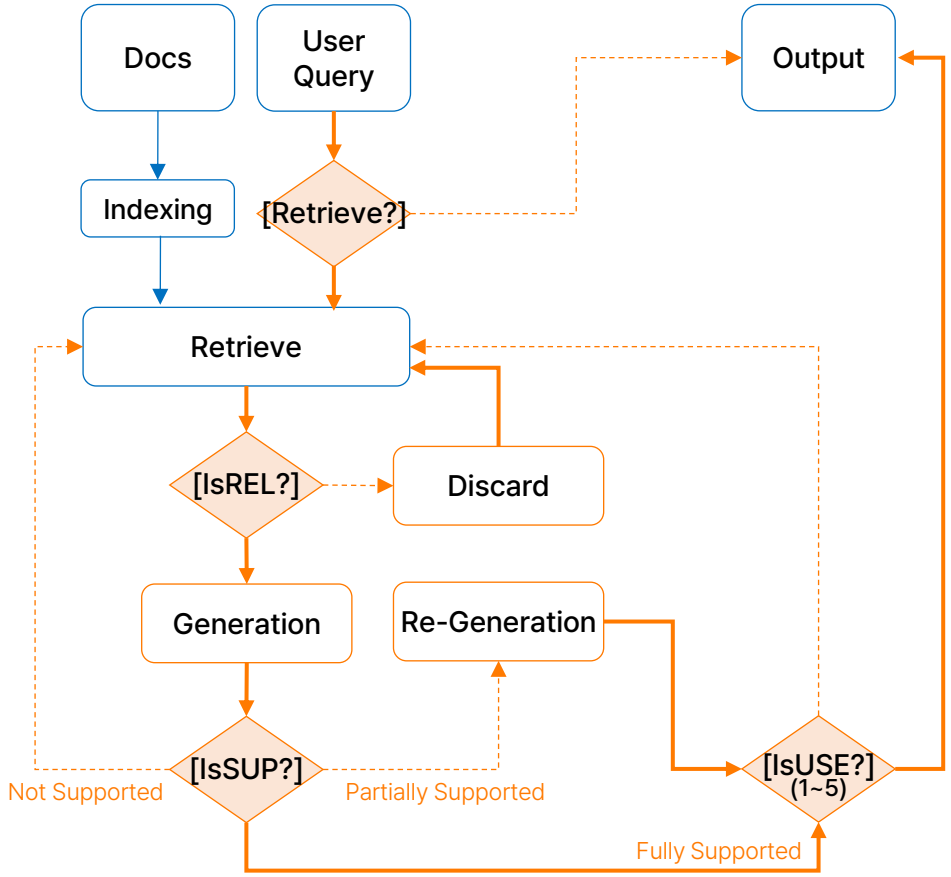
구분	Naïve RAG 한계	Advanced RAG	개선 여부
검색 정확도	• 유사도만으로 문서 선택	• Hybrid Search, Reranker 등 도입	개선
질문 의도 파악	• Query 의미 왜곡 가능성	• Query Rewriting, Expansion 등 도입	개선
문맥 정렬	• 중요 정보가 뒤에 배치될 가능성	• Context Reorder 적용	개선
핵심 정보 전달	• 정보 중복, 노이즈 발생 가능성 높음	• Compressor 적용	개선
구조	• Linear Structured RAG • 모든 단계를 한번에 다 잘해야 함 • 이전 단계로 되돌아가기 어려움 (이전 단계의 결과물을 수정하기 어려움)		진행중
Hallucination	• 출처는 있으나 내용 왜곡	• 여전히 존재 • 검색된 정보 기반 응답이라 하더라도 LLM의 생성 특성상 완전한 통제 불가능	부분 개선
근거 추적	• 근거 문장과 응답 단절	• 정답 근거 불명확 • 출처 문서가 있더라도, 실제로 사용된 문장이 응답에 직접 반영되지 않을 수 있음	진행중
속도/비용	• 빠르지만 검색 속도 불명확	• 느리지만 정확...? • Pre/Post-Retrieval 단계가 추가되었지만 추론이 충분히 진행되지 못함 • 문서 형식이 다양한 경우 인덱싱, 압축, 요약 정확도가 기대만큼 개선되지 않음	새롭게 확인 (실시간 대응이 필요한 업무에서는 여전히 제약)

3. Self-RAG

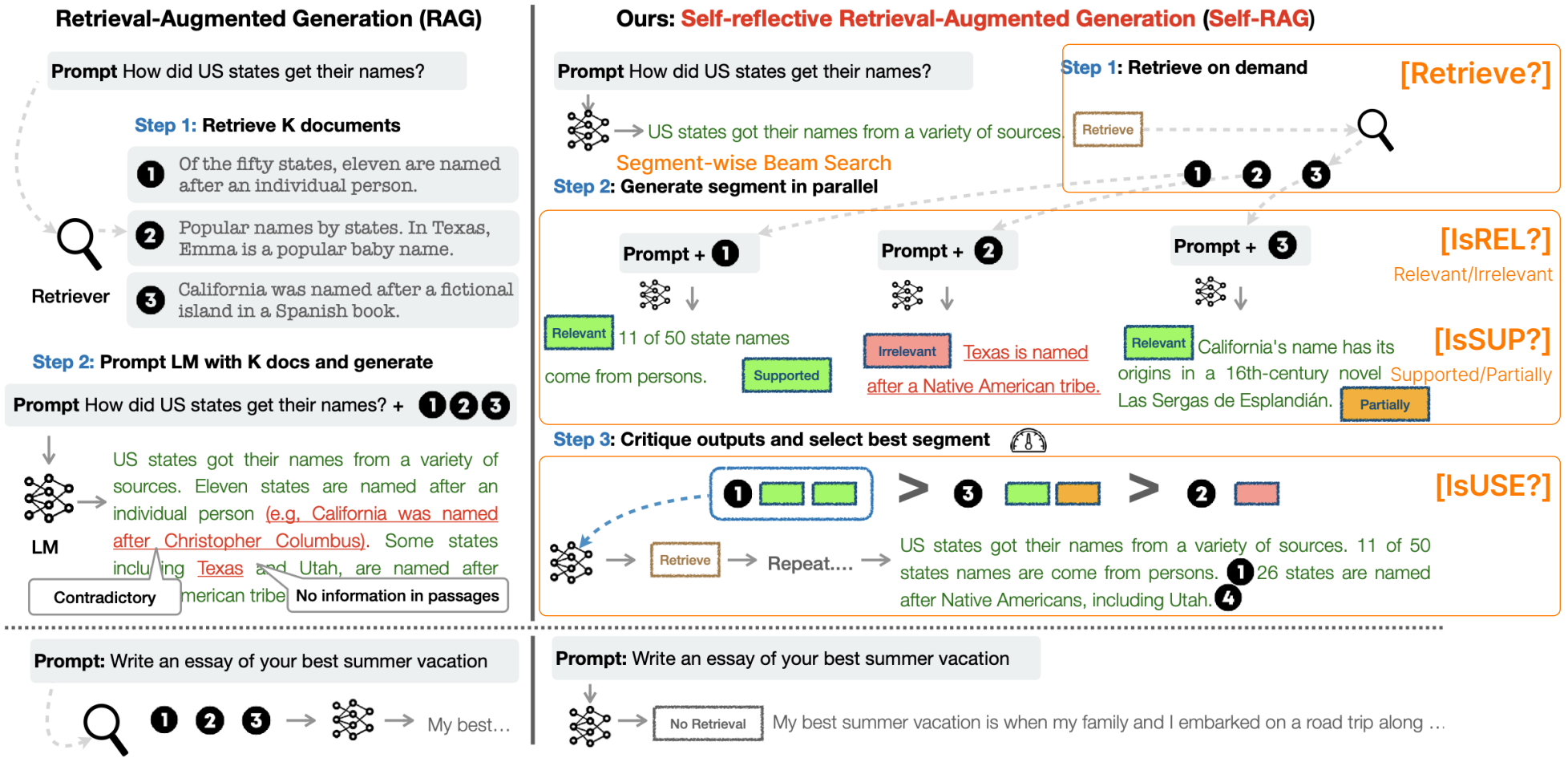
Key Point

답변 생성 흐름에 인라인으로 삽입된 Reflection Token 통해 검색 필요성을 스스로 판단하고, 응답의 근거 충실도를 평가



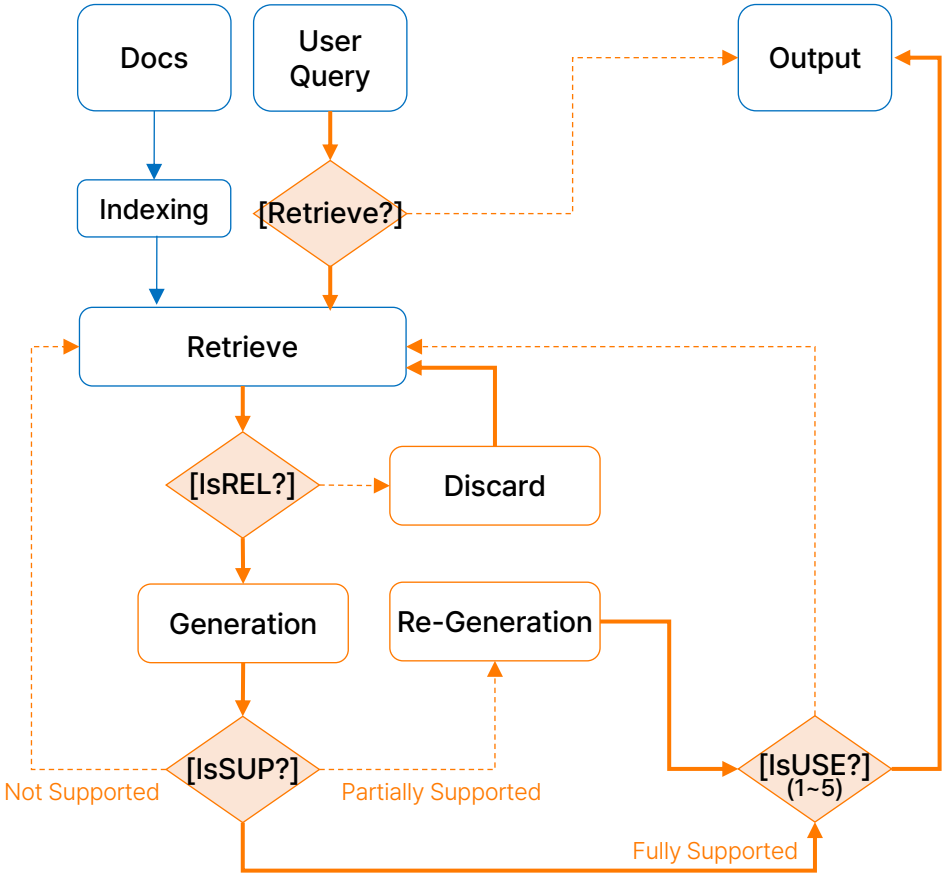


3. Self-RAG



Source : https://arxiv.org/pdf/2310.11511

[참고] Reflection Token



- [Retrieve?]
  - 검색이 필요한가?
  - User Query 수신 후 판단
  - Yes : RAG, No : Output
- [IsREL?]
  - 검색된 문서가 질문과 관련 있는가?
  - 검색된 문서 자체의 관련성 평가
  - Relevant, Irrelevant
  - Discard : Irrelevant 평가되면 해당 문서를 폐기 (이후 검색 재시도)
- [IsSUP?]
  - 생성한 응답이 문서 내용에 의해 뒷받침되는가?
  - 응답을 생성한 후, 응답과 문서 간의 일치 여부 확인 (Hallucination 체크)
  - Fully Supported, Partially Supported, Not Supported
- [IsUSE?]
  - 최종 응답이 질문에 유용한가?
  - 마지막으로 응답의 전반적인 품질 평가
  - 1~5점 척도로 평가

3. Self-RAG 한계점

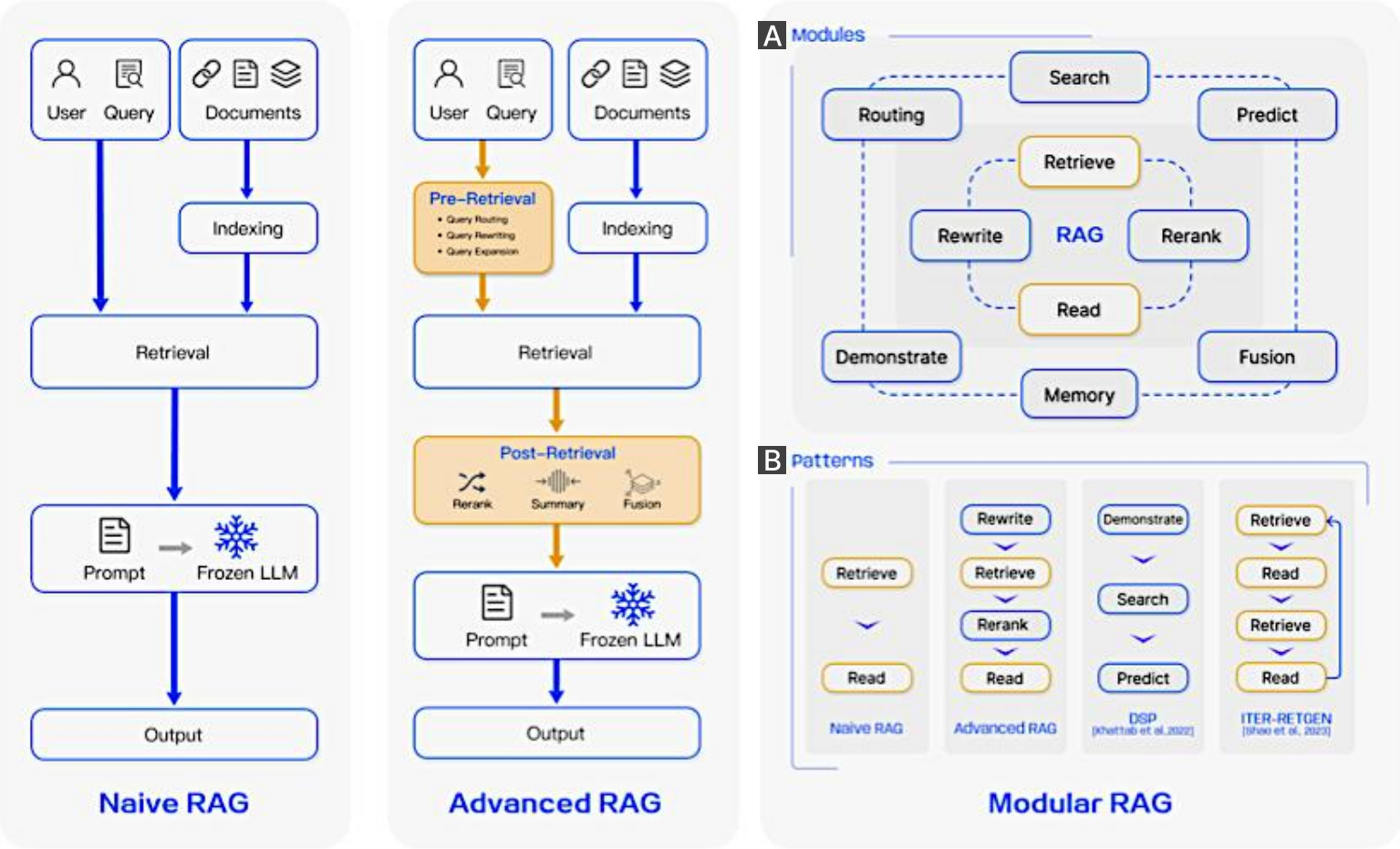
Advanced RAG의 한계 중 개선된 것, 여전히 남은 것

구분	Advanced RAG 한계	Self-RAG	개선 여부
구조	• Linear Structured RAG	• Reflection Token 기반의 Loop 구조 • 재검색, 재생성, 비선형 흐름 반영	개선
Hallucination	• 검색된 정보 기반 응답이라 하더라도 LLM의 생성 특성상 완전한 통제 불가능	• [IsSUP?] 통해 응답이 문서에 의해 뒷받침 되는지 생성 직후 자체 검증	개선
근거 추적	• 출처 문서가 있더라도, 실제로 사용된 문장이 응답에 직접 반영되지 않을 수 있음	• Segment 단위로 [IsSUP?] 평가하여 응답 구간별 근거 추적 가능	개선
불필요한 검색	• 모든 질문에 무조건 검색 수행	• [Retrieve?]으로 검색 필요성 판단 • 불필요 시 검색 생략	개선
속도/비용	• Pre/Post-Retrieval 단계가 추가되었지만 추론이 충분히 진행되지 못함	• Segment-wise Beam Search로 복수 개의 검색 결과에 대한 병렬 평가 • Advanced RAG 대비 연산량 추가 증가	진행중
복잡한 추론	• Multi-hop 질문 처리 제한적	• 단일 LLM 기반의 실행이라는 구조적 한계 • 다단계 추론, 외부 도구 연동 불가	진행중

Key Point

4. Modular RAG

LEGO와 같이 재구성이 용이하고 보다 유연한 흐름을 만들 수 있는 프레임워크

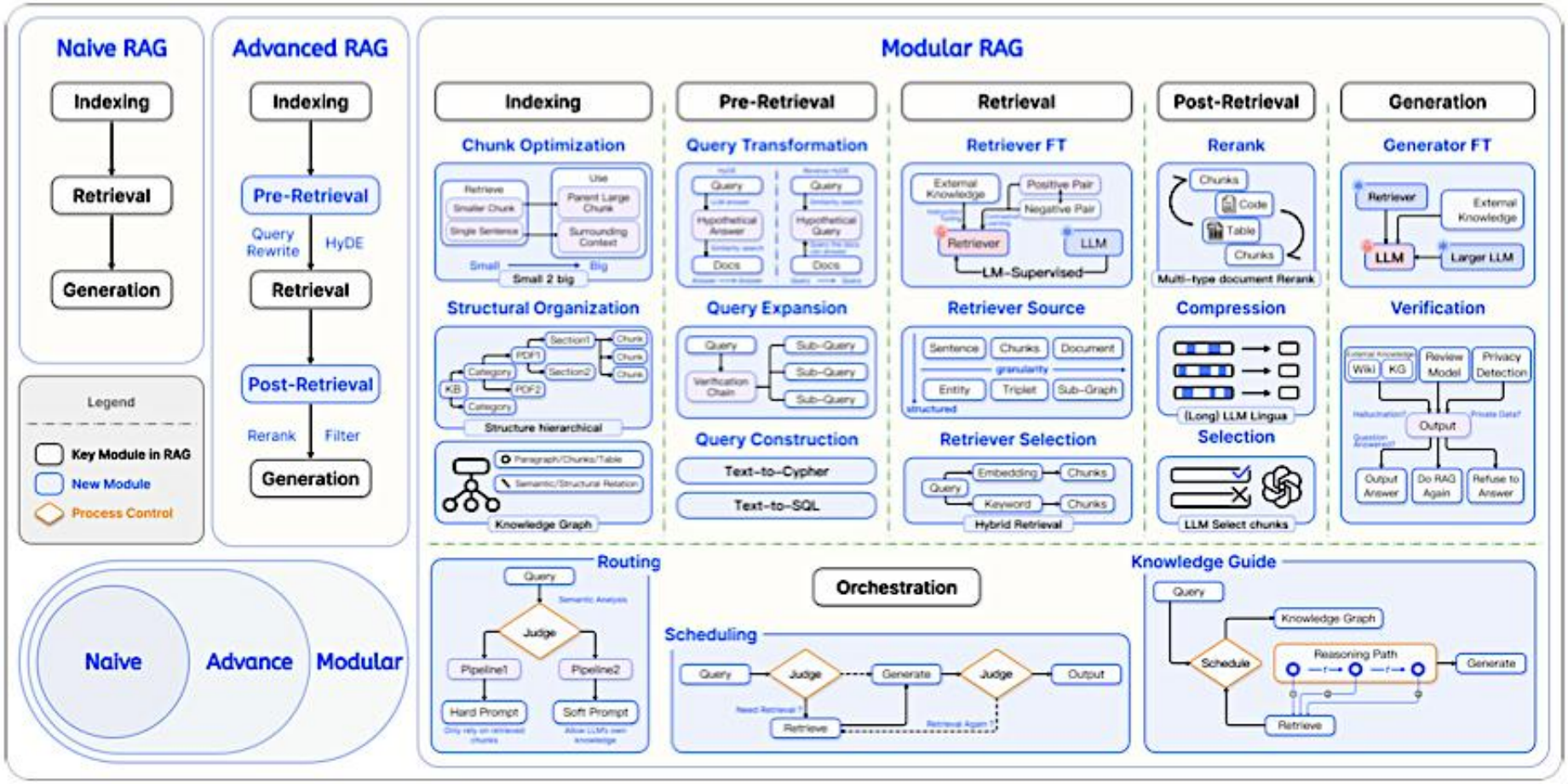


- Independent
  : 기능 위주 세분화
- Flexible & Scalable
  : 모듈의 추가/변경 쉬움
- Dynamic Graph
  : 그래프 형식의 흐름 구성,
  상황에 따라 분기 처리.

Source : https://arxiv.org/pdf/2312.10997

4-A. Modules

RAG를 기능 블록, “Module”로 나누고 그 블록들을 조합하여 사용하는 구조



Source : https://arxiv.org/pdf/2407.21059v1

[참고] Sub Modules

Module	Sub Module	주요 내용
Indexing	Chunk Optimization	청크 크기, 위치 최적화
	Structure Hierarchical	문서의 계층적 구조로 구조화
	Knowledge Graph	문서에서 추출한 Entity-Relation 기반으로 구조화
Pre-Retrieval	Query Transformation	질문 구조 변경 → 의미 정렬
	Query Expansion	질문 확장 → 누락 방지
	Query Construction	구조화 질의 생성
Retrieval	Retriever Finetuning	검색기 학습 최적화
	Retriever Source	다양한 정보 단위 활용
	Retriever Selection	적절한 검색 방식 선택
Post-Retrieval	Rerank	검색된 문서 재정렬
	Compression	LLM 전달할 정보 요약
	Selection	전달할 청크 선택
Generation	Generator Finetuning	LLM 자체 튜닝
	Verification	결과 검증 및 위험 방지
Orchestration	Routing	쿼리 유형별 적절한 파이프라인 선택
	Scheduling	처리 순서 제어
	Knowledge Reasoning Path	지식 흐름 기반 응답 설계

- 단계를 모듈로 정의
- 모듈은 하위 모듈로 구성 (Module – Sub Module)
- 하위 모듈은 Plug-in 형식
  - 독립 구조 (likely LEGO)
    : 교체/추가 용이
  - 병렬 구조
  - 분기 구조

4-B. Patterns

Linear Pattern

- 가장 단순하고 가장 일반적으로 사용되는 패턴
- RAG 기본 패턴은 Query → Retrieve → Generation 모듈로 구성

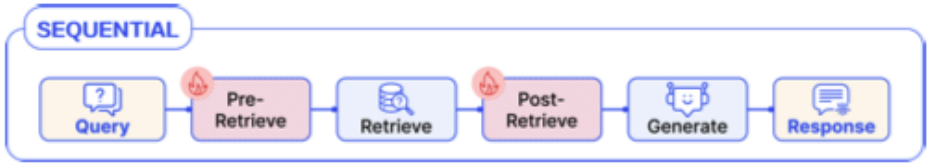


Fig. 4. Linear RAG flow pattern. Each module is processed in a fixed sequential order.

- SEQUENTIAL
  - Advanced RAG 구조
    - Pre-Retrieve : Query Transformation
    - Post-Retrieve : Reranker, Reorder
  - 장점 : 구조가 단순하고 구현이 쉬움
  - 단점 : 모든 질의에 동일한 모듈 경로 적용 → 불필요한 모듈도 진행
  - Point : 불완전한 쿼리, 부정확한 필터링, 낮은 유사도 정렬 등

Source : https://arxiv.org/pdf/2407.21059v1

4-B. Patterns

Routing Pattern

- 질문의 특성과 조건에 따라 가장 적합한 RAG 흐름을 선택하는 분기 구조
- 고정된 순서가 아닌, 맞춤형 경로로 RAG 실행한다는 점이 가장 큰 특징 ➔ Query Routing 모듈

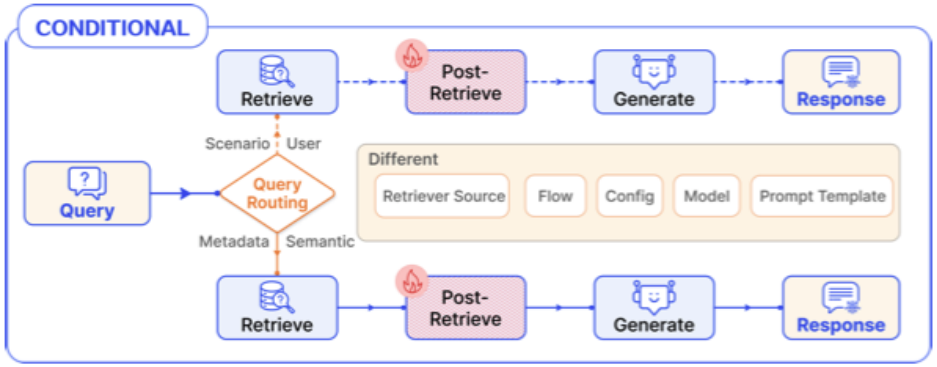


Fig. 6. The conditional flow pattern. There is a routing module that controls which RAG flow the query is directed to. Typically, different flows are used for various configurations to meet the general requirements of the RAG system.

- CONDITIONAL
  - Query Routing
    - 입력된 질문을 분석하여, 어떤 검색기 또는 어떤 처리 흐름으로 보낼지 결정하는 모듈
      - Retrieve Source : 검색기 (벡터 기반, BM25 기반, Hybrid 등)
      - Flow : Post-Retrieval 전략 선택 (압축, 요약, Rerank 등)
      - Config : 문서 타입, 도메인에 따라 설정
      - Model : 어떤 LLM을 사용할지
      - Prompt Template : 질문에 유형에 따라 적절한 템플릿 선택
    - Post-Retrieve : Reranker, Reorder
  - 장점 : 유연한 구조 설계, 여러 RAG 흐름을 동시에 운용
  - 단점 : 구현 복잡도 높은 편
  - Point : 라우팅 판단에 따라 정렬 결과에도 영향 미침

Source : https://arxiv.org/pdf/2407.21059v1

4-B. Patterns

Branching Pattern

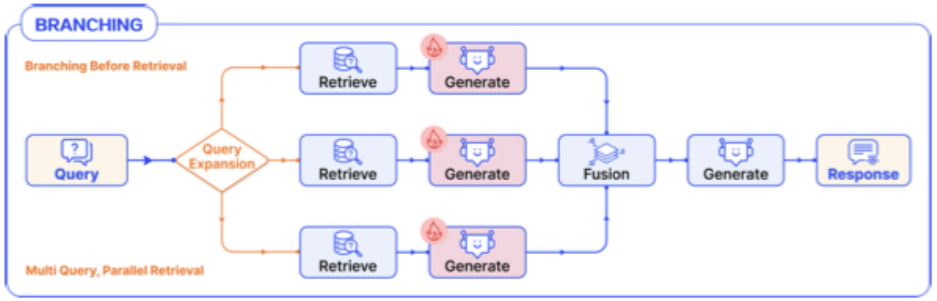


Fig. 7. Pre-retrieval branching flow pattern.Each branch performs retrieval and generation separately, and then they are aggregated at the end.

- Pre-Retrieval Branching
  - Query → Query Expansion 통해 Sub-query 다수 생성
    - 각 Sub-query 마다 독립적으로 Retrieve 수행
    - 각 검색 결과에 대해 각각 결과 생성
    - 마지막에 Fusion → Generation
  - 장점 : 질문 의도 세분화, 검색 누락 감소
  - 단점 : 처리량 증가, 중복 응답 가능성
  - Point : Sub-query별 생성 품질에 영향 받음

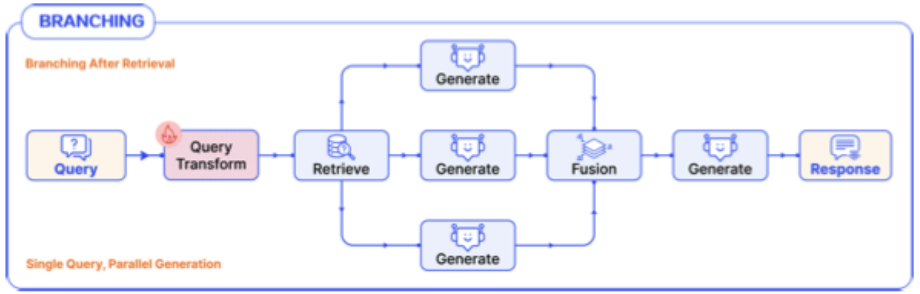


Fig. 8. Post-retrieval branching flow pattern.Only one retrieval performed, and then generation is carried out separately for each retrieved document chunks, followed by aggregation.

- Post-Retrieval Branching
  - Query → Query Transform 통해 구조 개선
    - 전체 문서에서 단일 Retrieve 수행
    - 검색 결과된 여러 청크들에 대해 개별 Generation 수행
    - 각각 생성된 응답을 Fusion 처리
  - 장점 : 구조 단순, 다양한 관점에서 문서 검색
  - 단점 : 검색은 1회만 수행, 검색된 청크들의 문맥 단절 가능성, Fusion 품질의 의존성 높음
  - Point : Context에 맞는 세분화된 쿼리로 변환 되는 것이 중요

Source : https://arxiv.org/pdf/2407.21059v1

4-B. Patterns

Loop Pattern (feat. JUDGE)

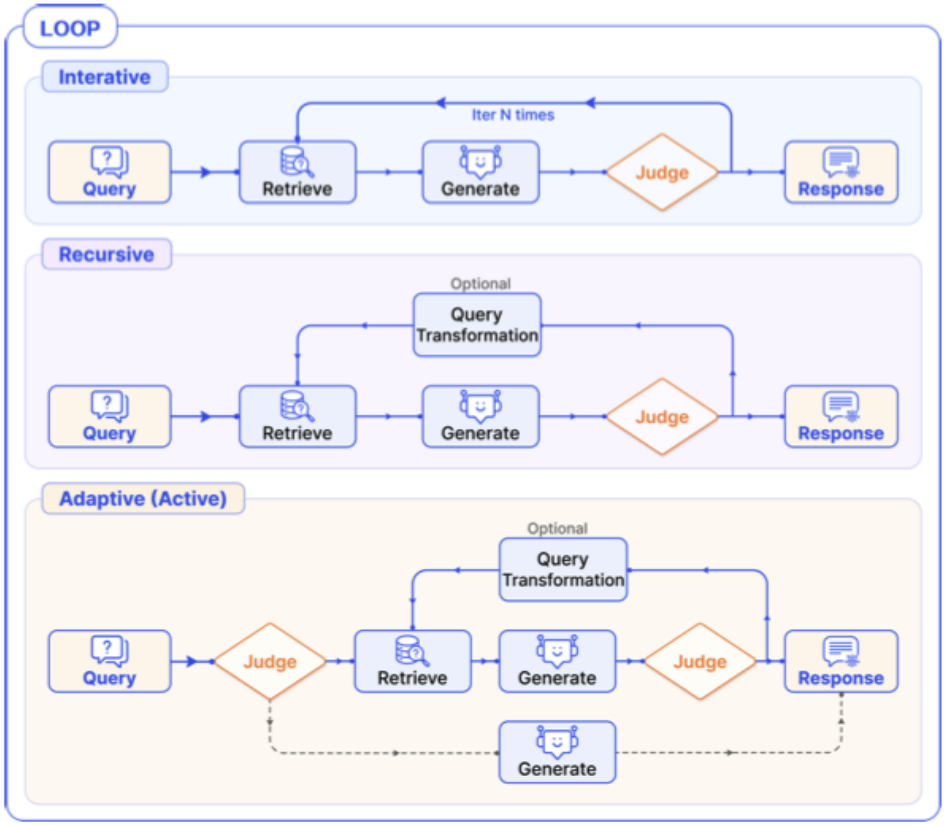


Fig. 10. Loop flow pattern. Typically, a RAG system performs multiple rounds of retrieval and generation. It can be categorized into three forms: iterative, recursive, and adaptive.

- Interactive Loop
  - Query → Retrieve → Generate → Judge
    - Judge에 따라 고정된 횟수 N만큼 반복
    - 이후 최종 응답 출력
  - 장점 : 정보 누락 시 재시도 가능
  - 단점 : 반복 횟수 고정 → 불필요하게 반복되거나, 부족할 수 있음.
    Query가 고정적이기 때문에 정교한 검색 한계

- Recursive Loop
  - Query → Retrieve → Generate → Judge
    - Judge 결과 따라 Query Transformation 수행
    - 필요 시 재귀적으로 답변 재생성
  - 장점 : 질문 정제 통해 검색 정확도 향상, 복합 질의에 효과적
  - 단점 : 무한 루프 방지 로직 필요

- Adaptive Loop
  - Query → Judge → [Retrieve → Generate → Judge] → Response
    - Judge가 루프 진입 여부 자체를 판단
    - 필요 시 루프 진입, 그렇지 않으면 바로 응답 처리
  - 장점 : 가장 적극적인 제어 가능, 질의 난이도에 따라 차별적 적용 가능
  - 단점 : Judge 로직 설계 중요

Source : https://arxiv.org/pdf/2407.21059v1

4. Modular RAG 한계점

Modular RAG의 한계 중 개선된 것, 여전히 남은 것

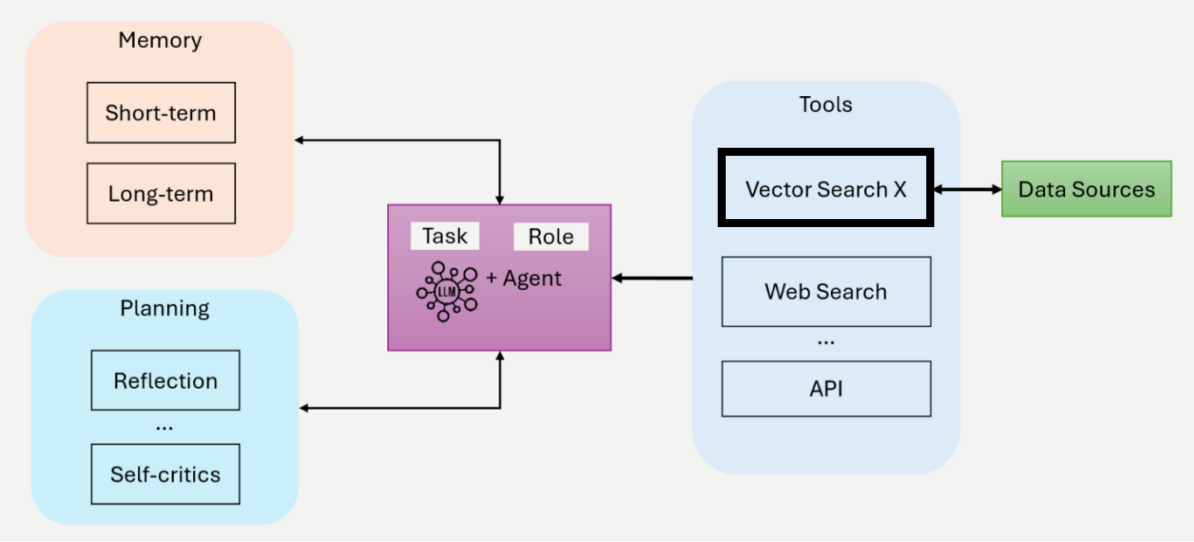
구분		Advanced RAG 한계	Modular RAG	개선 여부
구조		• Linear Structured RAG	• 조건 기반 분기 구조로 진화 (Routing, Loop 등)	개선
그래프 구조 (Pattern)		• (-)	• Loop, Branching Pattern 등은 품질은 높지만, 실시간 시스템 적용할 경우 Latency 발생 가능성 • 전체 시스템을 평가하는 프레임워크 부족	새롭게 확인
JUDGE	응답 품질	• Judge 모듈 부재 • 응답 품질에 대한 사후 평가	• Judge 모듈로 다단계 평가 • Fusion된 응답의 논리 연결 부족	부분 개선
	품질 평가	• (-)	• 좋은 응답인지 판단하는 Judge or Verification 모듈의 기준이 설계자 주관적이거나 LLM 의존적임	새롭게 확인
	Hallucination	• 여전히 존재 • 검색된 정보 기반 응답이라 하더라도 LLM의 생성 특성상 완전한 통제 불가능	• 응답 품질 검증(Judge) 및 • 오류 응답 회피 (Refuse) 전략 도입	부분 개선 (통제 가능성 높아졌으나, 완전하지 않음)
근거 추적		• 정답 근거 불명확 • 출처 문서가 있더라도, 실제로 사용된 문장이 응답에 직접 반영되지 않을 수 있음	• Retrieval-to-Generation 경로 추적 실험적 지원 (근거 문장 기반 평가 가능)	진행중
속도/비용		• 느리지만 정확...? • Pre/Post-Retrieval 단계가 추가되었지만 추론이 충분히 진행되지 못함	• 질문 유형별 모듈 경량화 및 병렬 처리로 일부 속도/효율 개선 가능	진행중 (유연성 개선 vs 복잡도 증가)

5. Agentic RAG

Key Point

AI Agent-based implementation of RAG

- Agent 통해 검색 전략을 스스로 설계하고, 도구를 선택하며, 응답을 생성하기 위한 최적 흐름을 능동적으로 제어하는 구조
- 기존의 검색 기반 구조에서 벗어나 "이 질문에 어떤 도구, 데이터가 적절한가?"를 에이전트가 판단하고 선택

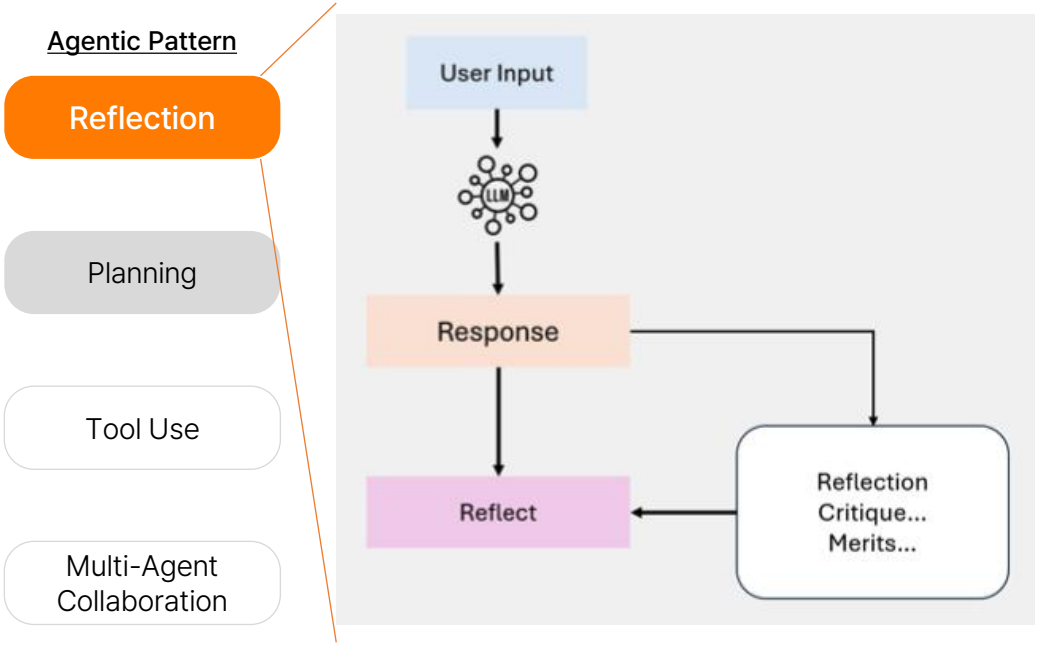


Source : https://arxiv.org/pdf/2501.09136

5-A. Agentic Pattern

Reflection : 자기 평가

- 에이전트가 자신의 행동을 결정을 검토하고, 이를 기반으로 성과를 평가하여 향후 전략을 개선하는 과정
- 이를 통해 에이전트는 이전 경험에서 학습하고, 반복적인 실수를 줄여 복잡한 문제 해결 능력을 향상시킬 수 있음



- Response : 에이전트는 자신의 응답(text), 외부 피드백(사용자 평가), 성공/실패 여부, Thumbs-up/down 등을 바탕으로 reflection 수행

- Reflection 역할 :
  - Self-Assessment
    : 에이전트는 자신의 이전 행동을 평가하여 무엇이 효과적이었는지, 무엇이 그렇지 않았는지를 파악
  - Strategy Adjustment
    : 평가 결과를 바탕으로 에이전트는 향후 유사한 상황에서 더 나은 결과를 얻기 위해 전략 수정
  - Knowledge Accumulation
    : 반복적인 자기 반성을 통해 에이전트는 지식을 축적하고, 이를 활용하여 더욱 정교한 의사결정을 내릴 수 있음

← 보통 단일 세션 내에서 임시로 유지하거나, 필요하다면 Reflection Cache 구성하여 관리

Source : https://arxiv.org/pdf/2501.09136

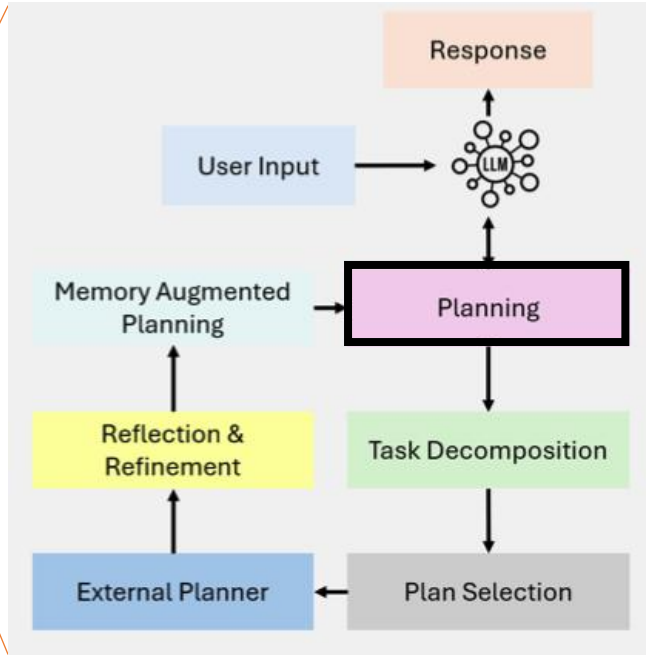
5-A. Agentic Pattern

Agentic Planning : 계획 수립

- 에이전트가 주어진 목표를 달성하기 위해 어떤 순서와 전략으로 행동할지 스스로 설계하는 과정
- 과거 경험(Memory), 실패로부터의 학습(Reflection), 외부 도구(External Planner)까지 활용

Agentic Pattern

- Reflection
- **Planning**
- Tool Use
- Multi-Agent Collaboration



- Planning은 전체 흐름 설계자 :
  - Agent는 먼저 사용자 목표(User Input)을 바탕으로
    → Task Decomposition : 무엇을 할지
    → Plan Selection : 어떻게 할지
    → External Planner : 누가할지 (자체 실행 포함)
  - 부가적으로 :
    - Reflection & Refinement : 계획 검토 및 개선
    - Memory Augmented Planning : 과거 경험 반영 (장기 메모리)

[쉽게 이해하기]
Agent는 공사 현장의 '현장 소장' 입니다. 설계도(Planning)는 직접 그리지만, 배관, 전기, 목공 작업은 전문가들(External Tool, Planner 등)에게 맡깁니다. 잘못되면 다시 돌아가서 재설계하거나 반영 합니다 (Reflection).

Source : https://arxiv.org/pdf/2501.09136

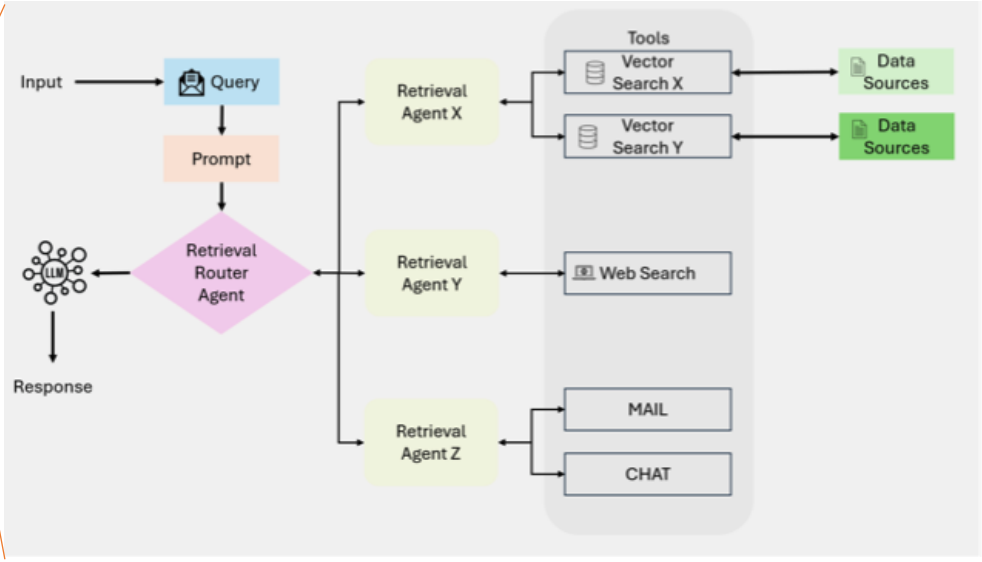
5-B. Agentic RAG Architecture

Multi-Agent Agentic RAG

- 각 에이전트는 특정 기능을 수행하며, 다른 에이전트와의 협력 통해 전체 시스템의 성능을 향상시킴
- 에이전트는 각각 명확한 역할을 가지고, 작업 특성에 따라 에이전트에게 동적으로 작업 할당

Architecture

- Single-Agent
- **Multi-Agent**
- Hierarchical
- Agentic Corrective
- Adaptive



- Retrieval Router Agent :
  - 사용자 입력(input)을 바탕으로 Query + Prompt 분석
  - 어떤 Retrieval Agent가 해당 질의에 가장 적합한지 판단

  → 작업을 분배하는 '중간 조정자' 역할
- Specialized Retrieval Agent :
  - Retrieval Agent X : 문서 기반 검색
  - Retrieval Agent Y : 웹기반 최신 정보 검색
  - Retrieval Agent Z : 이메일/채팅 로그 분석
- Router → Agents → 결과 병합 → 응답 생성

Source : https://arxiv.org/pdf/2501.09136

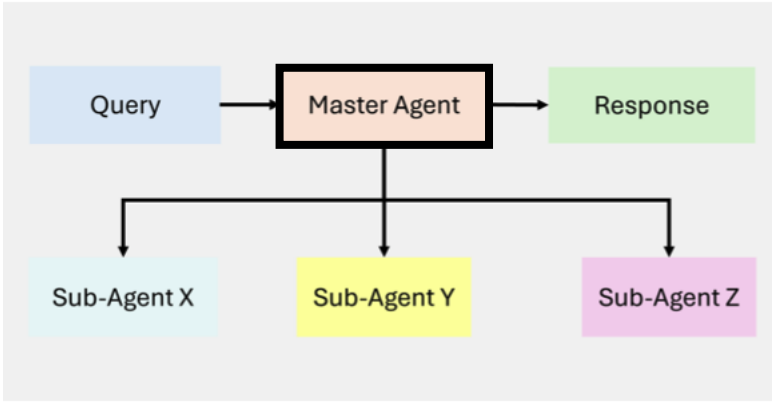
5-B. Agentic RAG Architecture

Hierarchical Agentic RAG

- 에이전트 간 계층 구조를 갖는 시스템
- 상위 에이전트인 Master Agent가 질의를 총괄하고, Sub-Agent에게 하위 작업을 분담하여 최종 응답 생성

Architecture

- Single-Agent
- Multi-Agent
- **Hierarchical**
- Agentic Corrective
- Adaptive



Master Agent or Supervisor

- Query Reception :
  - 사용자 질의를 수신
  - 수신한 질의 내용 통해 하위 에이전트에게 작업 분배 담당
- Strategic Decision-Making :
  - 질의 복잡도를 평가하고,
  - 어떤 Sub-Agent 또는 데이터 소스를 우선할지 결정
- Delegation to Subordinate Agents :
  - 특정 검색 방식에 특화된 Sub-Agent에게 작업 배정
  - 각 Sub-Agent는 할당된 작업을 독립적으로 수행
- Aggregation and Synthesis :
  - Sub-Agent로부터 수집된 결과들을 종합
  - 일관된 응답으로 통합 및 정제

Source : https://arxiv.org/pdf/2501.09136

[참고] Supervisor : Still Valid?!

명확하고 체계적인 흐름 설계에는 효과적이나, 복잡한 문제에는 동적 상태 기반 구조로 확장 필요

구분	PROS	CONS
제어 및 관리	• 명확한 계층 및 제어 : 시스템 전반의 관리 및 모니터링 용이 • 작업 분해 및 전문화 : 복잡한 문제의 효율적 해결 및 전문성 향상	• 단일 실패 영향도 : Supervisor 오판 시 전체 시스템 영향 미침 • 병목 현상 가능성 : Supervisor 과부하 시 시스템 지연 및 성능 저하
확장성 및 유연성	• 확장성 : 작업량 증가 시 하위 에이전트 추가 용이 • Fault Tolerance : 하위 에이전트 실패 시 작업 재할당 가능	• 제한된 자율성 및 적응성 : 하위 에이전트의 낮은 자율성으로 변화에 대한 적응력 부족 • Over-Head : Supervisor-Sub Agent 간 잦은 통신으로 인한 지연 가능성
자원 활용	• 자원 관리 효율성 : 공유 자원 할당 및 활용 최적화	• 설계 복잡성 : 효과적인 감독관 시스템 구축의 어려움 • 예측 할 수 없는 행동 : 복잡한 상호작용으로 인한 예측 불가능성 • 특정 문제에 대한 제한적 적용 : 고도로 분산된 문제 또는 복잡한 구조의 문제에는 부적합 → 동적 그래프 고려 필요

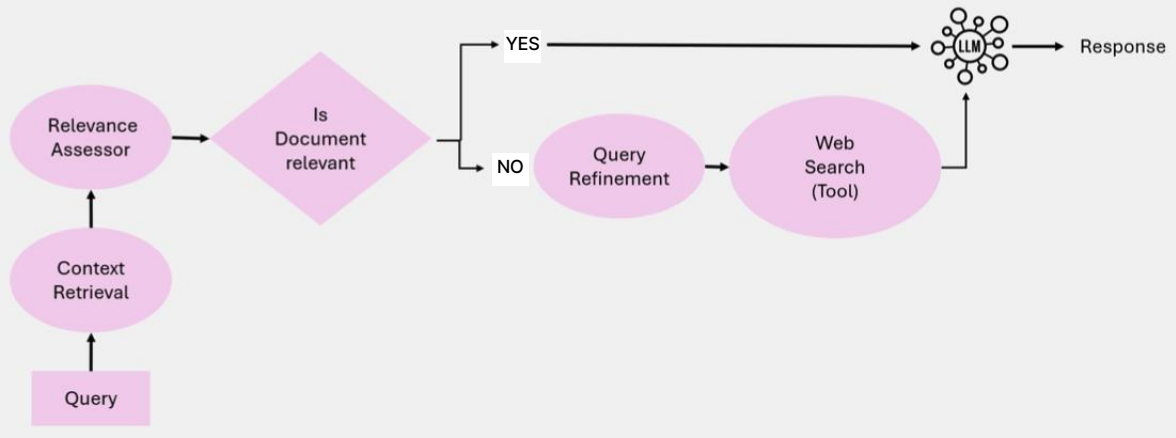
5-B. Agentic RAG Architecture

Agentic Corrective RAG

- 에이전트가 초기 검색 결과의 품질을 스스로 평가하고, 부적절한 경우 질의를 수정하거나 대체 검색 경로를 사용하는 구조
- Relevance 평가 기반으로 조건 분기 기반 설계
  (질문에 직접적으로 관련된 내용인지 평가)

Architecture

- Single-Agent
- Multi-Agent
- Hierarchical
- **Agentic Corrective**
- Adaptive



- **Context Retrieval Agent :** VectorDB에서 Query에 포함된 (initial) context 기반으로 문서 검색
- **Relevance Assessor Agent :** 검색된 문서들의 관련성 여부 평가, 관련성이 없거나 모호한 문서면 수정 조치가 필요함을 표시
- **Query Refinement Agent :** 질의를 재작성하여 검색 효율 높이고, 의미 기반 검색으로 결과 최적화 유도
- **External Knowledge Retrieval Agent :** 문서가 부족할 경우 웹검색 또는 대체 데이터 소스에 접근하여 외부 정보 수집 (Tool)
- **Response Synthesis Agent :** 검증된 모든 정보를 통합하여 일관되고 정확한 응답 생성

Source : https://arxiv.org/pdf/2501.09136

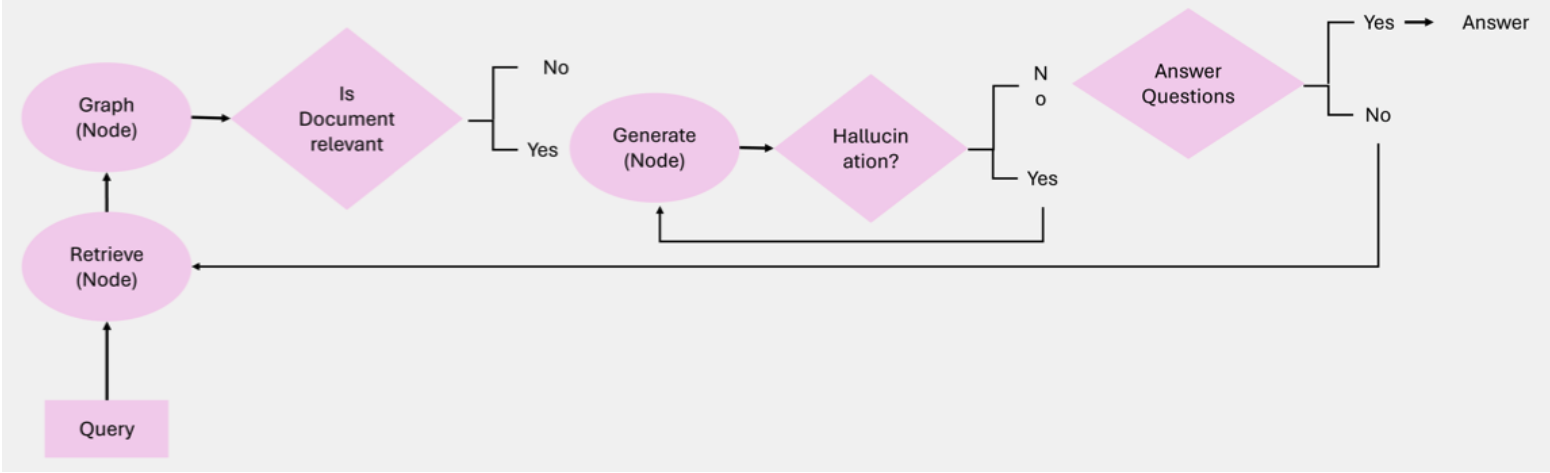
5-B. Agentic RAG Architecture

Adaptive Agentic RAG

- 상황에 따라 동적으로 판단하고, 에이전트의 경로와 전략을 변경하는 구조
- 질의 상황에 따라 전략이 충분하지 않다는 판단되면 → 검색 반복 → 새로운 경로로 전환하는 반응형 에이전트 구조

Architecture

- Single-Agent
- Multi-Agent
- Hierarchical
- Agentic Corrective
- **Adaptive**



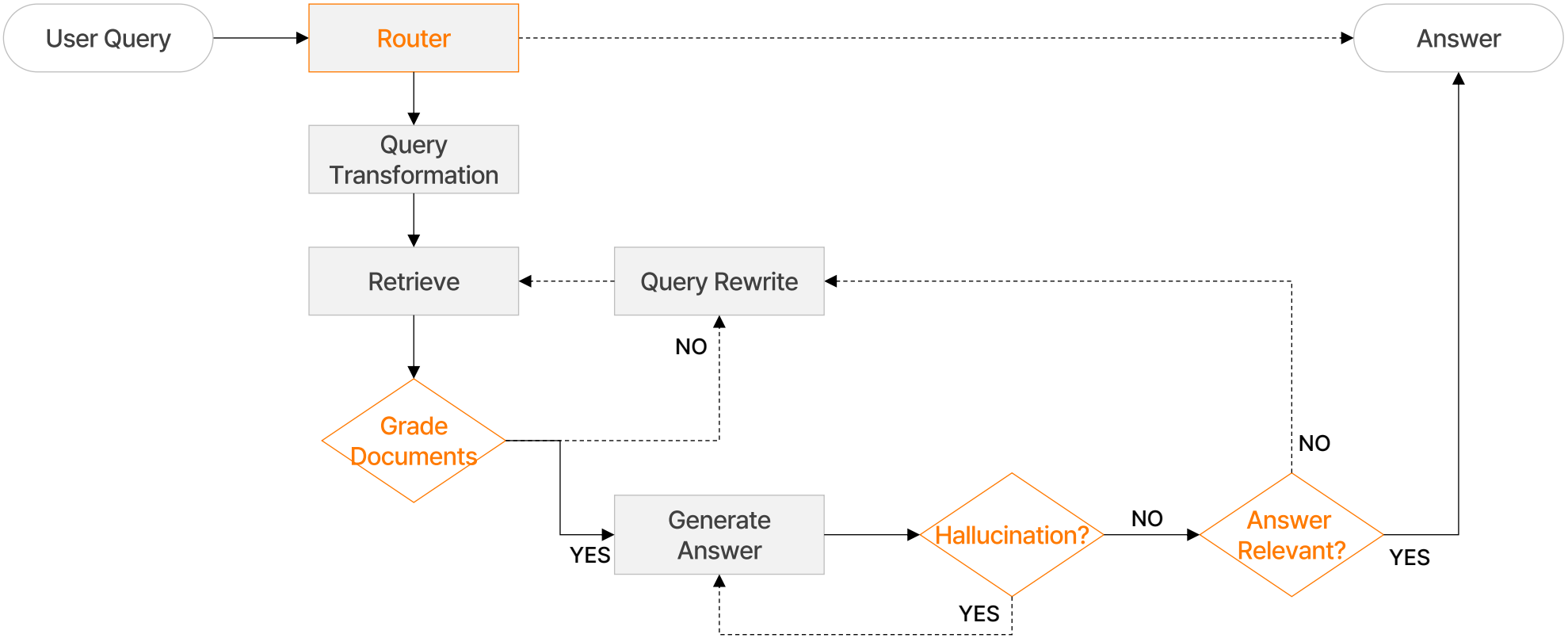
Example :

- **Prompt :** "Why is my package delayed, and what alternatives do I have?"
- **Workflow**
  - **Query Classification :** 질의를 분석하여, 복잡한 다단계 추론이 필요한 질의로 판단
  - **Dynamic Strategy Selection :** 질의 복잡성에 따라 다단계 검색 프로세스 활성화
  - **Multi-Step Retrieval :** 배송 정보 조회**(DB)** → 실시간 상태 업데이트**(API)** → 현황 파악 위한 웹검색**(**기상, 교통정보 등**)**
  - **Response Synthesis :** 위 정보를 통합하여 실행 가능한 응답 생성
- **Response :** "Your package is delayed due to severe weather conditions in your region. It is currently at the local distribution center and will be delivered in 2 days. Alternatively, you may opt for a local pickup from the facility."

Source : https://arxiv.org/pdf/2501.09136

Agentic RAG Workflow

LLM이 Reasoning 통해 RAG Pipeline을 Tool로 동적 호출 및 제어

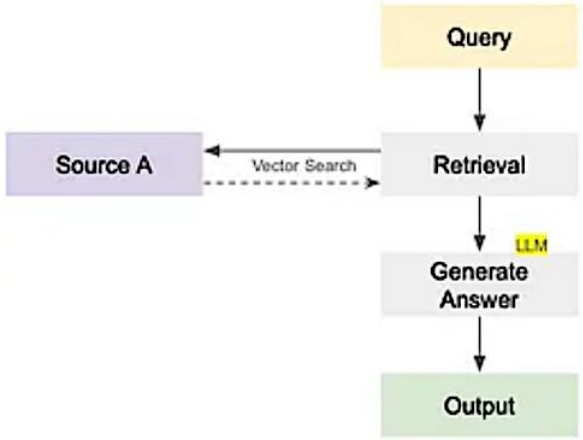
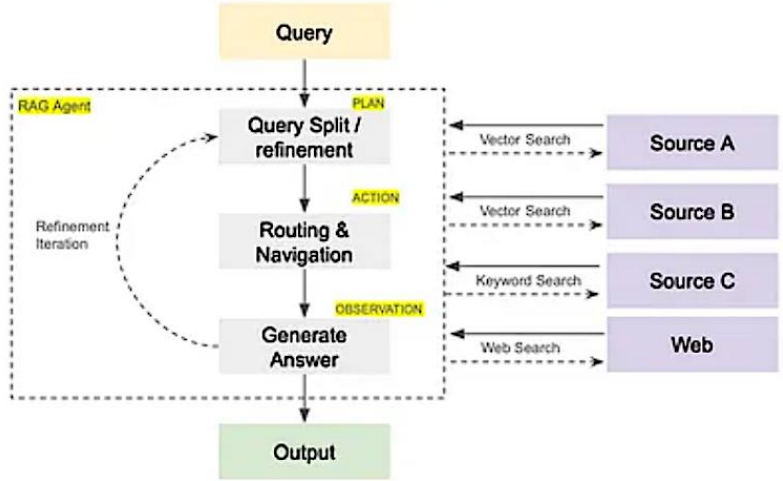


※ Remark

- Router : 질문 유형에 따라 경로 분기하는 역할로 확장

4. Agentic RAG 특징

Naïve vs Agentic RAG

구분	NaïveRAG	Agentic RAG
프로세스		
정보 검색 방식	정적이고 단일 단계의 검색	반복적이고 능동적인 검색
응답 생성 과정	단순히 검색된 정보를 바탕으로 응답 생성	검색된 정보를 요약, 통합, 분석 후 응답 생성
복잡한 요청 처리	제한적인 복잡성 처리	다단계 요청을 처리하며, 불확실성 해결하도록 반복 처리
실시간성	최신 정보 반영 어려움 (확보된 문서 기준 처리)	최신 정보와 실시간 데이터 반영 가능
적용 사례	Simple Q&A, FAQ	In-depth Q&A, 검색 기반 보고서 생성

Source : https://ai.plainenglish.io/build-agentic-rag-using-langgraph-b568aa26d710

4. Agentic RAG 한계점

자율성과 유연성을 강조한 지능형 구조이지만, 설계자 입장에서는 여전히 고민해야 할 사항들이 많음

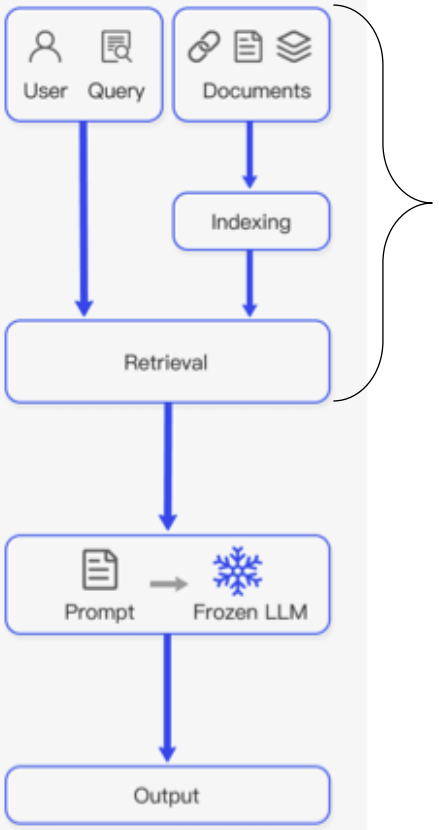
에이전트	품질관리	구조설계	전략 설계	• Reflection → Planning → Tool Use → Response • 이를 구체적으로 어떤 순서와 방식으로 조합할지는 설계자 의도 반영 • 구현 시 성능 편차가 발생하고, 재현성이 떨어질 수 있음
			루프 구조	• 반복적인 재질문, 재생성 흐름은 처리시간 증가 및 비용 상승 초래 • 특히 적절한 종료 조건을 설정하지 않으면, 무한 루프 또는 불필요한 검색/생성 프로세스가 진행될 수 있음
			Collaboration	• Multi-Agent 구조는 매력적이나, 실제 협업은 제한적임 • 각 에이전트가 독립적으로 작동해도 서로의 상태나 결과를 정교하게 해석하거나 피드백 하는 능력은 부족 • 토론, 설득, 반론 등과 같은 협력형 문제 해결까지는 제한적임
		Self-Evaluation		• Reflection, Critique 기능은 존재하지만 정확도, 신뢰도, 편향 여부를 에이전트 스스로 판단하는데 한계 • 내부적으로 판단한다고 해도, Fact-checking 기능이 내장되지는 않음 • 외부 피드백(HITL, 평가데이터 등)을 고려한 설계가 필요할 수 있음
		Graph Control		• 에이전트가 자동으로 판단하고 흐름을 바꾸기 때문에 예측 불가능성 증가 • Adaptive 구조일수록 사용자가 어떤 경로를 통해 응답이 생성되었는지 추적하기 어려움
	Memory			• 과거 경험은 반영하나, 지속적으로 유지하는 것은 어려움 • Reflection이 끝나도 그 학습이 장기 메모리에 반영되지 않으면 다음 작업에 전혀 영향을 미치지 못함 • 메모리 기반으로 구성하지 않을 경우, 단기적인 판단 반복만 수행

3. RAG 확장 전략

SUMMARY

SUMMARY

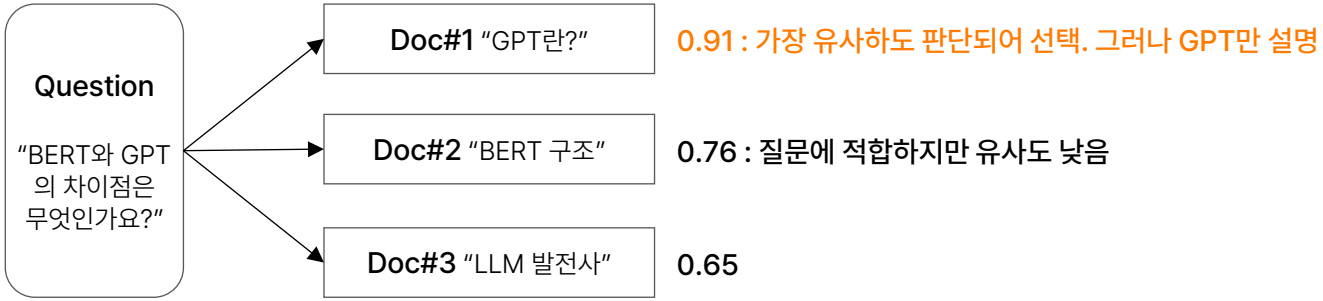
1. Naïve RAG 한계점



Query에 대한 얕은 이해 :

- 쿼리와 문서 청크 사이의 의미론적 유사성이 항상 일치하는 것은 아님
  - (eg) 질문 의도를 반영하지 못한 경우:
  - BERT와 GPT의 차이를 묻는 질문에 대해 임베딩 기반 검색 통해 GPT 설명만 포함된 문서가 높은 유사도를 받을 수 있음
  - 질문의 일부 주제와는 유사하지만, 질문 전체의 의미(비교)는 담고 있지 않음
    → 의미론적 유사성 ≠ 의도 적합성

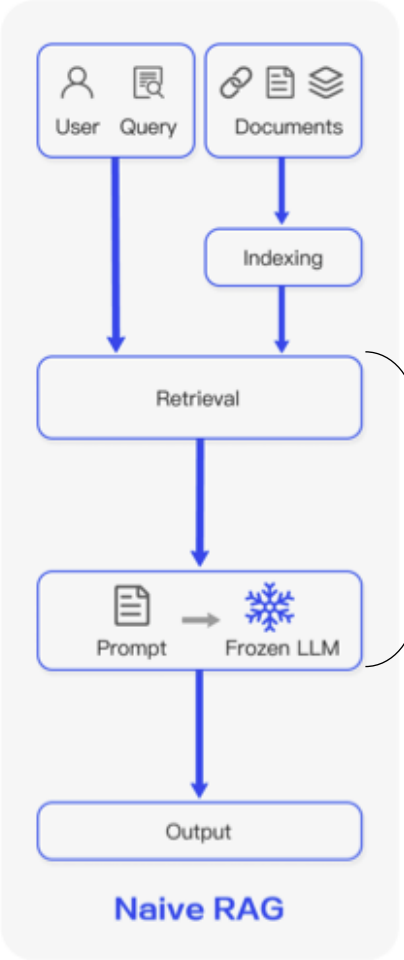
- 검색을 위해 유사도 계산에만 의존하는 것은 쿼리와 문서 간의 관계 파악에 부족



Source : https://arxiv.org/pdf/2312.10997

SUMMARY

1. Naïve RAG 한계점



정보 과잉이 부르는 **Hallucination :**

- 검색된 모든 청크를 LLM에 직접 공급하는 것이 항상 유익한 것은 아님
  - LLM은 맥락 안에서 중요한 내용을 구분해서 답을 생성해야 하는데,
  - 불필요한 정보까지 모두 전달되면 오히려 정답을 품질을 저해할 수 있음

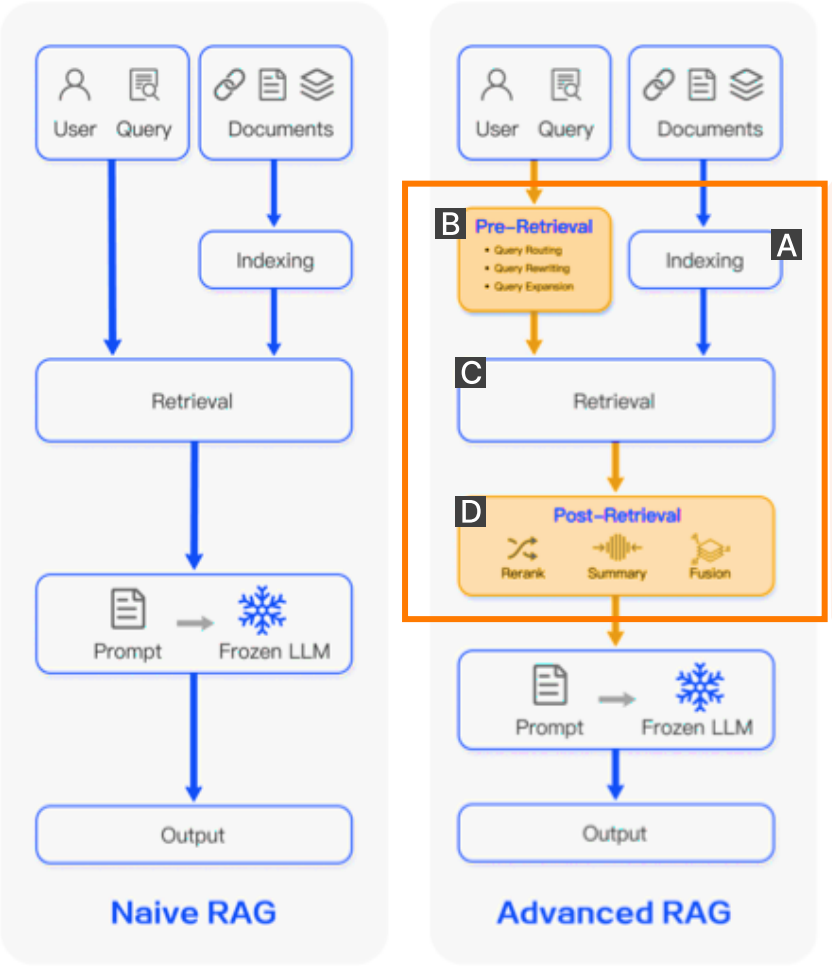
- 중복되고 노이즈가 많은 정보는 LLM이 핵심 정보를 식별하는데 방해가 됨
  - LLM이 정작 중요한 정보가 무엇인지 구별하지 못하여,
  - Hallucination 위험이 높아질 수 있음

Source : https://arxiv.org/pdf/2312.10997

SUMMARY

2. Advanced RAG

"검색 품질"을 개선하기 위한 단계별 핵심 전략



Retrieval 이전/이후 단계 정교화 :

A. Indexing
- 문서 데이터를 벡터 형태로 변환하여 검색에 활용할 수 있도록 준비하는 단계
- Hierarchical Structure
- Semantic Chunking

B. Pre-Retrieval
- 검색하기 전에 질문을 더 좋은 쿼리로 바꾸거나, 라우팅/확장 통해 검색 정확도를 높이는 단계
- Query Rewrite, Query Routing, Query Expansion
- Query Transformation

C. Retrieval
- 벡터 유사도 기반으로 문서를 검색하는 단계
- Hybrid Retrieval : Keyword + Semantic Search
- Multi-hop Retrieval : 여러 문서를 단계적으로 연결하여 복합 질문에 대응

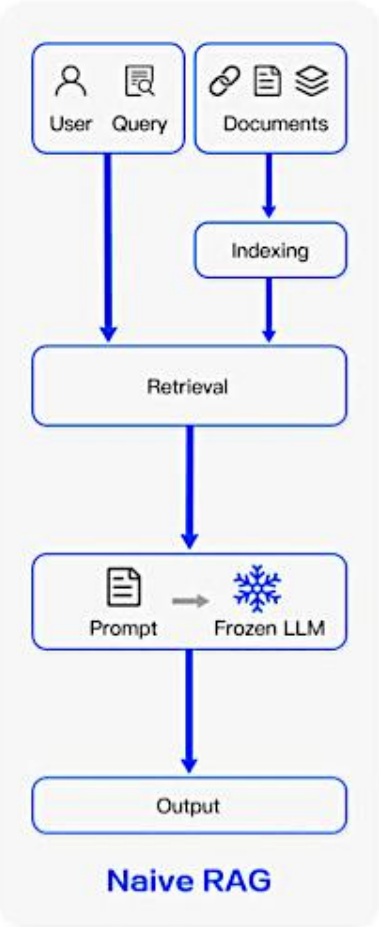
D. Post-Retrieval
- 검색된 결과를 정제하고 최적화하여 LLM 입력 품질을 높이는 단계
- Reranker, Reorder

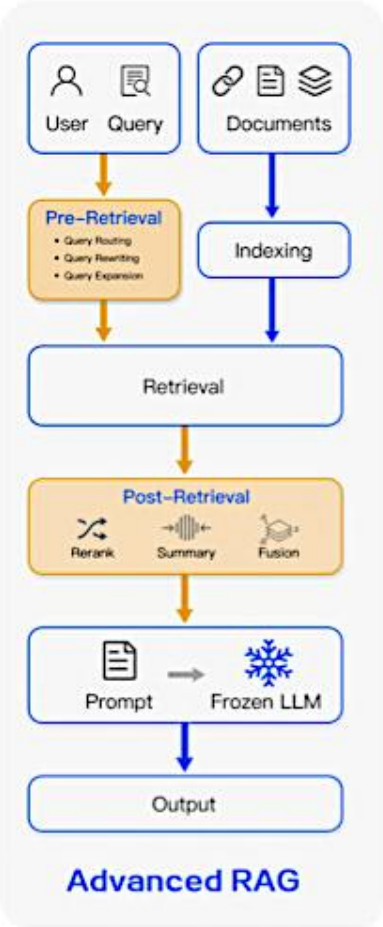
Source : https://arxiv.org/pdf/2312.10997

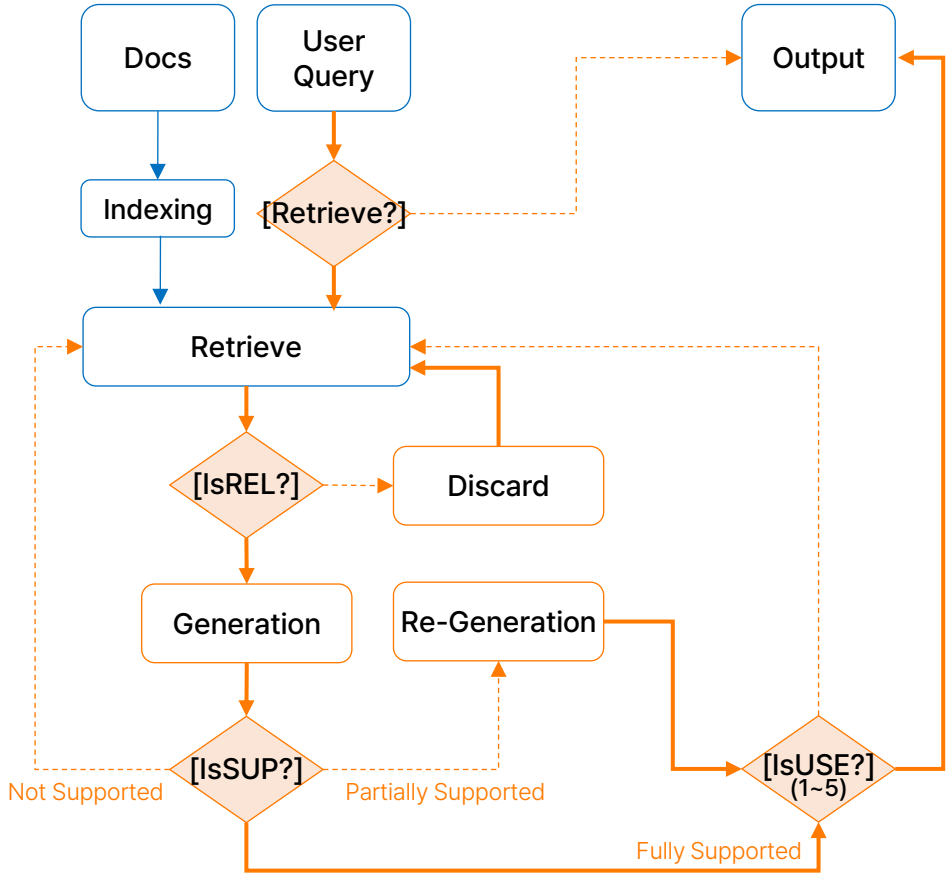
SUMMARY

3. Self-RAG

답변 생성 흐름에 인라인으로 삽입된 Reflection Token 통해 검색 필요성을 스스로 판단하고, 응답의 근거 충실도를 평가



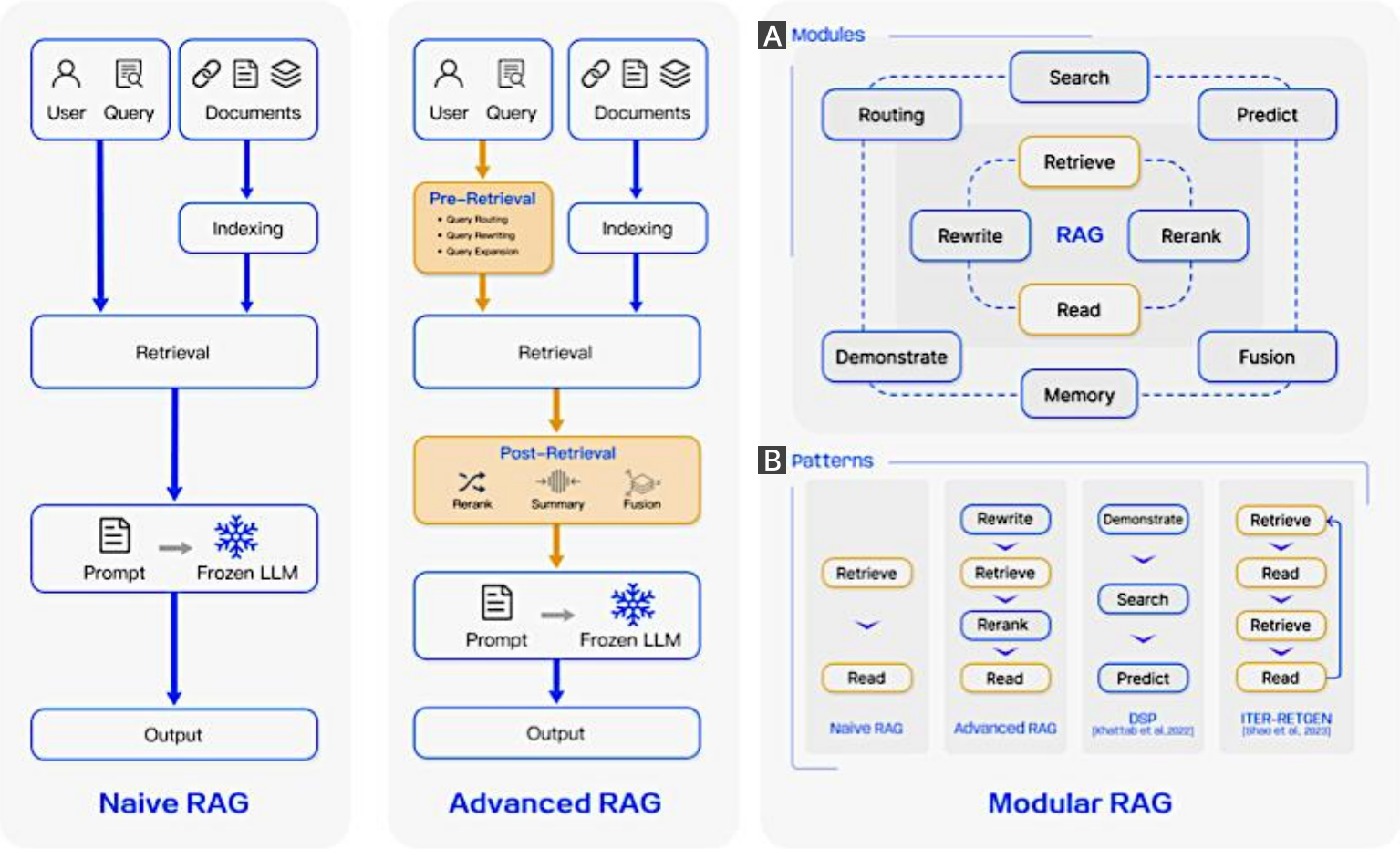




SUMMARY

4. Modular RAG

LEGO와 같이 재구성이 용이하고 보다 유연한 흐름을 만들 수 있는 프레임워크



- Independent
  : 기능 위주 세분화
- Flexible & Scalable
  : 모듈의 추가/변경 쉬움
- Dynamic Graph
  : 그래프 형식의 흐름 구성,
  상황에 따라 분기 처리.

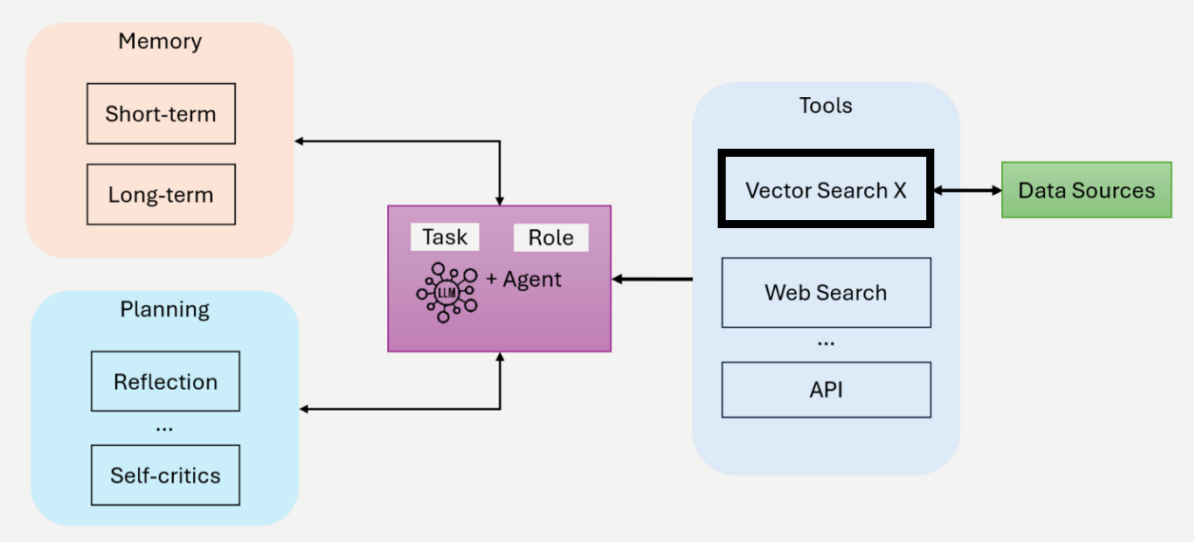
Source : https://arxiv.org/pdf/2312.10997

SUMMARY

5. Agentic RAG

AI Agent-based implementation of RAG

- Agent 통해 검색 전략을 스스로 설계하고, 도구를 선택하며, 응답을 생성하기 위한 최적 흐름을 능동적으로 제어하는 구조
- 기존의 검색 기반 구조에서 벗어나 "이 질문에 어떤 도구, 데이터가 적절한가?"를 에이전트가 판단하고 선택



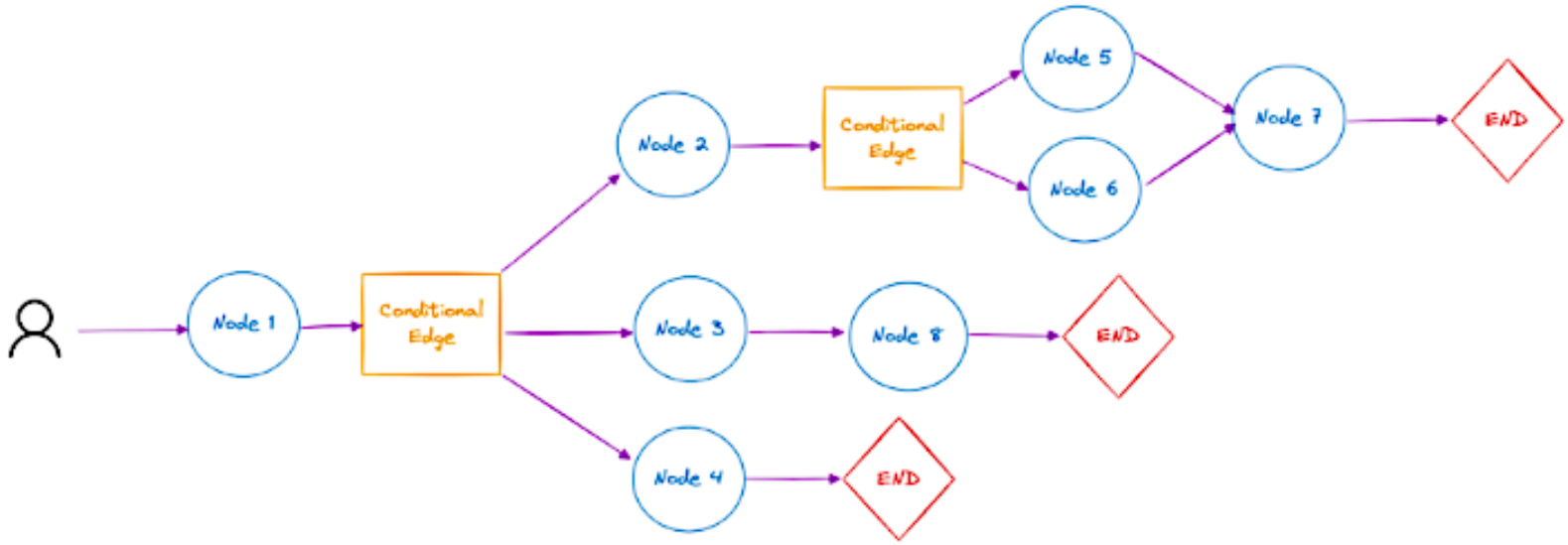
Source : https://arxiv.org/pdf/2501.09136

LangGraph

Quick Start

Agentic RAG 코드를 이해하기 위한 기본 구조만 우선 정리합니다.
상세 내용은 AI Agent 과정에서 다룹니다.

LangGraph 구성 요소



구성 요소	목적	역할
Node	어떤 작업(task)을 수행할지 정의	• 특정 로직 수행 또는 상태 업데이트(입력으로 현재 상태를 받고, 업데이트된 상태 반환)
Edge	다음으로 실행할 동작 정의	• 워크플로우의 흐름 제어 • 조건부 라우팅, 다음에 실행할 노드 결정
Conditional Edge	조건에 따른 분기 처리	
State	현재의 상태값을 저장 및 전달	• 전체 워크플로우의 컨텍스트 유지, 노드 간 정보 공유

Source : https://www.ionio.ai/blog/a-comprehensive-guide-about-langgraph-code-included

A. State

LangGraph 전체 흐름의 기억 저장소이며, 노드 간 연결성과 판단을 위한 핵심 데이터 구조

- 노드와 노드 간에 정보를 전달할 때 상태(State) 객체에 담아 전달 → 데이터 공유
- 에이전트가 현재까지 수행한 작업의 결과를 담고, 이후 판단에 기억처럼 사용 → Context 유지, 흐름 유지

LangGraph > State

What to Design:

- 어떤 값을 저장할지 정의 : 질문, 응답, 평가 등 단계별 필요한 항목
- TypedDict :
  - State 스키마를 정의하는 수단 → 구조 선언
  - Python dict에 Typehinting을 추가한 방식으로 명세
- Reducer : 자동으로 리스트에 메시지 추가
  - (활용) 여러 값을 하나로 합치는(fold) 연산
  - (코딩) 필드 업데이트를 어떻게 병합할지 정하는 함수 (fold 연산)
  - 이전 대화 내용에 새 메시지를 붙여서 하나의 대화 흐름으로 만들어 줌
- State 업데이트 방식
  - Reducer 미적용 (기본) : Overwrite
  - Reducer 적용 : 병합/누적
    (메시지: `add_messages`, 일반 리스트: `operator.add`)

Pitfalls :

- 노드 간 상태 공유를 위해서는 키 이름이 정확히 일치해야 함
- State에 너무 많은 내용을 저장하면 흐름이 복잡해지고 디버깅 어려움

```python
import operator
from typing import Annotated, TypedDict
from langgraph.graph.message import add_messages

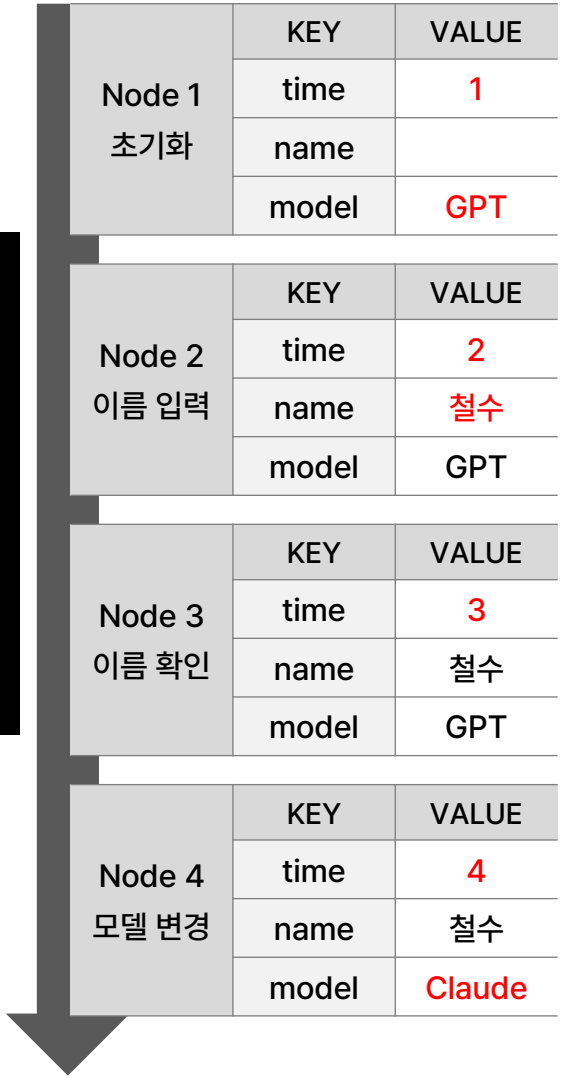

# State 정의
class GraphState(TypedDict):
    messages: Annotated[list, add_messages]  # 메시지 (누적)
    question: Annotated[list, operator.add]      # 질문 (누적 관리 필요 시)
    context: str                                 # 문서 검색 결과
    answer: str                                  # 답변
    relevance: str                               # 답변 관련성 (Good/Bad)
```

※ messages: Annotated[list, add_messages]
- messages : LLM과 주고 받는 대화 내용
- (left) list : 기본 메시지 리스트
- (right) add_messages : 병합할 메시지 리스트 또는 단일 메시지

A-(1). Overwrite

Reducer 없는 필드는 노드를 거치며 덮어쓰기로 정보 갱신

```
class GraphState(TypedDict):
    time: int
    name: str
    model: str
```



- 각 노드에서는 특정 KEY의 값을 새로 설정하며, 이는 기존 값을 덮어쓰기 함 (선택적 덮어쓰기)
- 필요한 Key만 갱신되고 나머지는 유지
- 위와 같은 방식으로 State가 관리되기 때문에 이전 정보도 유지하면서 새로운 작업을 추가할 수 있음 (필요한 부분만 갱신 !!)

A-(2). Reducer

Reducer 적용 필드는 정보 병학/누적 하여 관리

```
class GraphState(TypedDict):
    steps     : Annotated[list, operator.add]
    question  : str
    messages  : Annotated[list, add_messages]
```

Node 1 질문 입력

KEY	VALUE
steps	[“ask”]
question	Q1
messages	[Q1]

Node 2 검색

KEY	VALUE
steps	[“ask”, “search”]
question	Q1
messages	[Q1]

Node 3 답변

KEY	VALUE
steps	[“ask”, “search”, “answer”]
question	Q1
messages	[Q1, A1]

- 하나의 State 안에서도 필드 마다 업데이트 방식이 Reducer 적용 여부에 따라 다름
- `add_messages` : 메시지 전용
- `operator.add` : 범용 리스트 결합

  (e.g.)
  - `operator.add(3,5)` = 3+5
  - `operator.add([1, 2], [3]) = [1, 2, 3]

※ steps : Annotated[list, operator.add]
: steps 노드는 반드시 list를 반환해야 함 → 초기값 필요
단 `add_messages`는 빈 리스트에서 시작 가능

B. Node

LangGraph의 각 단계를 수행하는 작업 유닛 (함수 레벨)

- LangGraph 내에서 실제로 어떤 행동(작업)을 수행하는 단위로, 각 노드는 하나의 함수로 정의
- State를 입력 받아, 변경된 키만 부분 업데이트(dict)를 반환 → LangGraph가 State에 병합

LangGraph > Node

What to Design:

- 어떤 기능을 수행할지 함수로 정의 : (eg) 질문 분석, 검색, 답변 생성, 평가 등
- 노드는 state를 입력 받고, 변경된 키만 담은 dict 반환
  (State 전체를 반환하지 않음)
- `add_node` : 이전에 정의한 함수를 Graph에 추가
- 내부에서 LLM, API, DB 호출 등 다양한 작업 수행 가능

Pitfalls :

- 반환값은 부분 업데이트 dict : State 스키마에 정의된 키 이름과 정확히 일치해야 병합됨
- 리듀서가 없는 키는 덮어쓰기(last-write-wins), 있는 키는 누적/병합 처리
- 하나의 노드에 여러 책임(e.g., 검색+생성+분기)을 몰지 않음
  - 디버깅이 어려워지므로 기능은 단순하게
  - 단 라우팅 판단을 노드에서 내리는 것 자체는 정상 패턴

```python
# Node 정의 : 문서 검색
def retrieve_document( state: GraphState ) → dict :

    # Question에 대한 문서 검색을 retriever로 수행
    retrieved_docs = pdf_retriever.invoke(state["question"])

    # 검색된 문서를 context 키에 저장
    return {"context": format_docs(retrieved_docs)}
```

NODE A	
KEY	VALUE
context	
question	질문1
answer	
score	

NODE B	
KEY	VALUE
context	문서1
question	질문1
answer	
score	

C. Edge

Edge, Conditional Edge : 노드 간의 흐름을 정의하는 연결선

- Edge : A 다음에 B와 같이 순서를 연결하는 연결선
- Conditional Edge : 신호등을 보고 갈림길을 선택하는 로직과 같이 상황을 보고 다음 노드를 판단/분기

LangGraph > Edge

What to Design:

- Edge : 단순 연결 흐름 (`add_edge`)
- Conditional Edge : 조건 따라 다음 노드 분기 (`add_conditional_edges`)
- 조건 판단 함수 : 분기를 위해서는 State를 판별하는 함수 필요
  → 반환값 = 다음 노드 키
- 흐름 설계 : 평가, 피드백, 사용자 개입, 반복 등을 위한 설계 전략 및 반영

Pitfalls :

- 조건 판단 함수의 반환값이 매핑 딕셔너리의 키와 정확히 일치해야 연결됨
- Command 사용 시, 갈 수 있는 노드를 타입 힌트로 선언해야 그래프가 경로를 인식함 (`Command[Literal[...]]`)
- 반복/재귀 흐름은 exit 조건을 명확히 정의해야 함
  → 그렇지 않으면, 무한루프 발생 → 토큰 낭비

```
from langgraph.graph import StateGraph, START, END

# 시작점 정의
workflow.add_edge(START, "retrieve")

# Node 연결 (순차 흐름)
workflow.add_edge("retrieve", "llm_answer")
workflow.add_edge("llm_answer", "relevance_check")

# Conditional Edge
workflow.add_conditional_edges(
    "relevance_check",
    is_relevant,
    {
        "grounded" : END,                  # 관련성이 있으면 종료
        "notGrounded" : "llm_answer",      # 관련성 없으면 다시 답변 생성
        "notSure" : "llm_answer",          # 모호하면 다시 답변 생성
    },
)
```

AI Campus 생성형AI

[부록] RAG 평가

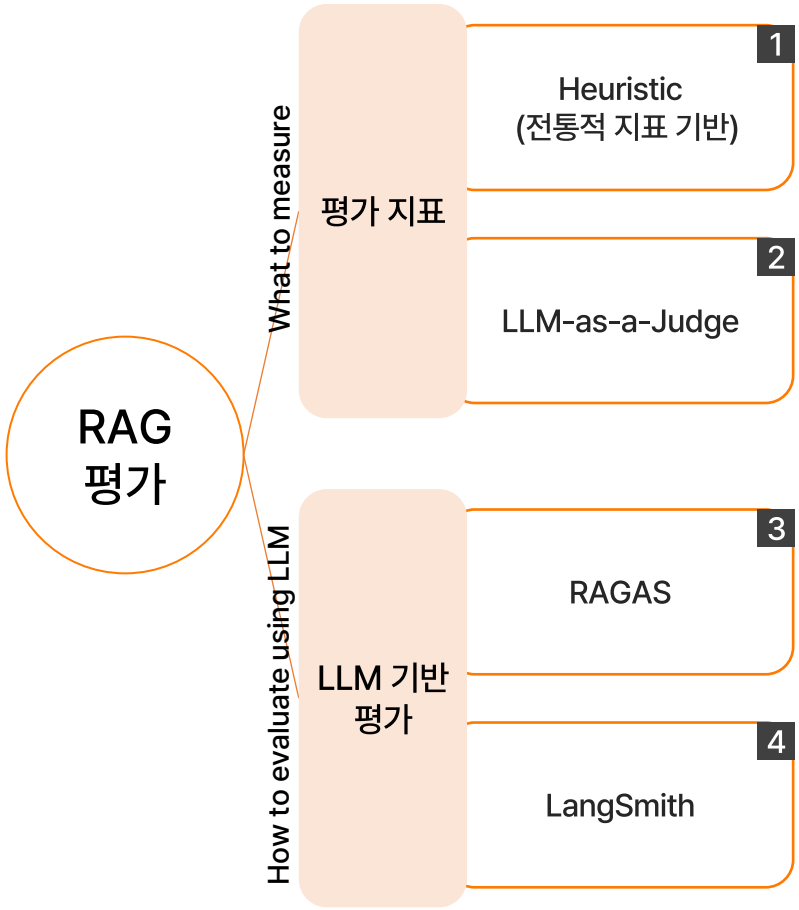
- 평가 방법
- 평가 지표 기반 : What to measure
- LLM 기반 평가 : How to evaluate using LLM

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

RAG 평가 방법

RAG 평가를 위한 다차원적 접근: Heuristic, LLM 기반 평가부터 RAGAS와 LangSmith API의 통합 활용까지



텍스트 유사도 기반 평가 (aka. 기계번역, Surface-level Similarity)

- Rouge : 단어 중복과 구문 유사도 평가
- BLEU : n-gram 기반 유사도
- METEOR : 동의어와 문법까지 포함한 유사도 평가

의미적 유사성 기반 평가

- SemScore : LLM이 문맥적 유사도 측정
- LLM-as-a-Judge : LLM 자체로 생성 답변 품질 평가
- G-Eval (평가 프롬프트 기반), AlginScore (사실성 기반 평가), RAGEval (RAG 증거 기반 평가)

RAGAS, **R**etrieval-**A**ugmented **G**eneration Assessment Score

- Test Dataset : Synthetic과 Manual Dataset 활용
- 평가 기준 : Context Recall, Context Precision, Faithfulness, Answer Relevancy, Conciseness, Fluency 등

LangSmith를 활용한 평가 (**LLM Application** Evaluator)

- Test Dataset 관리
- API 기반 평가 : Q-A, A-G, Summary 관련성, Hallucination 평가, 온라인 평가(사용자 평가) 등
- Trace 기반 분석, 사용자 피드백 반복 수렴 등 온라인 시스템 평가 지원

1. Heuristic 평가 지표

(1) Rouge, Recall-Oriented Understudy for Gisting Evaluation

- 원래는 기계 요약(Gisting) 품질 평가 위해 고안되었으나, 현재는 AI의 출력 품질 평가에 사용되는 지표
- 생성된 텍스트가 참조 텍스트와 얼마나 많은 n-gram(단어의 연속 묶음)을 공유하는지를 계산하여 유사도 측정

- 원본 : 오늘 아침 공원에서 조깅을 하다가 귀여운 강아지를 보았다.
- 생성 : 오늘 아침 공원에서 산책을 하던 중에 작고 귀여운 고양이를 우연히 만났다.

1. Rouge-1
   - 각 단어를 개별적으로 비교: Unigram
   - 일치 단어 : “오늘”, “아침”, “공원”, “귀여운”
   - 위 단어들이 양쪽 문장에 모두 나타나므로 점수에 반영

단어 일치도

---

2. Rouge-2
   - 연속된 두 단어씩 비교: Bigram
   - 일치 조합 : “오늘 아침”, “아침 공원에서”
   - 위 단어 조합들이 양쪽 문장에 모두 나타나므로 점수에 반영

작은 문맥 고려

---

3. Rouge-L

LCS (Longest Common Subsequence)

   - 순서를 유지하면서 가장 긴 공통 단어 조합을 찾아냄
   - 일치 조합 : “오늘 아침 공원에서 귀여운”
   - 위 단어 조합들이 순서를 유지하면서 양쪽 문장에 모두 나타나므로, 해당 시퀀스가 점수에 반영

전반적인 유사성

1. Heuristic 평가 지표

(2) BLEU, Bilingual Evaluation Understudy

- 생성된 텍스트가 참조 텍스트와 얼마나 유사한지 측정. 주로 기계 번역 평가에 사용됨
- 단어의 의미나 문맥은 고려하지 않으며, 정확히 일치하는 n-gram 수를 기반으로 유사도 계산

BLEU 평가

1. N-gram 정밀도 (precision)

- BLEU는 1-gram ~ 4-gram까지의 정밀도를 계산
- 각각의 정밀도는 생성된 텍스트에서 참조 텍스트와 일치하는 n-gram 비율

2. Brevity Penalty, 간결성 패널티

- 생성된 텍스트가 참조 텍스트 보다 짧을 경우 패널티 부과
- 시스템이 짧은 문장만 생성하여 높은 정밀도를 얻는 것을 방지
- 즉 정밀도만 높고 길이가 너무 짧은 경우 점수 보정

3. 기하평균

- 최종 BLEU Score는 각 n-gram precision의 기하평균에 간결성 패널티를 곱하여 계산됨

○ 원본 : 오늘 아침 공원에서 조깅을 하다가 귀여운 강아지를 보았다.
○ 생성 : 오늘 아침 공원에서 산책을 하던 중에 작고 귀여운 고양이를 우연히 만났다.

1. N-gram precision

구분	일치 단어	Precision
1-gram	오늘, 아침, 공원에서, 귀여운	4/10 = 0.4
2-gram	오늘 아침, 아침 공원에서	2/10 = 0.2
3-gram	N/A	0/10 = 0
4-gram	N/A	0/10 = 0

2. Brevity Penalty
   - 두 문장 길이 비슷
   - Penalty = 0

한 단어라도 일치하지 않으면,
n-gram precision=0이 되어 BLEU도 "0"
→ 의미 유사도 반영하지 않음
→ 단순 문자 비교 이상의 해석에는 한계 있음

3. 기하평균 $= (0.4 * 0.2 * 0 * 0)^{1/4} = 0$

1. Heuristic 평가 지표

(3) METEOR, Metric for Evaluation of Translation with Explicit ORdering

- BLEU와 같이 기계번역 품질을 추정하는 지표이지만, 의미 기반 단어 매칭과 재현율을 고려한 유연한 평가 지표
- METEOR는 정확도*precision*, 재현율*recall*, 단어 순서 및 어휘 다양성 등을 모두 고려

METEOR 평가

1. 단어 매칭 : 모든 가능한 매칭 고려

- 정확한 매칭 : 동일 단어 매칭
- 어간 매칭 : 같은 어근을 가진 단어 (eg, "run"과 "running")
- 동의어 매칭 : 의미가 같은 단어 (eg, "quick"과 "fast")
- 의역 매칭 : 구절 수준의 동의서 (주로 기계 번역에서 사용)

2. 정밀도와 재현율의 조화평균

- 정밀도, precision : 생성된 텍스트에서 참조 텍스트와 일치하는 단어 비율
- 재현율, recall : 참조 텍스트에서 생성된 텍스트와 일치하는 단어 비율

3. 순서 패널티

- 일치하는 단어들의 순서가 얼마나 다른지 고려
- 연속되지 않은 매칭에 대해 패널티 부과

○ 원본 : 오늘 아침 공원에서 조깅을 하다가 귀여운 강아지를 보았다.
○ 생성 : 오늘 아침 공원에서 산책을 하던 중에 작고 귀여운 고양이를 우연히 만났다.

1. 단어 매칭 (6)
   - 정확한 매칭 (4) : "오늘", "아침", "공원에서", "귀여운"
   - 동의어 매칭 (1) : ["조깅", "산책"]
   - 의역 매칭 (1) : ["강아지", "고양이"] ← 작은 동물
2. 조화평균
   - Precision $= \frac{\text{매칭 단어수}}{\text{생성문장의 단어수}} = \frac{6}{11} \approx 0.545$
   - Recall $= \frac{\text{매칭 단어수}}{\text{참조문장의 단어수}} = \frac{6}{8} = 0.75$
   - F-score $= \frac{2*P*R}{P+R} = \frac{10*0.545*0.75}{9*0.545+0.75} \approx 0.596$ ← Recall 중심의 F점수
3. 패널티 $= 0.5 * \frac{\text{매칭 그룹수}}{\text{매칭된 단어수}} = 0.5 * \frac{3}{6} = 0.25$

   문장 내에서 연속되지 않고 떨어져 있는 매칭 덩어리 개수
4. METEOR $= \boldsymbol{F * (1 - Penalty) = 0.596 * (1 - 0.25) = 0.447}$

→ F점수는 높지만, 순서가 불안정해서 METEOR 점수는 낮아짐

1. Heuristic 평가 지표

Heuristic 텍스트 생성 품질 평가 지표

구분	Heuristic		
	(1) ROUGE	(2) BLUE	(3) METEOR
특징	• N-gram 재현율 기반 (Rouge-N, Rouge-L) • 주요 단어(키워드) 포함 여부 평가 • 문장의 길이와 재현율 강조	• N-gram 기반 매칭 (단어 묶음) • 정밀도 중심 • 참조 문장과 생성 문장의 단어 순서를 중요하게 평가 • 길이에 따른 패널티 부과	• 다양한 관점에서 단어 매칭 기반 (정확한 일치, 어근, 동의어, 의미적 일치) • 정밀도와 재현율의 조화 평균 • 매칭 순서를 일부 고려
장점	• 재현율로 계산되어 요약 평가에서 강점	• 단순한 정량적 평가 가능	• 의미적 유사성 반영 (어근, 동의어 매핑) • 재현율을 강조하여 문장의 정보 손실을 줄임 • 매칭 그룹 기반 패널티로 문장의 연속성 평가 가능
단점	• BLEU처럼 의미적 유사성 반영 불가 • 정밀도에 대한 고려 부족 (특히 긴 문장에서 불리할 수 있음) • 단순 단어 매칭에 의존적	• 재현율을 고려하지 않아 중요 단어를 무시할 수 있음 • 단어 순서와 정확한 단어 일치에만 의존해 유연성 부족 • 의미적 유사성 반영 불가능	• 어근 및 동의어 매핑이 언어 종속적이라 구현에 제약 있음

2. LLM-as-a-Judge

(1) SemScore

- SemScore: Automated Evaluation of Instruction-Tuned LLMs based on Semantic Textual Similarity (2024)
- SemScore는 문장 임베딩 기반으로 cosine similarity 계산하여 의미적 유사도 평가
- 단순한 단어 매칭이 아닌, 문장 간의 의미적 일치 정보를 정량적으로 측정하며 기계 번역, 텍스트 요약, 문장 생성 품질 평가에 사용됨

Abstract

Instruction-tuned Large Language Models (LLMs) have recently showcased remarkable advancements in their ability to generate fitting responses to natural language instructions. However, many current works rely on manual evaluation to judge the quality of generated responses. Since such manual evaluation is time-consuming, it does not easily scale to the evaluation of multiple models and model variants. In this short paper, we propose a straightforward but remarkably effective evaluation metric called SEMSCORE, in which we directly compare model outputs to gold target responses using semantic textual similarity (STS). We conduct a comparative evaluation of the model outputs of 12 prominent instruction-tuned LLMs using 8 widely-used evaluation metrics for text generation. We find that our proposed SEMSCORE metric outperforms all other, in many cases more complex, evaluation metrics in terms of correlation to human evaluation. These findings indicate the utility of our proposed metric for the evaluation of instruction-tuned LLMs.

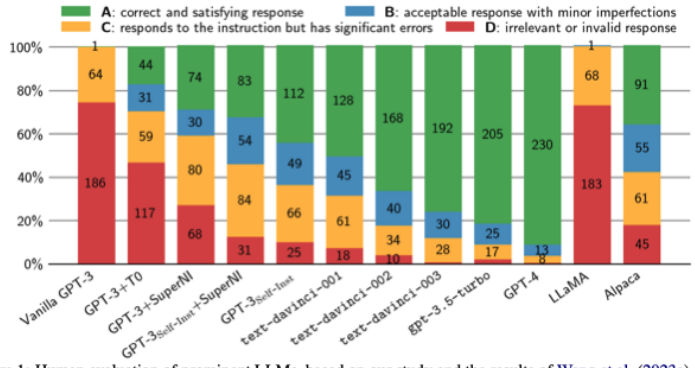


Figure 1: Human evaluation of prominent LLMs, based on our study and the results of Wang et al. (2023c). From this, we derive a human-judged ranking of LLMs as basis for comparison of automated evaluation metrics.

Model	Human	SEMSCORE	*G-Eval-4**	BERTScore	ROUGE-L	BARTScore	BARTScore$_{para}$	BLEU	BLEURT	DiscoScore
GPT-4	1	1		2	3	2	2	6	2	5
gpt-3.5-turbo	2	4	1	4	5	3	5	5	4	3
text-davinci-003	3	2	2	1	1	1	1	3	3	8
text-davinci-002	4	3	3	3	2	4	3	2	5	4
text-davinci-001	5	5	4	5	4	6	7	1	6	2
GPT-3$_{Self-Inst}$	6	7	5	7	6	7	4	7	8	9
Alpaca	7	6	11	6	8	5	8	4	9	1
GPT-3$_{Self-Inst}$+SuperNI	8	8	6	8	7	8	6	8	11	7
GPT-3+SuperNI	9	9	7	9	9	9	9	9	10	11
GPT-3+T0	10	10	8	10	10	12	12	10	12	6
Vanilla GPT-3	11	12	9	12	12	11	11	12	1	12
LLaMA	12	11	10	11	11	10	10	11	7	10

Table 2: Ranking of each model from best (rank 1) to worst (rank 12) according to each metric we consider. Since we use GPT-4 as backbone for G-Eval, we exclude GPT-4 outputs from its evaluation.

Human 평가 순위와 높은 상관관계

Source : https://arxiv.org/pdf/2401.17072

2. LLM-as-a-Judge

(1) SemScore

SemScore 특징

1. 의미적 텍스트 유사성 기반,
Semantic Textual Similarity (STS)

- 생성된 텍스트와 참조 텍스트 간의 의미적 유사성을 정량적으로 계산
- 단순한 단어 매칭을 넘어 문장의 전반적인 의미 고려
  (단순한 N-gram 일치 보다 더 풍부한 평가 가능)

2. 사전 훈련된 언어모델 활용

- BERT, RoBERTa 같은 사전 훈련된 언어 모델로 문장 임베딩 생성
- 문맥과 의미를 더 잘 포착할 수 있음
- 여러 개의 정답 참조를 고려할 수 있음

3. 평가

- Human Evaluation과 높은 상관관계 보임
- 전체 응답 뿐만 아니라 응답의 일부분 (eg, 문장 단위)에 대해서도 평가할 수 있음

- 원본 : 오늘 아침 공원에서 조깅을 하다가 귀여운 강아지를 보았다.
- 생성 : 오늘 아침 공원에서 산책을 하던 중에 작고 귀여운 고양이를 우연히 만났다.

1. Embedding
   - 참조 벡터(A) : [0.12, 0.45, -0.33, ...]
   - 생성 벡터(B) : [0.11, 0.47, -0.30, ...]

2. Cosine Similarity
   - 두 벡터 간의 방향적 유사성 측정
   - 두 벡터의 방향이 얼마나 비슷한지를 수치로 표현 (1에 가까울수록 유사)
   - Cosine Similarity $= \frac{A*B}{\|A\|*\|B\|} = 0.89$

3. 결과 해석
   - SemScore = 0.89
   - ["조깅"-"산책"], ["강아지"-"고양이"]와 같은 의미적 유사성 반영

2. LLM-as-a-Judge

(2) LLM 기반 직접 평가 방식

LLM-as-a-Judge 특징

1. 프롬프트 기반 직접 평가

- LLM에게 직접 평가 역할 부여 (e.g. "이 답변은 얼마나 정확한가요?")
- 응답 텍스트에 대해 정확성, 충실성, 관련성, 유창성 등 다양한 기점으로 채점

2. 평가 프롬프트 구성

- System Prompt: 평가자 역할 부여 (ex. "Your are a helpful and accurate evaluator.)
- User Prompt : 질문 + 참고 문서 + 응답 제공 → 평가 기준 따라 판단 요청
- 출력 형태 : 점수, 선택지, 설명 등

3. 평가 특성

- 정량/정성 평가 모두 가능 (평가 사유 포함)
- 사람 평가와 높은 상관성
- 의미 유사성, 논리적 일관성, 사실성 등 복합적 기준 반영 가능

- 질문 : 오늘 아침 공원에서 무슨 일이 있었나요?
- 문서 : 오늘 아침 공원에서 조깅을 하다가 귀여운 강아지를 보았다.
- 응답 : 산책 중에 귀여운 고양이를 봤다.

[평가 프롬프트]

Given the question, context, and response, rate the response on a scale from 1 to 5 for relevance and factual consistency. Explain your reasoning.

[LLM 평가 결과 (예시)]

- Score : 3/5
- Explains : "The response is somewhat related to the context, but it contains factual inconsistency ('dog' vs 'cat') and changes the activity ('jogging' vs 'walking')."

※ 주의사항

항목	내용
일관성 부족	같은 평가 프롬프트에 대해 LLM이 다르게 평가 할 수 있음
평가 기준 명확성 부족	평가 프롬프트가 모호하면 LLM이 일관되게 평가하지 못함
모델 편향	학습 데이터의 편향 등으로 점수에 영향 줄 수 있음
과잉 일반화 또는 **관대함**	정답에 가까운 문장을 그렇게 평가하지 않거나, 너무 관대한 채점을 할 가능성 잔존
'자기 모델 평가' 문제	GPT 답변 평가 시 GPT가 우수한 결과를 주는 현상

2. LLM-as-a-Judge

(2) LLM 기반 직접 평가 방식 : 프롬프트 표준화의 목적

- AMBIGUITY 제거 : 애매하거나 열린 질문이 LLM의 평가 범위 및 평가를 키우므로 명확하고 정형화된 프롬프트 구조 필요
- G-Eval 기반 평가 : CoT + Form-filling 방식으로 평가하도록 LLM 유도

```
Task: Rate the assistant's answer based on how correct and relevant it is.

Criteria:
• Correctness (1–5) – Does it align factually with the context?
• Relevance (1–5) – Does it directly address the question?

Steps:
1. Read the question and context.
2. Compare the answer with the context.
3. Assign a relevance and correctness score (1–5).

Question: {question}
Context: {context}
Answer: {answer}

Provide:
• Correctness score:
• Relevance score:
• Explanation:
```

프롬프트 구성 시 주의사항

- 역할 설정 : System prompt에 "You are an expert evaluator …" 등으로 평가자 역할 명시
- **명확한 기준** : 정밀히 나눈 평가 항목 (정확성, 충실성, 일관성, 유창성 등) 제시

항목	내용
정확성, Correctness	응답이 사실과 일치하는지 평가, 사실적 오류가 없는지 확인
충실성, Faithfulness	응답이 참고 문서 정보에 기반하고 있는지 평가, Hallucination 발생 시 감점
관련성, Relevance	응답이 질문에 직접적으로 답변하고 있는지 평가, 문맥이나 질문과 상관없는 내용이 포함되면 감점
일관성, Consistency	응답 내 논리적 흐름이 자연스럽고 앞뒤가 맞는지 평가
명확성, Clarity	문장이 이해하기 쉽게 명확히 표현되었는지 평가
유창성, Fluency	문장이 자연스럽고 매끄럽게 연결되는지 평가 (표현, 단어, 말투 등)
간결성, Conciseness	불필요한 반복이나 장황한 설명 없이 간단 명료하게 핵심을 전달하는지 평가

- 출력 양식 통제 : 점수, 설명을 포함하여 Form-filling 구조 권장
- 반복 & 교차 평가 : 동일 프롬프트로 반복 수행 또는 Multi-LLM 통한 교차 평가를 수행한 결과를 비교하여 편차 최소화

Source : https://www.comet.com/site/blog/g-eval-for-llm-evaluation

1+2. RAG 평가 지표 기반

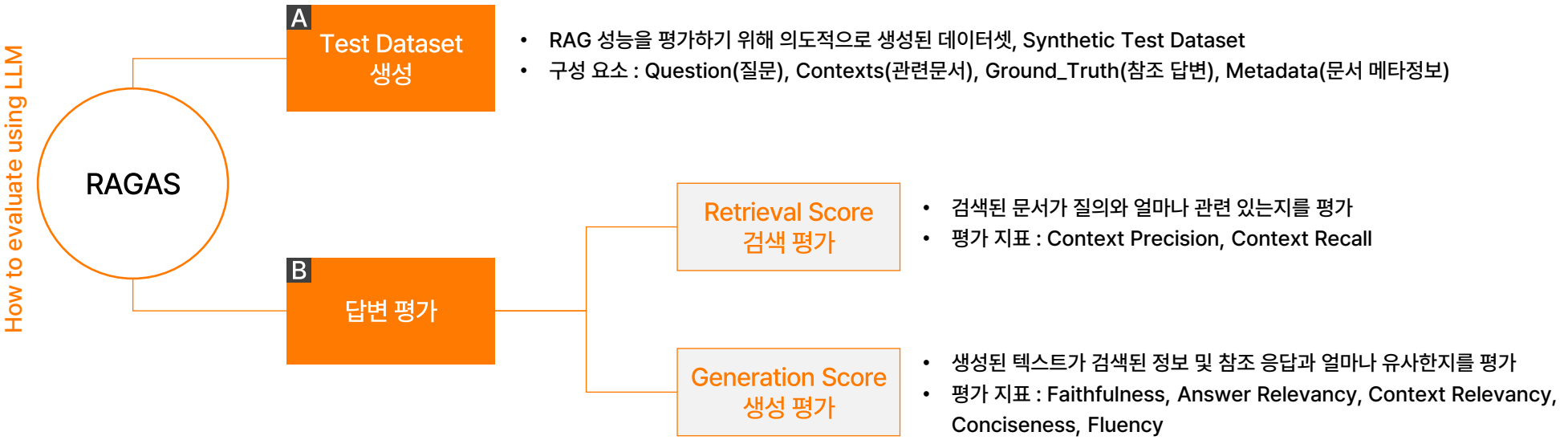
텍스트 생성 평가 지표는 Heuristic에서 LLM 기반으로 진화하고 있고, 평가 목적에 따라 적절한 지표를 선택하는 것이 중요

구분	Heuristic			LLM-based	
	ROUGE	BLUE	METEOR	SemScore	LLM-as-a-Judge
특징	• N-gram 재현율 기반 (Rouge-N, Rouge-L) • 주요 단어(키워드) 포함 여부 평가	• N-gram 기반 정밀도 중심 • 평가단어 순서, 길이 고려	• 다양한 관점에서 단어 매칭 기반 (어근, 동의어, 의미적 유사성) • 정밀도와 재현율의 조화 평균	• 의미적 텍스트 유사성(STS) 기반 • 임베딩 유사도 계산	• LLM이 직접 평가자(Judge) 역할 수행 • 프롬프트 기준 따라 점수 평가
장점	• 재현율로 계산되어 요약 평가에서 강점	• 단순하고 빠른 정량 평가	• 의미적 유사성 반영	• 문맥과 의미를 포함한 유사도 평가 • HUMAN 평가와 높은 상관성	• 복잡 기준 반영 가능 (정확성, 충실성 등) • 정량+정상 평가 가능
단점	• 단어 일치 기반 • 의미 고려 부족	• 의미 유사성 반영 불가 • 짧은 문장은 과대 평가 될 수 있음	• 언어 자원 의존 • 구현 복잡성	• 벡터 유사도 기반으로 점수 해석이 직관적이지 않음 • 모델 품질에 의존적	• 평가 결과의 일관성/ 신뢰도 확보 어려움 • 모델 편향, 자기 평가 문제 가능
주요 사례	• 문사 요약 평가 • 핵심어 평가	• 기계 번역 정확도 평가	• 문장 유사성 평가 • 기계 번역 적합성 평가	• Instruction-tuned LLM 비교 평가	• RAG 응답 평가 • AI Q&A 평가

3. RAGAS

Retrieval-Augmented Generation Assessment Score

- RAGAS는 검색 기반 생성 모델(RAG)의 성능을 평가하기 위한 지표
- 답변 평가는 정보 검색(Retrieval)과 텍스트 생성(Generation) 과정에서 각각의 성능을 평가하고 이를 통합하여 최종 점수 제공



3-A. RAGAS, Test Dataset 생성

평가 품질 확보 위해 RAGAS의 합성 테스트 데이터(Synthetic Test Data)을 생성하여 진행함

```
from ragas.testset.generator import TestsetGenerator
from ragas.testset.evolutions import simple, reasoning, multi_context, conditional

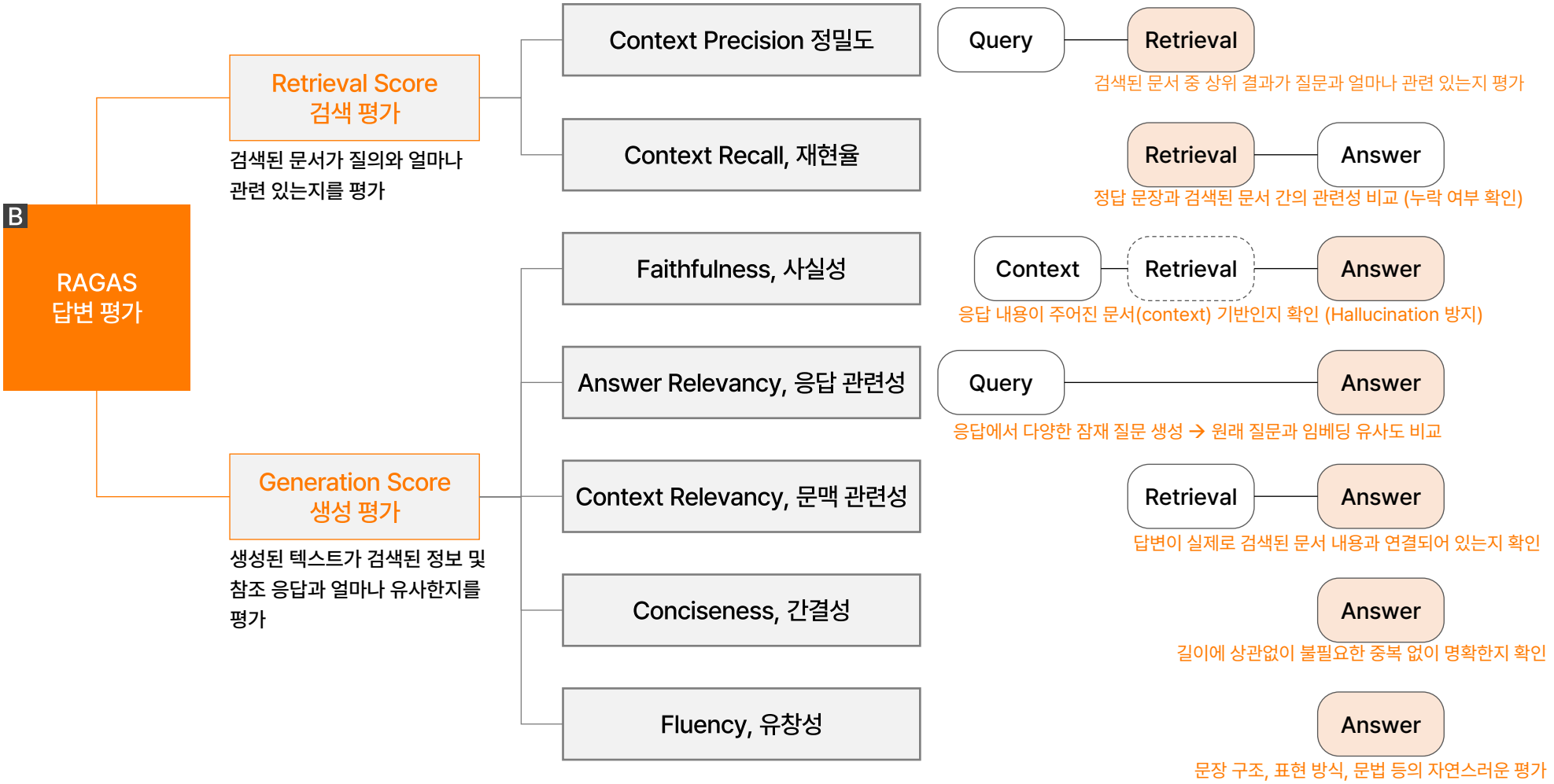
generator = TestsetGenerator.from_langchain(
    generator_llm,          # 데이터셋 생성기
    critic_llm,             # generator_llm 평가 위한 생성기
    ragas_embeddings,
    doc_store               # InMemory DB
)

# simple : 간단한 질문, reasoning: 추론이 필요한 질문, multi_context: 여러 맥락 고려한 질문, conditional: 조건부 질문
distributions = {simple: 0.4, reasoning: 0.2, multi_context: 0.2, conditional: 0.2}

# 테스트셋 생성 (질문 10개)
test_set = generator.generate_with_langchain_docs(
    documents=docs,
    test_size=10,
    distributions=distributions,
)
```

3-B. RAGAS, 답변 평가

정보 검색(Retrieval)과 텍스트 생성(Generation) 과정에서 각각의 성능을 평가하고 이를 통합하여 최종 점수를 제공함



[참고] RAGAS (예시)

정확성, 유사성, 표현력까지 다각도로 평가한 평균 점수

- 질문 : 인공지능이 의료 분야에 미치는 영향은 무엇인가요?
- RAG :
  1. Retrieval
     - 문서1 : AI를 활용한 질병 진단 정확도 개선
     - 문서2 : 의료 이미지 분석에서의 AI 역할
     - 문서3 : AI의 법적 책임 논의
     - 문서4 : 의료 산업의 데이터 보안
     - 문서5 : AI기반 약물 개발
  2. Generation
     - AI는 의료 분야에서 질병 진단의 정확도를 높이고, 의료 이미지 분석에서 중요한 역할을 하고 있습니다. 또한 약물 개발에서도 사용됩니다.

- RAGAS Score : 0.90
  (weighted average)

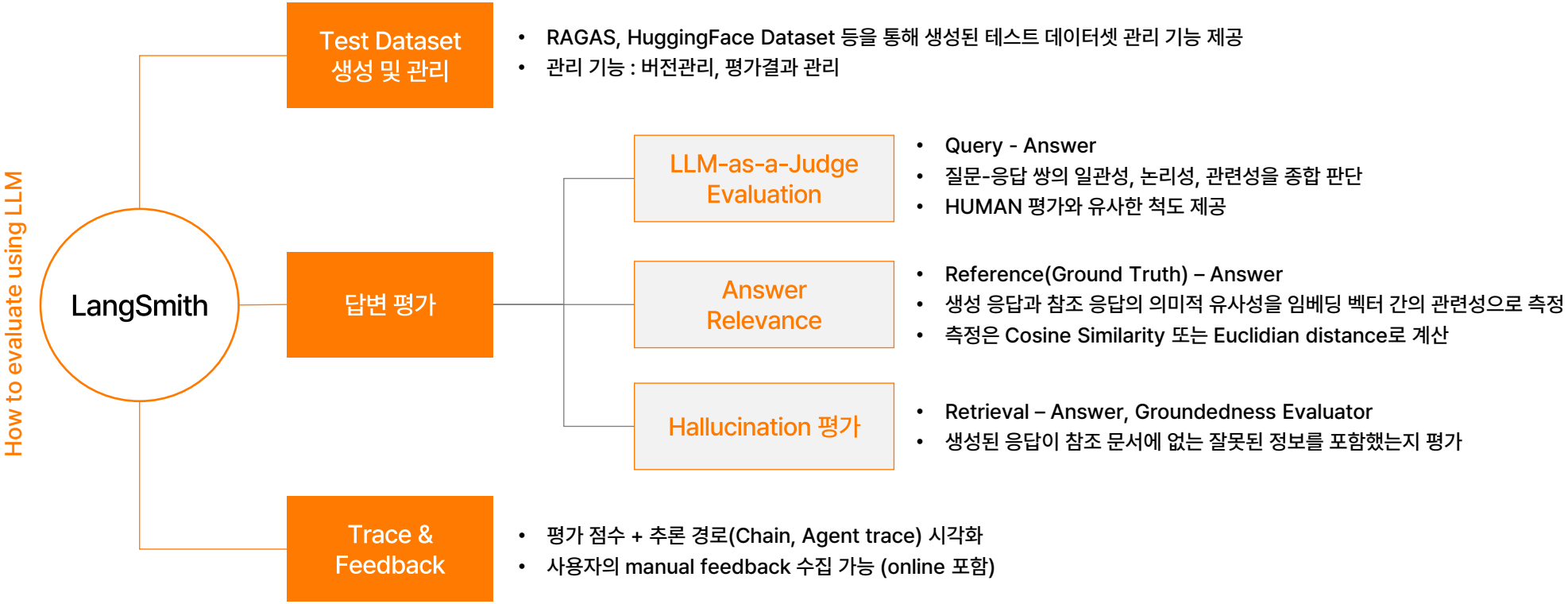
※ 가중치 : 각 항목 동등하게 산정한 예시 가중치 적용

	항목	Score	내용
Retrieval Score	Context Precision	$\frac{\text{관련 문서수}}{\text{검색된 문서수}} = \frac{3}{5} = 0.6$	검색된 5개 문서 중 질문과 질적 관련 문서는 3개(문서 1, 2, 5)
	Context Recall	$\frac{\text{정답과 일치하는 관련 문맥수}}{\text{전체 관련 문맥수}} = \frac{3}{3} = 1.0$	질문에 답변하기 위해 모든 문서는 검색됨 (문서 1, 2, 5)
Generation Score	Faithfulness	$\frac{\text{문맥에서 추론 가능한 주장 개수}}{\text{전체 주장 개수}} = \frac{3}{3} = 1.0$	응답 내용에서 제시한 세 가지 주장: “질병 진단”, “의료 이미지 분석”, “약물 개발” 모두 문서(1, 2, 5)에 의해 뒷받침됨
	Answer Relevancy	$\frac{1}{N}\sum_{i=1}^{N} \cos(E_{g_i}, E_o) = 0.92$	생성된 답변과 질문 간의 의미적 유사성을 코사인 유사도로 계산
	Context Relevancy	응답 문장 ↔ 참조 문서 간 cosine similarity (mean) = 0.85	응답 내용이 Retrieval 문서(1, 2, 5)에 기반하여 작성되었으며, 문서 내용과의 연결성 확보됨
	Conciseness	핵심 정보 외 불필요 정보 여부 평가 = 0.95	길이 대비 핵심 정보 밀도 높고 불필요 반복 없음
	Fluency	LLM 통한 평가 = 1.0	표현이 매끄럽고 문법적 오류 없음

4. LangSmith

LangSmith 통해 LLM 기반 자동 평가와 사용자 피드백을 결합하여 RAG 응답 품질을 체계적으로 측정 및 관리

- LangSmith는 RAG 평가 전과정을 관리하며, 다양한 자동화 지표와 사용자 피드백을 통합 관리 할 수 있는 도구
- 자동화된 평가 메트릭을 활용하여 정확성, 관련성, 사실성 등을 체계적으로 측정하고 관리할 수 있음





수고하셨습니다.

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX